



Universidad Nacional de la Matanza

Programación Inicial

UNIDAD 1. INTRODUCCION A LA PROGRAMACION

INDICE

1.	INTRODUCCIÓN.....	2
2.	ESQUEMA BÁSICO DE UNA COMPUTADORA - MODELO DE VON NEUMANN.....	2
3.	EVOLUCIÓN DE LAS COMPUTADORAS	3
4.	CARACTERÍSTICAS GENERALES	4
5.	¿QUÉ ES UNA COMPUTADORA Y QUE PROBLEMAS PUEDE RESOLVER?	5
6.	PROGRAMACIÓN	6
6.1	¿QUÉ ES PROGRAMAR?	6
6.2	LENGUAJES DE PROGRAMACIÓN.....	6
6.3	COMPILADORES E INTÉRPRETES.....	7
7.	ETAPAS PARA SOLUCIÓN DE PROBLEMAS CON LA COMPUTADORA	7
8.	CONSTRUCCIÓN DE ALGORITMOS	8
9.	PROGRAMACIÓN ESTRUCTURADA	8

UNIDAD 1

INTRODUCCION A LA PROGRAMACION

OBJETIVOS: *Informar al Alumno del mundo Informático, la computadora, sus características y evolución. Exponer el ámbito de la programación con distintos tipos de programas y lenguajes. Transmitir los pasos para encarar un problema y como diseñar una posible solución al mismo.*

1. Introducción

Antes de comenzar se introducirá de forma sencilla algunos conceptos:

- **Computadora:** es un equipo electrónico capaz de realizar muchas operaciones muy rápidamente, con la particularidad de que no está diseñada para hacer una sola tarea sino que es capaz de realizar tareas diferentes dándole las instrucciones de cómo hacerlo mediante un programa.
- **Programar:** es dar instrucciones a una computadora, decirle que es lo que queremos que haga.
- **Informática:** Las computadoras manejan datos y esos datos al procesarlos se transforman en información. El manejo de dicha información mediante un programa de computadora da origen a la informática haciendo que los datos e información puedan ser procesados en forma automática.
- **Digitalizar:** Hacer que los datos sean adaptados para ser procesados por un sistema informático.
- **Hardware:** Componentes físicos de la computadora.
- **Software:** Parte no tangible que se ejecuta en los equipos informáticos. Son los programas y/o aplicaciones.

Usted se encuentra en la era de la información. Cada vez más y más procesos cotidianos se encuentran digitalizados y los datos son manipulados por sistemas computacionales. Por ejemplo, al ir a un supermercado los productos son leídos con un lector de códigos de barra y procesados mediante un sistema que recupera su precio y va calculando el gasto realizado. También al subir a un colectivo y pasar la tarjeta SUBE, hay detrás un proceso informático que descuenta de nuestro saldo el viaje realizado. Estos son solo dos pequeños ejemplos, pero si piensa en cuantas cosas de la vida cotidiana dependen de un sistema informático se va a dar cuenta de que cada vez son más y más.

Las computadoras están presentes en la vida cotidiana y se presentan de muchas formas y tamaños: computadoras de escritorio, notebooks, tablets, teléfonos celulares, entre otros. Es decir, está rodeado de máquinas capaces de ejecutar distintos programas que permiten realizar tareas de una forma fácil y rápida.

2. Esquema Básico de una Computadora - Modelo de Von Neumann

John Von Neumann fue un matemático Húngaro nacido en 1903, que diseñó una arquitectura general que define como deber estar organizada una máquina de propósito general (computadora).

Esta arquitectura fue tomada en cuenta a la hora de diseñar las primeras computadoras y sigue siendo la base fundamental de las computadoras actuales.

El diseño realizado por Von Neumann plantea la arquitectura en tres bloques principales:

- **CPU:** sigla que viene del inglés, Central Processing Unit (Unidad Central de Procesamiento), es la encargada de interpretar y ejecutar las instrucciones de los programas que se quieren ejecutar.
- **Memoria Principal:** Es un espacio donde se almacenan tanto los programas que se quieren ejecutar como los datos que dichos programas manipulan.
- **Unidad de Entradas y Salidas:** Es el bloque encargado de la comunicación con el mundo exterior, y por lo tanto, con los usuarios. Por ejemplo, para mostrar información por una pantalla o recibir los datos ingresados por un mouse o teclado.

La figura 1 muestra los bloques principales de la arquitectura de Von Neumann.

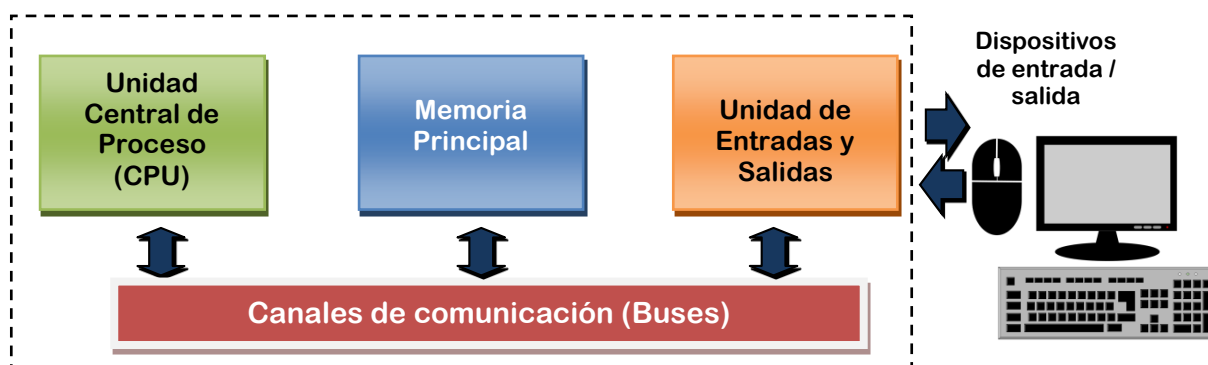


Figura 1: Modelo de Von Neumann

Este es un esquema general, ya que dentro de cada bloque hay varios componentes que trabajan en conjunto, para lograr que la computadora pueda ejecutar instrucciones y comunicarse con el usuario mediante dispositivos de entrada y salida como mouse, teclado, monitor, impresora, etc.

La encargada de ejecutar las instrucciones es la CPU, donde internamente incluye lo que se conoce como la unidad aritmético lógica, que es capaz de realizar operaciones matemáticas y lógicas para poder procesar datos. Esas instrucciones se dan a la computadora mediante un programa que es almacenado en la memoria principal, y de allí se lleva a la CPU cada una de las instrucciones, una a una para ejecutarlas. En este esquema la memoria principal también guarda los datos con los que el programa trabaja y los resultados obtenidos. Esta memoria se divide en distintos “casilleros” donde cada uno puede guardar una cierta cantidad de información. Para acceder a cada casillero se utiliza una “dirección” diferente, por lo tanto, siempre que se recupera un dato o instrucción de la memoria se dice que accede a una dirección de memoria particular.

El bloque de entradas y salidas es un intermediario, que permite realizar la conexión con los dispositivos externos para permitir ingresar datos, o mostrar información. Este bloque también adecúa velocidades ya que los dispositivos externos son mucho más lentos que los componentes internos de la computadora.

Los bloques se comunican entre sí mediante distintos canales de comunicación llamados buses, donde por ellos pueden viajar direcciones de memoria, datos y señales de control.

3. Evolución de las computadoras

Las computadoras de hoy en día se presentan en una amplia variedad de formas, tamaños y precios. Las grandes computadoras de propósito general se utilizan en muchos negocios, universidades, hospitales y agencias gubernamentales para desarrollar sofisticados cálculos científicos y financieros.

Se conoce a estas grandes computadoras o grandes sistemas como mainframes, que necesitan de equipamiento y mobiliario especial, en cuanto a espacio, electricidad, humedad, etc. También existen otras computadoras más chicas en tamaño, como las computadoras personales que con la incorporación de las redes, permiten lograr recursos importantes y a un menor costo.

A comienzo de los cincuenta se disponía de grandes sistemas, aunque muy poca gente sabía cómo utilizarlos, lo hacían generalmente científicos, ingenieros y analistas financieros.

Durante los sesenta, fue cada vez más frecuente que aprendieran a programar grandes sistemas los estudiantes en las universidades, esto produjo la aparición de jóvenes profesionales provocando la desaparición del temor y recelo existentes hasta el momento, con la utilización de los grandes equipos.

A finales de los sesenta y comienzos de los setenta, se desarrollaron mini computadoras, más pequeñas y menos caras, lo que permitió el acceso a las mismas de comerciantes más pequeños y en los hogares.

Hasta llegar a nuestra época, donde podríamos decir que es una herramienta imprescindible para cualquier profesional y comerciante para que pueda realizar sus actividades con eficiencia.

La evolución de las computadoras es el resultado de la innovación de los componentes electrónicos, donde con cada avance tecnológico nació una nueva generación de computadoras que mejoraban tanto en tamaño (haciéndose más pequeñas) como en velocidad (haciéndose más rápidas). Según la tecnología se pueden identificar 4 generaciones de computadoras:

- 1ra generación válvula de vacío
- 2da generación Transistor
- 3ra generación Circuito Integrado
- 4ta generación Microprocesador

Hoy en día si bien no hay un cambio radical en la tecnología, se avanza en la incorporación de varios núcleos de procesamiento en un mismo microprocesador, lo que permite ejecutar distintas instrucciones al mismo tiempo.

4. Características generales

Todas las computadoras digitales, independientemente de su tamaño, son básicamente dispositivos electrónicos que pueden transmitir, almacenar y manipular información (datos). Una computadora puede procesar distintos tipos de datos. Esto incluye datos numéricos, alfanuméricos (nombres, direcciones, etc.), datos gráficos (mapas, dibujos, fotografías, etc.), y sonido (música, lectura de textos, etc.). Desde el punto de vista del programador recién iniciado, los dos tipos de datos más familiares son los números y los caracteres, involucrando frecuentemente las aplicaciones de negocios el tratamiento de estos tipos de datos.

Para que la computadora procese un lote particular de datos es necesario darle un conjunto apropiado de instrucciones llamado programa. Estas instrucciones se introducen en la computadora y se almacenan en una parte de la memoria de la máquina.

Un programa almacenado se puede ejecutar en cualquier momento. La ejecución de un programa supone lo siguiente:

1. Un conjunto de datos, los datos de entrada, se introduce en la computadora (desde un teclado o un archivo), y se almacena en una porción de memoria de ésta.
2. Los datos de entrada se “procesarán” para producir ciertos resultados deseados, que son los datos de salida.

3. Los datos de salida, y probablemente algunos de los datos de entrada, se imprimirán en papel o se presentarán en un monitor.

Este procedimiento de tres pasos se puede repetir tantas veces como se desee. En cualquier caso, se debe tener presente que estos pasos, especialmente el 2 y el 3, pueden ser largos y complicados.

Ejemplo. Una computadora ha sido programada para calcular el área de un círculo utilizando la fórmula:

Área = $\pi \times r^2$, dando el valor numérico del radio como dato de entrada, es necesario dar los siguientes pasos para efectuar el cálculo:

1. Leer el valor numérico del radio del círculo.
2. Calcular el valor del área utilizando la fórmula anterior. Este valor se almacenará entre los datos de entrada en la memoria de la computadora.
3. Imprimir (mostrar en pantalla) los valores del radio y el área correspondiente.
4. Parar.

Cada uno de estos pasos requiere de una instrucción o más en un programa.

Lo anteriormente dicho ilustra dos características importantes de una computadora digital: memoria y capacidad de ser programada. Otras características importantes son su velocidad y fiabilidad.

5. ¿Qué es una computadora y que problemas puede resolver?

Para decidir cuáles y cómo se pueden resolver los problemas recordemos cómo funciona una computadora digital, comencemos por dar una definición:

“Una computadora es un sistema electrónico dedicado al **procesamiento de datos** (información), formado por varias unidades, cuyo funcionamiento viene dictado por el **programa** que se halla almacenado en su memoria principal”

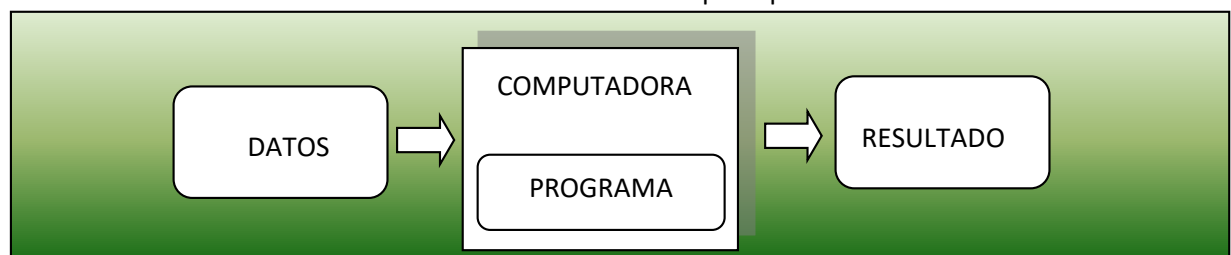


Figura 2: Resolución de problemas mediante una computadora

La computadora es un mecanismo complejo que solo opera sobre datos, resolviendo operaciones muy sencillas (sumas y comparaciones) a una elevadísima velocidad y con gran precisión.

Como la tarea es la resolución de problemas utilizando una computadora, se debe expresar una secuencia de operaciones lógicas, en un lenguaje suficientemente básico y preciso, en instrucciones que la computadora sepa resolver, éstas serán las instrucciones que componen a cada lenguaje de programación.

Básicamente, la computadora puede realizar los siguientes tipos de operaciones:

- Ingresar datos. Pueden ser varios y de diferente tipo – numéricos y no numéricos.
- Realizar cualquier tipo de operación algebraica y/o lógica, efectuando comparaciones y generando ciclos definidos o condicionados.
- Almacenar información en forma transitoria (memoria principal), o permanente (memorias auxiliares)
- Exhibir datos y/o resultados en diferentes dispositivos como ser en una pantalla, una impresora, etc.

6. Programación

6.1 ¿Qué es programar?

Programar es la actividad de comunicar a la computadora lo que se tiene que hacer. Cualquiera que desee que la computadora haga algunas cosas, deberá acoplarse a este tipo de comunicación persona-computadora, utilizando un lenguaje concreto. Una manera de apreciar lo que es un programa, es considerarlo como si fuera un plan detallado para transformar información, aceptando datos y produciendo una nueva información. A las reglas de transformación se las denomina algoritmos.

El problema que se establece al escribir un programa es básicamente el de la comunicación entre un ser humano y una computadora. Probablemente el ser humano sabe cómo encarar el problema, o al menos sabe cómo resolver algunos problemas afines, pero si el problema se va a resolver con la ayuda de una computadora, los humanos deben transmitirle su propósito.

Los lenguajes humanos no son apropiados para la comunicación con las computadoras, principalmente porque acarrearán una notable serie de ambigüedades. De persona a persona la comunicación puede tolerar esta ambigüedad debido a acuerdos tácitos, y a que la comunicación no verbal está también presente. Pero en la comunicación con una máquina todo se lleva a cabo de forma literal, de modo que se necesita ser muy preciso y específico en describir la labor que se tenga que realizar. De ahí que la gente haya elaborado lenguajes especializados, denominados lenguajes de programación, para la comunicación entre la persona y la máquina.

6.2 Lenguajes de programación

Se necesita un sistema para decirle a la computadora lo que se quiere hacer. Los lenguajes de programación permiten realizar dicha tarea. Se trata de un lenguaje especial que no incluye ambigüedades. Estos lenguajes deben tener descripciones precisas de las tareas que se deben realizar.

Las computadoras actuales trabajan en sistema binario, es decir, con dos niveles de corriente que se representa con 0 y 1. Para un programador sería muy difícil escribir instrucciones de un programa como una secuencia de ceros y unos, por lo tanto, surgen los lenguajes de programación.

Cada CPU, cuyo componente físico en una computadora personal es el microprocesador, puede interpretar y ejecutar distintas instrucciones. Hay conjuntos de instrucciones estandarizadas que fueron evolucionando a lo largo del tiempo, por lo tanto, los microprocesadores en general soportan la ejecución de varios set de instrucciones diferentes según su arquitectura.

El lenguaje de programación más cercano a la computadora es el Assembler, donde cada instrucción del lenguaje de programación se corresponde directamente con una instrucción que el microprocesador es capaz de ejecutar. Este lenguaje está muy ligado al hardware ya que solo funcionará con procesadores que entiendan dichas instrucciones. A este tipo de lenguajes se los conoce como lenguajes de bajo nivel por su cercanía al hardware.

Programar en Assembler no es una tarea fácil, ya que tareas sencillas muchas veces requieren de varias instrucciones para lograr realizarlas, como por ejemplo, una multiplicación se hará sumando sucesivamente un número, y además, el programa realizado va a funcionar en un hardware particular. Por este motivo, surgen lenguajes de programación más avanzados, donde una sola instrucción del lenguaje se traduce luego en varias instrucciones interpretables por el procesador haciendo la tarea del programador más sencilla. Además, estos lenguajes avanzados se independizan del hardware ya que el mismo programa puede ser traducido para ser ejecutado en distintos procesadores con distintos set de instrucciones. Por eso se los llama lenguajes de Alto Nivel.

Todo lenguaje de programación está compuesto por:

- **Léxico:** que símbolos del lenguaje puedo utilizar para escribir mi programa. Letras, números y algunos símbolos conforman el léxico de la mayoría de los lenguajes.
- **Sintaxis:** son las reglas que indican de qué forma se combinan los símbolos para lograr escribir instrucciones válidas. La sintaxis también define palabras que están reservadas por el lenguaje y son necesarias para la escritura de las instrucciones. Por ejemplo, la palabra `if`, es reconocida en muchos lenguajes como el comienzo de una condición lógica, por lo tanto, esa palabra no puede utilizarse para otro fin.
- **Semántica:** cada instrucción válida del lenguaje de programación tiene un propósito particular. Indica algo que se quiere que la computadora haga. Ese sentido de cada instrucción es la semántica. La semántica, por lo tanto, es el significado de las instrucciones de un programa.

Un programa de computadora es básicamente una colección secuencial de “sentencias” siguiendo cada una de ellas las reglas sintácticas del lenguaje. Cada sentencia contiene información para la computadora. Usted puede considerar estas sentencias declarativas como si fueran absorbidas por la computadora una a una en un orden secuencial; entonces la computadora “ejecuta” cada sentencia, hace lo que la sentencia le pide que haga.

Sentencia 1	(primera en ejecutarse)
Sentencia 2	(segunda en ejecutarse)
---	-----
Sentencia n	(enésima en ejecutarse)

6.3 Compiladores e intérpretes

El programa en el lenguaje en que se escribe las instrucciones se llama programa fuente. Este programa fuente es un archivo de texto que la computadora no puede comprender directamente, y por lo tanto, necesita que sea traducido al lenguaje máquina.

Hay dos mecanismos diferentes para realizar esa traducción, mediante un compilador o mediante un intérprete. El compilador toma el programa fuente, analiza si está bien escrito según la sintaxis del lenguaje de programación elegido y genera un programa en un lenguaje que la computadora puede interpretar llamado programa objeto. Ejemplos de lenguajes compilados son C, C++, Java, Fortran. En cambio, un intérprete va tomando una a una las instrucciones del programa fuente, las traduce y las ejecuta, es decir, no analiza todo el programa sino que va traduciendo y ejecutando las instrucciones a medida que lee el programa fuente. El problema de los lenguajes interpretados es que al no analizar el programa completo, puede ser que se comience a ejecutar un programa hasta que se llegue a una instrucción que no pueda interpretarse por lo que se terminaría abruptamente la ejecución del programa. Un ejemplo de un lenguaje interpretado es javascript, utilizado para ejecutar instrucciones en una página web y agregarle funcionalidad.

7. Etapas para solución de problemas con la computadora

La solución de problemas mediante la computadora consta de ocho etapas, articuladas de tal forma que cada una depende de las anteriores, lo cual indica que se trata de un proceso complementario y, por lo tanto, cada paso exige el mismo cuidado en su elaboración. Las ocho etapas son:

1. **Comprensión, definición y delimitación del problema a solucionar.**
Para resolver el problema es fundamental conocerlo y delimitarlo por completo, determinar con qué datos se cuenta, la información a obtener. Es importante tener claridad sobre el problema a resolver para no dar soluciones incorrectas.
2. **Estrategia - definir los algoritmos.**

Disponer de los algoritmos necesarios para resolver los problemas. Identificar en forma manual como resolvería sin la computadora el problema propuesto, realizando todos los pasos necesarios en una hoja entendiendo bien el proceso.

3. Diagrama de flujo.

Es la primera etapa a realizar luego de tener en claro el problema y su manera de resolver. En esta etapa, es donde se determinan los pasos e instrucciones que deben llevarse a cabo y el orden lógico de su ejecución, para una eficiente solución al problema representado en forma gráfica con los símbolos de diagramas propuestos. Se puede decir que es aquí donde radica toda la dificultad para solucionar un problema a realizar.

4. Prueba de escritorio.

Consiste en hacer un seguimiento manual, a través, de un lote de prueba que permita verificar los pasos que realiza el diagrama desarrollado, esto permitirá reconocer a través del resultado si funciona correctamente o no.

5. Codificación.

Es la escritura de las instrucciones, determinadas en la etapa de la diagramación, en un lenguaje de programación determinado.

6. Compilación o interpretación del programa.

En esta etapa, la computadora chequea si todas las instrucciones están escritas correctamente en el código utilizado, respetando, simbología y sintaxis.

7. Ejecución del Programa.

Luego de la etapa anterior, si el compilador no dio errores, el programa objeto es ejecutado, para llegar a los resultados esperados.

8. Evaluación de los resultados.

Obtenidos los resultados, se los evalúa para verificar si son correctos. En caso contrario, se revisa las etapas anteriores para detectar la falla o error.

8. Construcción de Algoritmos

De las etapas mencionadas, ésta es la que traerá las mayores dificultades por lo cual será la principal del trabajo a realizar. Se debe destacar que el concepto de "algoritmo" es similar al de "estrategia", pero con la diferencia que en lugar de hablar de "procesos o tareas", se hablará de "instrucciones", lo cual involucra un mayor grado de detalle, donde estarán presentes operaciones minuciosas y precisas.

El algoritmo se lo puede describir en forma gráfica mediante un diagrama, y luego, codificarlo en un lenguaje de programación particular generando un programa.

La utilización de diagramas para pensar y diseñar los algoritmos antes de codificarlos tiene las siguientes ventajas:

1. permite una sencilla y rápida visualización total del problema y sus distintas alternativas.
2. es un verdadero medio de comunicación entre quien explica y quien aprende.
3. es un medio claro y conciso de documentación de los problemas.
4. facilita los posibles cambios y visualiza distintas alternativas posibles.

Existen varios métodos para encarar la construcción del algoritmo. A estas diferentes formas de encarar la construcción de una solución a un problema mediante la computadora, se las conoce como paradigmas de programación. Entre los principales paradigmas se puede mencionar: programación estructurada, orientada a objetos, funcional, lógica, etc. Un buen punto de partida para comenzar a comprender y dominar el diseño de algoritmos es utilizar el enfoque de la programación estructurada, ya que nos dará la base necesaria para poder luego utilizar otros paradigmas diferentes.

9. Programación estructurada

Este método tiene como soporte teórico al teorema fundamental de la programación estructurada, (Jacopini y Bohn) que afirma que todo algoritmo puede ser construido en forma correcta utilizando "únicamente" tres estructuras básicas:

- **SECUENCIAL:** las instrucciones del programa se ejecutan una a continuación de la otra, en forma ordenada, y siguiendo la secuencia que el programa indica.
- **SELECCIÓN:** permite tomar decisiones en el programa, abriendo distintos caminos haciendo que se ejecuten determinadas instrucciones solo si se cumple alguna condición particular. De esta forma el programa puede tener distintas variantes de ejecución y adaptarse a los datos que se están procesando.
- **ITERACION:** esta estructura permite repetir un conjunto de instrucciones, ya sea una cantidad de veces fija o dependiendo de alguna condición. Al utilizar instrucciones de iteración los programas quedan más cortos, y se pueden modificar más fácilmente que si se escribieran varias veces las mismas instrucciones.

También la programación estructurada indica que todo programa o módulo de un programa debe tener un comienzo y un único punto de finalización. Esto ayuda a la comprensión y diseño de los algoritmos haciendo que el proceso se ejecute en forma secuencial hasta llegar al final de mismo, existiendo un único punto donde el programa finaliza. Más adelante se verá que hay lenguajes que soportan que existan múltiples puntos de finalización de un programa. En este tipo de lenguajes para seguir la programación estructurada se recomendará unificar las salidas a un único punto para simplificar la lógica.

Una vez comprendidas las estructuras básicas de la programación estructuradas se verán distintas herramientas y estructuras de datos que permiten realizar programas mas complejos de forma más reducida y óptima.



Universidad Nacional de la Matanza

Programación Inicial

UNIDAD 2. ESTRUCTURA SECUENCIAL

INDICE

1.	ESTRUCTURA SECUENCIAL	2
2.	ALMACENAMIENTO DE LOS DATOS	2
2.1	VARIABLE:	2
2.2	TIPOS DE DATOS:	2
2.3	IDENTIFICADORES:	3
2.4	CONSTANTE:	3
3.	REPRESENTACIÓN GRÁFICA DE UN ALGORITMO - DIAGRAMA	4
3.1	INGRESO DE DATOS	4
3.2	OPERACIONES	5
3.2.1	OPERACIÓN DE ASIGNACIÓN	5
3.2.2	OPERACIONES MATEMÁTICAS	5
3.2.3	SALIDA DE RESULTADOS / MENSAJES	7
3.2.4	SÍMBOLOS DE INDICACIÓN	7
4.	ALGUNOS EJEMPLOS	8
	EJEMPLO 1	8
	EJEMPLO 2	8
	EJEMPLO 3	9

UNIDAD 2

Estructura Secuencial

OBJETIVOS: *Construir algoritmos utilizando la programación estructurada. Resolver sencillos ejemplos con “estructura secuencial” representándolos gráficamente mediante diagrama de flujo.*

1. Estructura Secuencial

La primera estructura básica de la programación estructurada es la secuencial. Consiste en una sucesión de instrucciones que se ejecutan en el mismo orden en que fueron escritas, una a continuación de la otra.

Un algoritmo secuencial se expresa como una sucesión de instrucciones que se ejecutan TODAS, una a continuación de la otra, teniendo un único punto de inicio y un único punto de fin. Esas instrucciones pueden ser de distintos tipos:

- Operaciones aritméticas
- Operaciones de asignación
- Ingreso de datos
- Salida / Mostrar información

2. Almacenamiento de los datos

Antes de poder escribir las instrucciones que forman parte de los algoritmos, debe saber cómo se guardan los datos internamente en la computadora, ya que las instrucciones serán operaciones que se realizarán sobre dichos datos.

Cada fragmento de información almacenado en la memoria de la computadora es codificado como una combinación de ceros y unos. Estos ceros y unos se llaman bits (dígitos binarios). Un dispositivo electrónico representa cada BIT, el cual estará de alguna forma: “apagado” (cero) o “encendido” (uno).

A un grupo de 8 bit, se lo denomina byte. Normalmente, un carácter (una letra, un dígito, un carácter especial) ocupará un byte de memoria.

2.1 Variable:

Una variable es un espacio de memoria reservado para almacenar un valor que corresponde a un tipo de dato soportado por el lenguaje de programación. Una variable, es representada y usada a través de una etiqueta (un nombre) que le asigna un programador, o que ya viene predefinida.

Por ejemplo, se define una variable con el nombre NUM, en la cual se almacena el número 8, NUM es la etiqueta que le asigna el programador y ocho es su contenido. Lo que se carga en NUM puede ser cualquier número, 1, 2, 3, 0, 12, 33, etc., (variable). Una variable, por lo general, como su nombre lo indica, puede variar su valor durante la ejecución del programa.

2.2 Tipos de Datos:

Los tipos de datos indican que es lo que se puede almacenar en una variable. Por ejemplo, un número entero, un carácter, un número con decimales, etc.

Cada tipo de dato ocupa en la memoria de la computadora una cantidad distinta de bytes. Por lo tanto, para optimizar un programa, por ejemplo, si se necesita trabajar con números pequeños, se puede utilizar una variable con un tipo de dato que permita almacenar números pero hasta cierta cantidad haciendo que esa variable ocupe menos espacio en memoria.

Dependiendo del lenguaje de programación usado, también es posible cambiar el tipo de dato que almacena una variable a lo largo del programa, pero en el lenguaje C, que es el que se verá en esta materia, el tipo de dato no puede modificarse, por lo que debe definirse la variable correctamente según lo que se necesite almacenar en ella.

Los tipos de dato que se utilizarán para los algoritmos son los siguientes:

- **Caracter:** para almacenar un único caracter, una letra, un símbolo, un dígito numérico.
- **Entero:** para almacenar un número entero (sin decimales).
- **Real:** para almacenar un número con decimales.

2.3 Identificadores:

Los nombres de variables o identificadores deben ser nemotécnicos, es decir, que con solo leer el nombre de la variable se pueda entender o determinar con facilidad lo que ella significa o contiene. En el ejemplo anterior la variable con el nombre NUM almacena un número por lo que se ve a simple vista, pero si a esta variable se le hubiese dado el nombre X o DS, estos nombres pueden significar muchas cosas o, tal vez, no significar absolutamente nada haciendo que lectura del programa sea más compleja. Los identificadores tienen ciertas reglas de escritura que van a depender del lenguaje de programación que se esté utilizando. Como regla general, un identificador tiene que cumplir con las siguientes reglas:

- No puede comenzar con números.
- No puede tener espacios.
- No puede tener acentos ni letra ñ.
- No puede tener símbolos (excepto el guión bajo que es el único permitido).

A continuación se muestra la tabla 1 con ejemplos de identificadores:

Tabla 1: ejemplo de identificadores

Identificador	¿es correcto?
Num	CORRECTO
Numero_1	CORRECTO
_Numero1	CORRECTO
Num1	CORRECTO
Número	INCORRECTO
1Numero	INCORRECTO
Num 1	INCORRECTO
año	INCORRECTO

Algunos lenguajes son sensibles a los cambios entre mayúsculas y minúsculas (case-sensitive), y otros no. En aquellos que son sensibles, por ejemplo num1 y Num1, hacen referencia a dos identificadores diferentes por lo que debe respetarse siempre el mismo nombre cada vez que se lo utiliza.

2.4 Constante:

Constante son todos aquellos valores que no cambian en el transcurso de un algoritmo y son introducidos en el momento de utilizarse.

Por ejemplo, el número 10 es una constante del tipo entera que puede asignarse a una variable de dicho tipo. En cambio, si se quiere representar una constante del tipo caracter se expresa entre comillas simples, por ejemplo, 'A' , '1' , 'a' , '-' son constantes de tipo caracter.

3. Representación gráfica de un algoritmo - Diagrama

Para comenzar a diseñar los algoritmos, y ayudar a encontrar una solución al problema planteado, se va a utilizar una representación gráfica de las instrucciones que se le darán a la computadora, de esta forma se podrá visualizar más fácilmente el algoritmo y una vez terminado y probado se puede codificar en el lenguaje de programación elegido.

Todo diagrama tendrá un símbolo de inicio, una serie de operaciones y un símbolo que indica el fin del mismo. Recuerde que inicialmente realizará algoritmos en forma secuencial, por lo tanto, los símbolos que utilizará son los que están en la Figura 1.

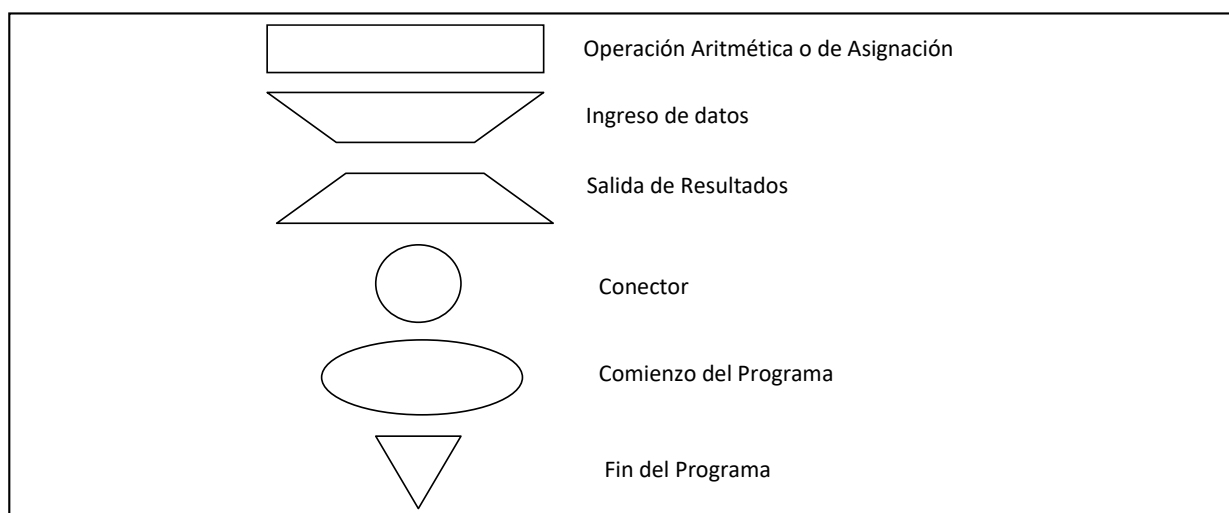


Figura 1. Símbolos para la construcción de diagramas que representan algoritmos secuenciales.

3.1 Ingreso de datos

Para resolver un problema, se necesitan datos, esos datos serán procesados para alcanzar el resultado esperado.

Los algoritmos, en general, se diseñan para que puedan funcionar con distintos datos según la necesidad del usuario que esté utilizando el programa. Por ejemplo, poca utilidad tendrá un programa que calcule una suma de números pre-establecidos: si se suma $5 + 2$, siempre el resultado será 7, por lo tanto, hacer un programa con esos números fijos (constantes) no tiene mucha utilidad. En cambio si esos dos números a sumar pueden ingresarse y variar cada vez que se ejecute el programa, se podrá utilizar para sumar $5 + 2$, $9 + 4$, $25458 + 24577$, o para cualquier par de números que se desee.

Analice el problema a resolver: Se necesita realizar un programa que permita sumar dos números cualquiera. Una forma de darle la información a la computadora es ingresar ese número por teclado, ¿pero una vez ingresado donde queda ese número? Bien, se necesita entonces un lugar para guardar cada uno de los números que se ingresan en el programa, y como vimos anteriormente, los datos se guardan en la memoria de la computadora en espacios reservados que se llaman variables.

El programa va a necesitar entonces dos variables para los datos de entrada, una variable para guardar el primer número a sumar, y la otra para guardar el segundo número a sumar:

- En la variable que se llama Num1 se va a guardar el primer número

- En la variable que se llama Num2 se va a guardar el segundo número

En el algoritmo cuando se quieran ingresar uno o más datos se utilizará el símbolo de ingreso, y dentro del símbolo se pondrá el nombre de la o las variables donde se guardará la información a ingresar. Si se ingresa más de un dato, las variables se pondrán separadas por coma. La figura 2 muestra la presentación gráfica para permitir el ingreso de datos desde teclado y que se almacenen los datos ingresados en las variables Num1 y Num2.

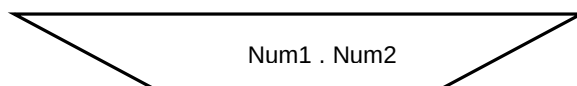


Figura 2. Simbología para el ingreso de dos datos que se almacenarán en las variables Num1 y Num2

Num1 y Num2 son dos variables distintas. Num1 va a contener el primer dato que el usuario ingrese por teclado, y Num2 el segundo. Antes de ingresar los datos en dichas variables no se sabe que hay, por lo tanto, no se pueden utilizar si antes no se leyó un dato o se les asignó un valor (proceso que se verá a continuación).

3.2 Operaciones

Sobre los datos ingresados se realizan operaciones para obtener el resultado deseado. Se pueden realizar operaciones matemáticas y de asignación.

3.2.1 Operación de Asignación

La asignación consiste en guardar un dato en una variable. El dato puede ser:

- Otra variable
- Una constante
- El resultado de una operación matemática

La asignación se realiza de derecha a izquierda, es decir, que a la izquierda se escribe el identificador de la variable en la cual desea guardar el dato, luego se escribe el símbolo = (igual) que indica la operación de asignación, y a la derecha se coloca la constante, variable u operación matemática. Gráficamente la operación de asignación se representa con un rectángulo. En la Figura 3 pueden verse distintos ejemplos de asignación.

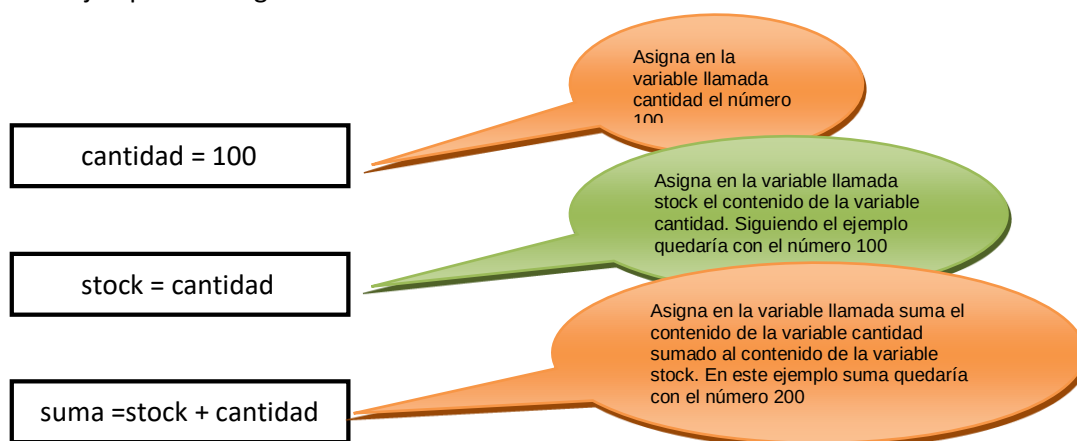


Figura 3. Ejemplos de asignación

3.2.2 Operaciones matemáticas

Sobre los datos se pueden realizar distintas operaciones. En la tabla 2 se detallan las operaciones posibles y el símbolo que se utiliza para expresarlas. A estos símbolos se los denomina operadores matemáticos.

Tabla 2. Operadores Matemáticos

Operación	Operador matemático
Suma	+
Resta	-
Multipliación	*
División	/
Resto de la división entera	%

Las operaciones se pueden realizar tanto sobre variables (tomando el contenido que tienen almacenado), como sobre constantes. En la figura 4 pueden verse distintos ejemplos de operaciones que se realizan una a continuación de la otra.

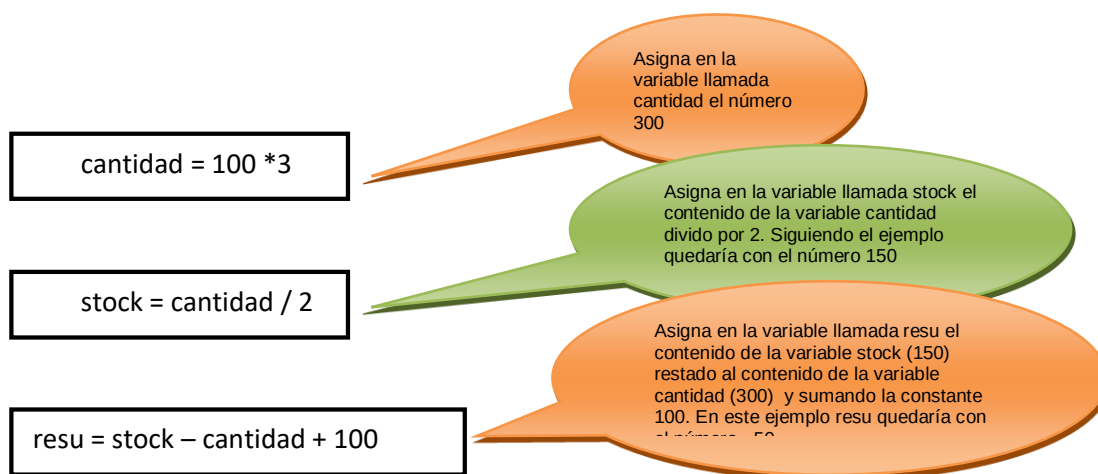


Figura 4. Ejemplos de operaciones matemáticas

Una mención especial requiere la operación de división ya que se comporta diferente según sea el tipo de dato de las variables o constantes sobre las que se está operando. Por ejemplo, al dividir el número 15 con el número 2 ($15 / 2$) como ambas son constantes de tipo entero la operación dará por resultado la de división entera. Es decir, que el resultado de $15 / 2$ será 7 y no 7.5 como sería lo esperado. Esta característica hay que tenerla en cuenta al realizar las operaciones ya que puede llevar a resultados erróneos pero brinda la posibilidad de realizar muchas otras tareas de forma sencilla, como por ejemplo, al descomponer un número. Siguiendo el ejemplo anterior si se quiere obtener 7.5 como resultado al menos una de las constantes debe ser del tipo real, es decir, un número con decimales. Como separador de decimales se usa el punto, por lo tanto, el resultado de $15. / 2$ será 7.5. El mismo resultado puede obtenerse al realizar la operación $15 / 2.0$.

Los operadores tienen precedencia al igual que en matemática, por ejemplo, el operador suma (+) separa términos y se ejecuta luego de realizar las operaciones de mayor precedencia como la multiplicación o la división. Muchas veces es necesario cambiar dicha precedencia y, por lo tanto, debemos agrupar las operaciones. Para ello se utilizan paréntesis. En una expresión pueden utilizarse todos los niveles de paréntesis que sean necesarios (NO corchetes). Por ejemplo, en la siguiente instrucción:

$$\text{suma} = 3 + 2 * 5$$

La variable `suma` quedará con el valor 13 ya que primero realiza la multiplicación y luego la suma. En cambio, si la misma instrucción se escribiera de la siguiente manera:

$$\text{suma} = (3 + 2) * 5$$

La variable `suma` quedará con el valor 25 ya que se utilizaron los paréntesis para resolver la suma primero que la multiplicación.

3.2.3 Salida de Resultados / Mensajes

Una vez realizado los cálculos sobre los datos el resultado debe mostrarse al usuario. Para ello, se utiliza la simbología que se muestra en la figura 5, donde pueden verse distintas formas de mostrar un resultado, donde se puede combinar datos con mensajes literales. Por defecto, la salida se realizará por pantalla, es decir, que se especifica lo que el usuario va a visualizar en la pantalla. Los mensajes se escriben entre comillas, haciendo que todo lo que se ponga entre las comillas se vea tal cual en la pantalla. Ahora si se desea mostrar el resultado de una operación que está almacenado en una variable, el identificador de la variable NO se pone entre comillas, de esa forma lo que se verá en pantalla es el contenido que tiene la variable en ese momento.

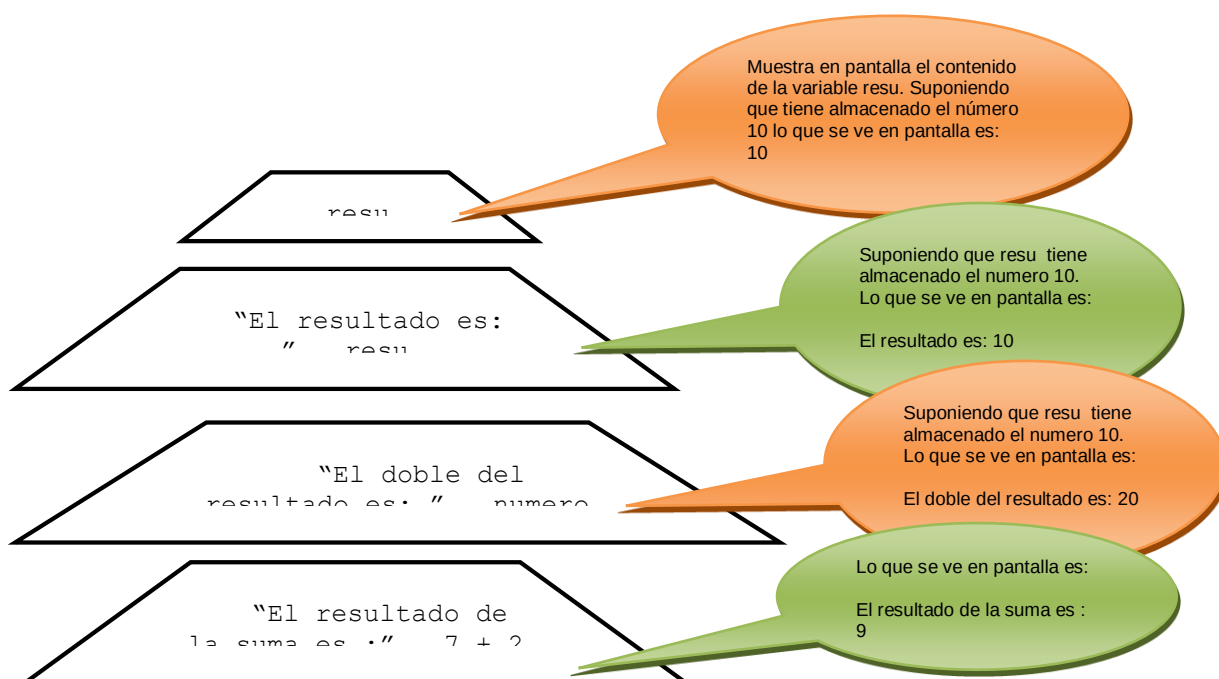


Figura 5. Ejemplos de salida por pantalla

3.2.4 Símbolos de indicación

Además de la entrada, salida y proceso hay tres símbolos adicionales que se utilizan para dar una mayor legibilidad al diagrama. Todo diagrama comenzará con un óvalo que indica el comienzo del algoritmo, dentro de este óvalo opcionalmente se puede escribir un nombre que represente el algoritmo que se está desarrollando. Al finalizar y para cerrar el diagrama se utilizará un triángulo invertido donde opcionalmente dentro se puede poner la letra F que indica fin. Por último, si se está desarrollando un diagrama largo que no entra en una hoja y se desea continuar en la hoja siguiente se utiliza un círculo como conector de la siguiente forma: al final de la hoja en lugar de la instrucción siguiente se pone el círculo con un número dentro, por ejemplo el número 1, ya que es el primer conector del algoritmo, luego en la hoja siguiente se repite en la parte superior el círculo con el mismo número para indicar que allí continua el diagrama. La Figura 6 muestra un ejemplo de la utilización de los símbolos de indicación.

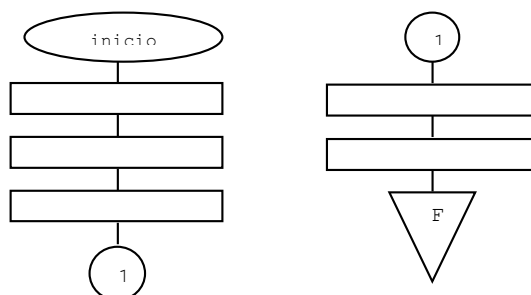
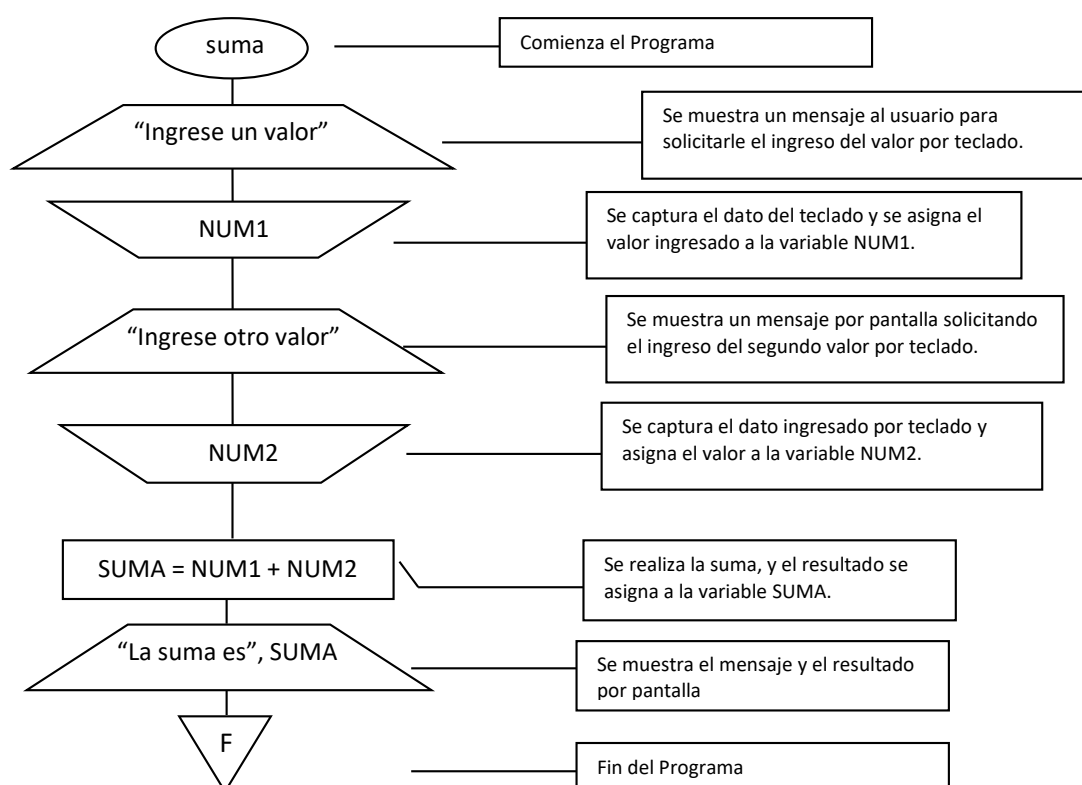


Figura 6. Ejemplos de utilización de símbolos de indicación

4. Algunos ejemplos

Ejemplo 1

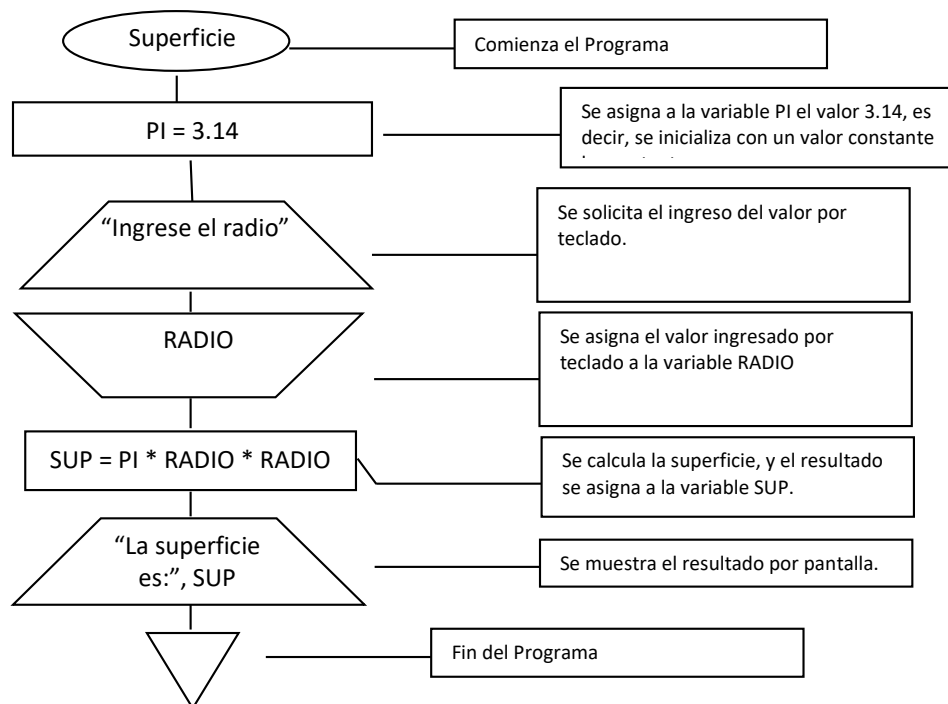
Realizar un programa que permita ingresar dos números enteros por teclado, realice la suma, y muestre por pantalla el resultado.



En este ejemplo se incorporaron al diagrama todos los mensajes que se muestran al usuario para indicar lo que debe hacer, en ejercicios siguientes, más complejos, los mensajes al usuario pueden ser omitidos para facilitar el diagrama pero luego sí, al codificar en el lenguaje de programación, es muy importante siempre poner mensajes aclaratorios indicándole al usuario lo que debe hacer y cómo se comporta el programa.

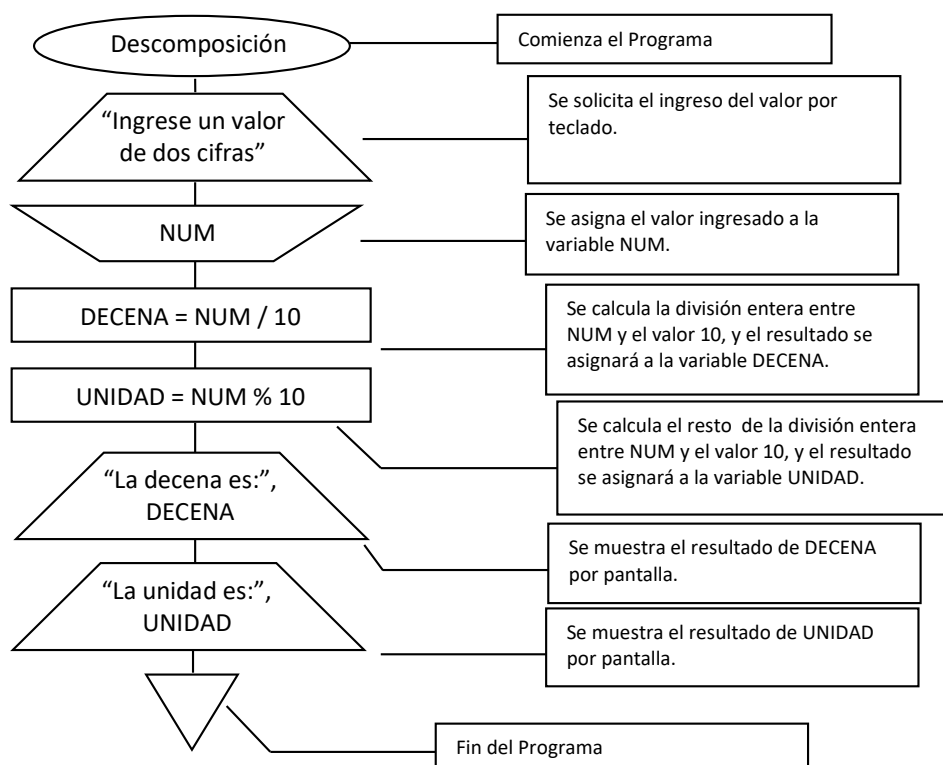
Ejemplo 2

Realizar un programa que permita ingresar el radio de un círculo, calcule la superficie, y muestre por pantalla el resultado.



Ejemplo 3

Realizar un diagrama que permita ingresar un número de dos cifras, y muestre por pantalla las unidades y las decenas.





Universidad Nacional de la Matanza

Programación Inicial

UNIDAD 3. EL LENGUAJE DE PROGRAMACION C

INDICE

1	UN POCO DE HISTORIA.	2
2	DESDE EL CÓDIGO FUENTE HASTA EL EJECUTABLE	2
3	ESTRUCTURA GENERAL DE UN PROGRAMA EN C	4
3.1	BLOQUE DECLARATIVO GENERAL	4
3.1.1	DIRECTIVAS AL PREPROCESADOR	5
3.1.1.1	INCLUDE	5
3.1.1.2	DEFINE	6
3.1.2	DEFINICIÓN DE VARIABLES GLOBALES Y ESTRUCTURAS	6
3.1.3	PROTOTIPOS DE FUNCIONES PROPIAS	6
3.2	BLOQUE PRINCIPAL	7
3.3	DESARROLLO DE FUNCIONES PROPIAS	7
4	ELEMENTOS DEL LENGUAJE	7
4.1	ALFABETO	8
4.2	IDENTIFICADORES	8
4.3	CONSTANTES	8
4.4	VARIABLES	8
4.5	TIPOS DE DATO	9
3.3.1	NÚMEROS ENTEROS	9
3.3.2	NÚMEROS REALES	10
3.3.3	CARACTERES	10
4.6	SENTENCIAS	10
4.7	REGLAS DE PUNTUACIÓN	11
5	DECLARACIÓN DE VARIABLES	11
6	FUNCIONES DE ENTRADA / SALIDA	12
6.1	SALIDA - FUNCIÓN PRINTF():	12
6.1.1	CADENA DE CONTROL	12
6.1.1	LISTA DE ARGUMENTOS:	14
6.2	ENTRADA - FUNCIÓN SCANF()	14
7	SENTENCIAS DE ASIGNACIÓN Y ARITMÉTICAS	16
7.1	COMBINACIÓN DE OPERADORES / OPERADORES ESPECIALES	16
7.2	EVALUACIÓN DE LAS SENTENCIAS DE ASIGNACIÓN	17
7.3	PAUSA PARA MOSTRAR LOS RESULTADOS	17
8	PROGRAMA DE EJEMPLO	18

UNIDAD 3

El lenguaje de programación C

OBJETIVOS: Conocer el lenguaje de programación “C”. Confeccionar programas en C con sentencias de entrada, de salida, de asignación, y utilizar “funciones de biblioteca”. Utilizar distintos tipos de datos simples.

1 Un poco de Historia.

El lenguaje C fue desarrollado por Dennis Ritchie en 1972, cuando trabajaba en los laboratorios Bell, junto a Ken Thompson en el diseño del sistema operativo UNIX. Es la evolución de dos lenguajes anteriores, el primero fue el BCPL (Basic Compiled Programming Language – Lenguaje Básico de programación compilado), luego como una simplificación del lenguaje BCPL adaptado a microcomputadoras (que son las computadoras actuales) crearon el lenguaje B. Por último, el lenguaje C fue la evolución del B y de allí provienen su nombre.

A raíz de la creciente popularidad de las microcomputadoras, comenzaron a surgir numerosas implementaciones de C que diferían en parte de la definición de lenguaje original, creando pequeñas incompatibilidades y disminuyendo la portabilidad del lenguaje. La portabilidad es la capacidad de ejecutar un mismo programa en distintas computadoras (con distinto hardware) y/o distintos entornos (por ejemplo, distintos sistemas operativos). Esto hizo necesaria la búsqueda de un C estándar que fue definido por el Instituto Nacional Americano de Estándares (ANSI) en el año 1983. Por lo tanto, cuando se habla de la versión estandarizada del lenguaje se hace referencia al ANSI C.

En 1977 Kernighan y Ritchie publicaron una descripción definitiva del lenguaje “El manual de referencia del lenguaje C”, que es una fuente de consulta fundamental para todo programador en este lenguaje.

A mediados de los 80, Bjarne Stroustrup, también en los laboratorios Bell, crea el lenguaje C++ que soporta el paradigma de programación orientada a objetos.

2 Desde el código fuente hasta el ejecutable

Como se mencionó en la unidad 1, un programa es una secuencia de instrucciones que se escriben en un lenguaje de programación que el programador entiende pero no la computadora, por lo tanto, necesita de un proceso de traducción. El lenguaje C, es un lenguaje compilado, por lo tanto, todo el código será analizado y traducido por completo hasta obtener un programa ejecutable. Para llegar desde el código fuente hasta el programa que se puede ejecutar en una computadora hay una serie de pasos que se deben llevar a cabo. Esos pasos pueden verse en la figura 1.



Figura 1: pasos para generar un programa ejecutable partiendo del código fuente

Inicialmente se escribe el programa en el lenguaje C generando un programa fuente. El programa fuente no es más que un archivo de texto escrito respetando las sentencias del lenguaje.

Dentro del código fuente, el programador puede utilizar partes de código ya desarrolladas, de forma que las cosas repetitivas no tenga que escribirlas en cada programa. Por ejemplo, para leer datos del teclado o para mostrar mensaje por pantalla no hay instrucciones del lenguaje que puedan hacerlo en forma directa, pero sin embargo, esa funcionalidad es parte de lo que se llaman bibliotecas de funciones. Una función es una secuencia de instrucciones que realiza una tarea específica. Las funciones se identifican por un nombre, mediante el cual desde el programa principal se “invoca” a dicha función. Invocar una función significa que el programa que se viene ejecutando en forma secuencial hace un salto, y comienza a ejecutar las instrucciones que contiene la función, cuando llega al final de la función vuelve a la línea del programa que la invoco. A las funciones se les puede pasar datos y pueden retornar un único valor. Por ejemplo, a una función que permite leer un valor de teclado se le indica en que variable va a guardar ese dato leído. A la función para mostrar un mensaje en pantalla se le envía el mensaje a mostrar, de esta forma las funciones se pueden reutilizar y adaptar a distintos programas con los datos que reciben.

Al escribir un programa si se quieren utilizar funciones ya desarrolladas se le debe avisar al compilador donde está la función que queremos utilizar, es decir, si se necesita utilizar, por ejemplo, las funciones para leer datos de teclado se le debe indicar al compilador en que biblioteca se encuentra dicha función para que la incorpore a nuestro código fuente y de ese modo podamos utilizarla. Para ello, en el código fuente se agrega una línea de código por cada una de las bibliotecas que se necesiten incluir. Las bibliotecas provistas por el entorno de programación están optimizadas de forma que cuando se incluyen el código fuente solo se incluye lo que se conoce como cabecera de las funciones. La cabecera de una función es una línea que indica el nombre de la función, que datos se le puede enviar, y si retorna o no un valor. El preprocesador es el encargado de agregar esas cabeceras de las funciones de las bibliotecas que el programador indica en el código fuente. Además, el preprocesador modifica el código fuente escrito originalmente reemplazando las constantes definidas por su valor correspondiente. Es decir, que el pre-procesador es el que genera el archivo fuente final a compilar.

El compilador es el encargado de tomar el programa fuente, chequear que cumpla con la gramática del lenguaje C y si no hay errores generar el programa objeto, que es el programa en un formato que la computadora puede comprender. Si hay errores se informan al usuario indicando el número de línea donde se encuentra.

El programa objeto si bien ya está en un formato entendible por la computadora, aún no puede ejecutarse porque está incompleto, falta el código objeto de las funciones de biblioteca que se utilizan. Es el linker el que toma el código de dichas funciones y lo junta con el código de nuestro programa creando finalmente el programa ejecutable completo. Las funciones de biblioteca estándar están optimizadas de forma que el linker solo incorpora el código de las funciones que realmente se utilizan en el programa y NO el código de todas las funciones de la biblioteca a la cual pertenece.

3 Estructura general de un programa en C

Todo programa en C está estructurado en funciones. El lenguaje C tiene una función principal, que es por la cual comenzará la ejecución del programa. Dicha función se llama main, no pudiendo faltar, ya que a ésta le transfiere el control el sistema operativo al ejecutarse un programa.

Un programa en C se estructura según la imagen de la figura 2.

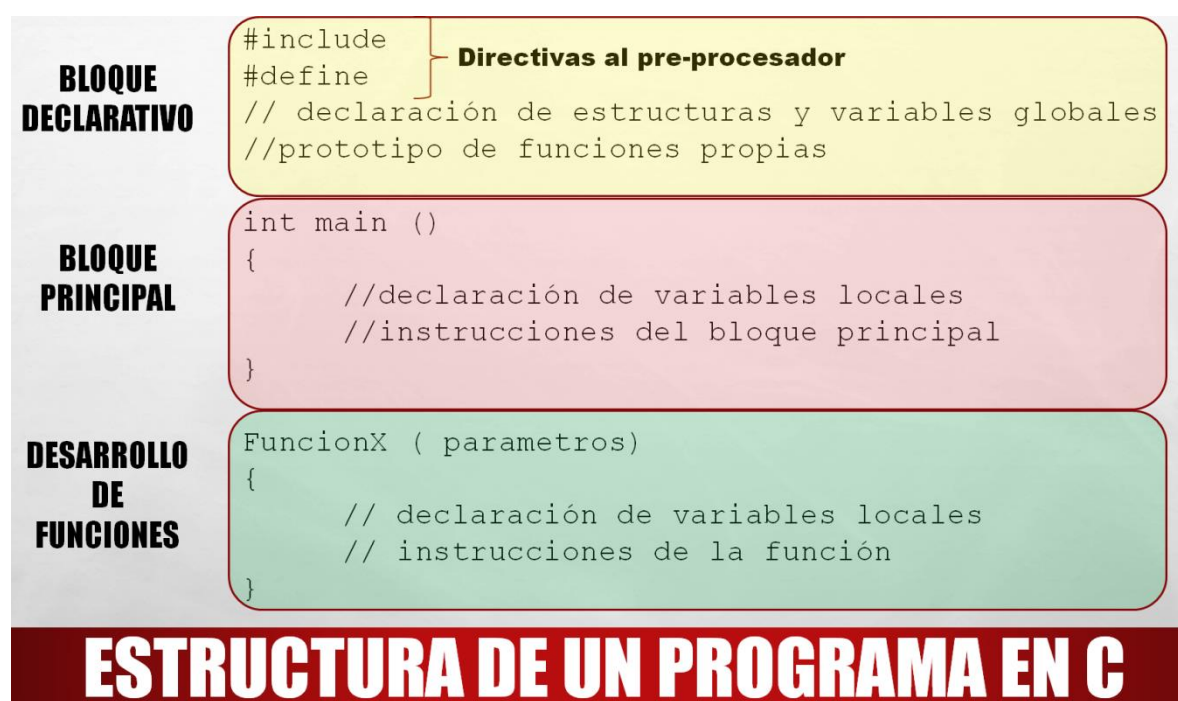


Figura 2: Estructura de un programa en C

Se analiza a continuación el contenido de los bloques enunciados anteriormente.

3.1 Bloque declarativo general

En este bloque se incorporan las directivas al preprocesador, la definición de variables globales, estructuras y prototipos de funciones propias.

3.1.1 Directivas al preprocesador

Todos los comandos al preprocesador comienzan con el símbolo #.

Si bien existen otros comandos se utilizarán solo dos: include y define.

3.1.1.1 Include

Agrega al archivo fuente el contenido de otro archivo. Se utiliza para incorporar el código de las bibliotecas al código fuente.

Para incluir bibliotecas estándar el nombre de la biblioteca se escribe entre los símbolos menor y mayor. Estas bibliotecas las busca dentro de la carpeta llamada include dentro de la estructura de directorios del compilador. Recuerde que las bibliotecas estándar solo tienen los prototipos de las funciones también llamados cabeceras, por lo tanto, estos archivos tienen la extensión .h que viene de header, cabecera en inglés.

La forma general es:

```
#include <nombrebiblioteca.h>
```

Existen muchas bibliotecas que se pueden utilizar algunas de ellas son:

stdio.h para operaciones de entrada y salida.

string.h para operaciones con cadenas de caracteres.

math.h declara funciones matemáticas.

stdlib.h declara funciones para conversiones numéricas, asignar memoria y otras funciones.

time.h funciones para manipulación de fecha y hora.

conio.h funciones de pantalla.

graph.h incluye funciones para gráficos.

dos.h incluye las funciones del sistema operativo DOS.

ctype.h funciones para operar con caracteres.

También puede incluirse cualquier otro archivo que no sea una biblioteca estándar, lo que nos dará la posibilidad de crear nuestras propias bibliotecas.

Para incluir cualquier otro archivo en lugar de escribir su nombre entre menor y mayor, se utilizan comillas. Dentro de las comillas se puede escribir una ruta completa o directamente el nombre del archivo. Si solo se pone el nombre del archivo lo buscará en la misma carpeta donde se encuentra el código fuente.

Por ejemplo, #include "misFunciones.c". Incluye todo el código de archivo misFunciones.c en el programa fuente. Dicho archivo DEBE estar en la misma carpeta que el archivo fuente sobre el que estamos trabajando sino no lo encontrará.

En cambio, #include "c:\\funciones\\misFunciones.c" buscará el archivo a incluir en el disco c dentro de la carpeta funciones. Observe que se utilizan dos barras en lugar de una, esto es porque la barra es un carácter llamado de escape utilizado para realizar funciones especiales dentro del lenguaje, esto se verá a continuación.

Siempre es recomendable para mayor portabilidad utilizar la primera opción y poner los archivos a incluir en la misma carpeta que el código original.

3.1.1.2 `define`

Permite definir un texto a reemplazar dentro del programa. Se utiliza para definir constantes y macros.

Forma general es:

```
#define nombre-simbólico texto de reemplazo
```

A partir de esta instrucción cualquier aparición del nombre-simbólico, dentro del programa, será reemplazada por el texto de reemplazo. Permite definir constantes globales ya que la etiqueta será buscada y reemplazada en todo el programa, siempre teniendo en cuenta que se trata de un reemplazo de texto, es decir, actúa como el comando buscar y reemplazar de cualquier editor de texto. El programa a compilar será el resultante luego de realizar los reemplazos.

Ejemplos:

```
#define MAXIMO 100

#define PI 3.14159

#define TITULO "RESUMEN"
```

También puede definirse lo que se conoce como macros haciendo que la etiqueta sea reemplazada por varias instrucciones incluso pudiendo enviarle datos para hacerlo más genérico. Las macros no se utilizarán en esta materia.

3.1.2 Definición de variables globales y estructuras

Al escribir un programa, para guardar los datos con los que trabajará, necesita variables. Dentro de los lenguajes de programación, las variables tienen lo que se conoce como “ámbito” de vigencia de la variable. El ámbito indica desde que parte de nuestro programa podemos acceder a esa variable. En general, cada bloque o función del programa define sus propias variables, lo que se conoce como variables locales. Si se necesita una variable que pueda ser accedida desde cualquier parte del programa se deberá declarar en forma global, antes del bloque principal. No es recomendable la utilización de variables globales ya que fácilmente puede perderse el control de donde fue modificada, y además, hace mucho más difícil que el código realizado pueda reutilizarse en otros programas.

Más adelante se verá el tema de estructuras de datos, en general una estructura deberá ser accedida desde todo nuestro programa por lo que su declaración será en el ámbito global.

3.1.3 Prototipos de funciones propias

Además de poder utilizar funciones de bibliotecas ya desarrolladas, más adelante se construirán funciones propias. En el bloque declarativo se incluyen lo que se conoce como prototipo de funciones, que es la especificación del nombre de la función, que datos recibe y que retorna. El código con las instrucciones de dichas funciones se pondrá debajo del bloque principal.

El prototipo de funciones se pone para que el programa quede más ordenado y poder encontrar rápidamente el bloque principal de nuestro programa. También es posible no poner los prototipos y escribir la función completa antes del programa principal, pero esto haría más difícil ubicarse dentro

del código fuente. Si dentro de una función se llama a otra función, debe ponerse primero el prototipo de la que es llamada, y luego, el prototipo de la función que la llamó.

3.2 Bloque Principal

Es el inicio del programa. Como el lenguaje C está estructurado en funciones, esta es la función principal que siempre debe estar presente y es lo primero que se ejecuta. El formato general es el siguiente:

```
int main ()
{

    return 0;
}
```

Donde main es el nombre de la función (main significa principal en inglés). La palabra int significa que esta función retorna un número entero. Todo programa debe retornar al sistema operativo un número entero que indica si la ejecución fue exitosa o no, retornar un 0 significará que no hubo errores y cualquier otro valor servirá para indicar error. Los paréntesis vacíos indican que la función no recibe datos, en materias siguientes se trabajara con datos enviados al programa al iniciar el mismo. También para indicar que no recibe datos en lugar de los paréntesis vacíos puede ponerse dentro la palabra void, que es el tipo de dato vacío, indicando que no recibe nada.

Las llaves forman el bloque de la función principal, dentro de las llaves se escribirán todas las instrucciones del programa, comenzando siempre por la declaración de variables locales que se verán en la sección siguiente.

Antes de cerrar la llave del bloque del programa principal se pone la palabra return y en este caso se retorna un 0 indicando al sistema operativo que el programa finalizó correctamente.

3.3 Desarrollo de Funciones Propias

Si se utilizan funciones propias y en el bloque declarativo se incluyó el prototipo de las mismas, debajo de la función principal se escribe el código completo de dichas funciones. Cada una de ellas tendrá su nombre, datos que recibe, valor de retorno y su bloque definido entre llaves. Nuevamente dentro de cada función al comienzo del bloque se definen las variables que se utilizan localmente en la misma, y luego, las instrucciones para realizar la funcionalidad deseada.

4 Elementos del lenguaje

Todo lenguaje de programación está compuesto por una serie de elementos que se detallan a continuación y son los elementos básicos que componen el C:

1. Alfabeto
2. Identificadores
3. Constantes
4. Variables
5. Tipos de datos
 - números enteros o de punto fijo
 - números reales ó de punto flotante
 - caracteres – cadenas de caracteres – string
 - modificadores de tipo
6. Sentencias
7. Reglas de puntuación

4.1 Alfabeto

Como en cualquier lenguaje este consta de:

- Letras mayúsculas y minúsculas (son distintas MAYUSCULAS y minúsculas)
- Dígitos del 0 al 9
- Caracteres especiales: () { } ; : " ' , . & * % #
- Palabras reservadas: main, for, while, include, if, switch, auto, etc (estas palabras se llaman reservadas ya que no pueden ser utilizadas como identificadores)
- Operadores (aritméticos, de relación, lógicos, de incremento, de asignación, etc.). Ej.: + *, < > , && , ++ , =

4.2 Identificadores

Se llama así, a los nombres que se utilizan para nombrar a las constantes, variables de cualquier tipo, y a las funciones.

Puede ser una sucesión de letras, dígitos y algunos caracteres especiales, comenzando siempre con letra. Se admite como letra al carácter _ (guion bajo). No hay límites en cuanto a cantidad máxima, como mínimo una letra.

Ejemplos:

a A suma x33 G lote superficie AutoLuz k_k23

4.3 Constantes

Son aquellos valores que no se alteran durante la ejecución del programa. Hay dos tipos de constantes, según su contenido, numéricas (pueden escribirse en sistema decimal, octal y hexadecimal) y NO numéricas. Por ejemplo:

- Numéricas : 56, 5, 8.90E12, -1.25, 3.1415
- No numéricas: '\$', "mts", "Grados", '5', "34-35"

Las constantes pueden utilizarse como parte de operaciones, por ejemplo, si a la variable suma se le quiere sumar diez entonces se puede escribir:

```
suma = suma +10;
```

En este caso 10 es una constante numérica entera.

También pueden definirse constantes con un identificador de dos formas:

- En el bloque declarativo usando el define, visto anteriormente.
- Anteponiendo al tipo, en la declaración de variables, la palabra const.

4.4 Variables

Se define una variable, como la representación simbólica de un lugar de la memoria, donde se guarda un valor, que es el valor de la variable. Su nombre es un identificador, cuyo valor se puede alterar durante la ejecución de un programa.

A cada variable se le debe asignar un tipo de datos e inclusive, se le puede asignar un valor inicial, como se verá luego.

Debemos tener clara la diferencia entre letras minúsculas y mayúsculas, ya que se asumen como caracteres distintos en el lenguaje C.

4.5 Tipos de dato

Una de las características más importantes del lenguaje C es su capacidad para operar con una gran diversidad de tipos de datos, admitiendo la creación por parte del usuario de nuevos tipos.

Los algoritmos operan sobre datos, los cuales pueden ser de distinto tipo y se caracterizan por:

- Un rango de valores posibles
- Un conjunto de operaciones realizables sobre dicho tipo de datos
- Su representación interna.

Significa que al asumir un tipo de dato se indica la clase de valores que pueden tomar las variables, y las operaciones que se pueden hacer sobre ellos.

Por ahora se analizarán los tipos de datos “simples”, que son aquellos donde una variable ocupa “un lugar de memoria” y contiene un único valor.

Los tipos de datos simples más comunes se muestran en la tabla 1.

Tabla 1: Tipos de dato simples

Tipo	Nombre	Espacio que ocupa	Rango
Entero corto	short int	2 bytes	-32768 a 32767
Entero	int	4 bytes	-2.147.483.648 a 2.147.483.647
Caracter	char	1 byte	-128 a 127
Real	float	4 bytes	$1.7 \cdot 10^{-38}$ a $1.7 \cdot 10^{38}$ (7 díg.)
Real doble	double	8 bytes	$1.7 \cdot 10^{-308}$ a $1.7 \cdot 10^{308}$ (16 díg.)

Si no se necesitan utilizar números negativos, al tipo de dato sin decimales, puede anteponerse el modificador unsigned de forma que el rango ya no se divide entre positivos y negativos sino que toma solo números positivos quedando los rangos según la tabla 2.

Tabla 2: Modificador unsigned

Tipo	Nombre	Espacio que ocupa	Rango
Entero corto sin signo	unsigned short int	2 bytes	0 a 65535
Entero sin signo	unsigned int	4 bytes	0 a 4.294.967.295
Carácter sin signo	unsigned char	1 byte	0 a 255

El lenguaje también dispone de tipos de datos ESTRUCTURADOS, que como su nombre lo indica, cada variable puede tener una adecuada estructura conteniendo una serie de valores de igual ó distinto tipo.

Ejemplos: cadenas, arrays, estructuras, etc.

4.5.1 Números enteros

Son aquellos que solo contienen dígitos, precedidos o no por el signo. No tienen exponente (E) ni punto decimal.

Ejemplos:

- En decimal: 0 32767 -12345 231 89

- En octal: Se antepone al nro un 0 (cero). Ej 071 en octal es igual al 57 en decimal
- En hexadecimal: Se antepone al nro 0X. Ej: 0x100 en hexadecimal es igual al 256 en decimal

4.5.2 Números reales

Pueden escribirse de forma diferente, incluyendo dígitos, “un punto decimal”, y opcionalmente, pueden incluir un signo y/o un exponente.

Ejemplos:

12.3 -9.0009 3.0E12 4E-15 -2.5E-12 0.235

4.5.3 Caracteres

Las variables del tipo char pueden guardar un solo símbolo, un carácter. Pero en realidad internamente guarda un número que corresponde al código ASCII (sigla en inglés de American Standard Code for Information Interchange - Código Americano Estándar para el Intercambio de Información). Por ejemplo, el código ASCII de la letra A es el 65 el de la B el 66, y así sucesivamente. El código de las letras minúsculas comienza en el 97 para la letra a.

Por lo tanto, una variable del tipo char se puede utilizar para guardar un carácter o números pequeños, dependerá luego de cómo mostremos dicha información.

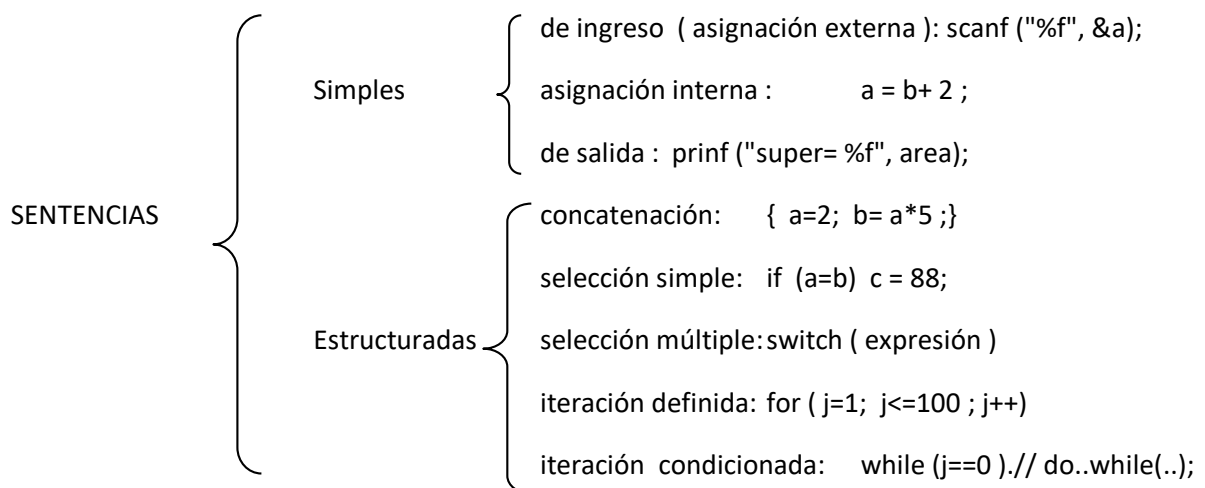
Las constantes del tipo char se escriben entre apostrofes (o comillas simples)

Ejemplos: 's' 'v' '*' '@' '9'

Es importante destacar que las letras tienen números correlativos lo que servirá más adelante para realizar validación de los datos ingresados.

4.6 Sentencias

Una sentencia en C puede ser una sola instrucción ó un grupo de instrucciones encerradas entre llaves. Hay dos tipos de sentencias, las simples y las estructuradas. Más adelante se verá en detalle cada una de ellas.



4.7 Reglas de puntuación

A medida que se avance en el desarrollo de los temas se verá puntualmente la forma correcta de escribir cada una de las sentencias y bloques del lenguaje. Como regla general debe tener en cuenta que:

- Las instrucciones finalizan con ; (punto y coma)
- Los bloques se delimitan entre { } (llaves)
- Debe respetarse las minúsculas y mayúsculas ya que el lenguaje es sensible, teniendo en cuenta que la mayoría de palabras reservadas se escriben todas en minúscula.
- Si dentro del código fuente quieren poner comentarios se utiliza:
 - // para los comentarios de una sola línea.
 - /* comentario */, para comentarios de varias líneas.
 - Los comentarios se utilizan para clarificar el código fuente agregando notas que expliquen lo que se está haciendo. Los comentarios no agregan peso al programa final ya que son eliminados por el pre-procesador. Solo quedan en el código fuente.

5 Declaración de variables

Todas las variables que se utilicen en el programa (main) deben ser declaradas al inicio del mismo o de cada función que las use, asignando valores iniciales, si se requieren. Esta declaración tiene por objeto asociar a cada variable un tipo de dato y efectuar una reserva de memoria.

Forma general es:

tipo de dato lista de variables ;

En realidad, antes del tipo de dato en el lenguaje C se especifica donde se va a guardar dicha variable. Pero en esta materia se usa solo el almacenamiento por defecto y por lo tanto no se escribe. Es equivalente de escribir la palabra `auto` delante del tipo de dato para indicar el tipo de almacenamiento automático. Es por eso que `auto` es una palabra reservada del lenguaje.

Ejemplos:

- Para definir una única variable del tipo entera llamada n1:

```
int n1;
```

- Para definir tres variables enteras:

```
int num1, num2, suma;
```

- Para definir dos variables reales, el segundo inicializado en un valor:

```
float a, b=7.2;
```

- Para definir dos variables char, el primero inicializado en un valor:

```
char letra='s', simbolo;
```

A la declaración de tipo se le puede anteponer la palabra `const`, en cuyo caso no serán variables sino constantes y luego se asigna el correspondiente valor.

Ejemplo:

```
const float pi = 3.14159;
```

Esto significa que pi ocupa un lugar en memoria pero su valor al ser constante se asigna por única vez y ya no puede ser modificado en todo el programa.

AMBITO DE INCUMBENCIA: Las variables según el lugar en el cual se las declare pueden ser:

- **VARIABLES GLOBALES.** Son las definidas en el bloque declarativo general y su campo de validez es todo el programa, incluyendo todas las funciones.
- **VARIABLES LOCALES.** Son las definidas dentro de cada función, incluso en el main(), y su campo de validez se limita solo a la función donde fueron definidas. Cabe destacar que es posible crear variables locales con el mismo identificador en distintas funciones, teniendo siempre en cuenta que a pesar de tener el mismo nombre son variables distintas ya que cada función define sus variables locales en áreas separadas de memoria. También dentro de una función es posible definir un bloque interno (encerrado entre llaves) pudiendo definir variables locales a ese bloque pero pudiendo acceder a las variables del bloque de la función que lo contiene.

6 Funciones de entrada / salida

El lenguaje C no posee sentencias con capacidad de generar la entrada y/o salida de datos, por lo cual se recurre a funciones de biblioteca. Estas funciones permiten la transferencia de información entre los dispositivos de entrada y/o salida con la computadora. Se puede invocar a estas funciones desde cualquier punto del programa. Las que más se utilizarán son scanf() y printf(), que son sentencias con formato, ambas están incluidas en la biblioteca stdio.h.

6.1 SALIDA - función printf():

Permite la transferencia de información entre la computadora y un dispositivo de salida como ser la pantalla o la impresora.

Forma general es:

```
printf ( "cadena de control", lista de argumentos );
```

6.1.1 Cadena de Control

La cadena de control: se compone de una lista de:

1. Especificaciones de formatos
2. Secuencias de escape
3. Leyendas, títulos ó mensajes aclaratorios

Con estos elementos entre comillas puedo diseñar la salida que deseo exhibir.

1) Especificaciones de formato.

Se debe indicar una especificación de formato para cada variable a informar, el cual se compone del símbolo % seguido del carácter de conversión. Indica al compilador que tipo de variable va a ser exhibida. Los códigos de formatos más utilizados son:

% d	para un número entero con signo
% f	para un número real
% c	para un carácter

- % s para una cadena de caracteres (un texto)
- % e para números reales con exponente
- % hd para entero corto (short int)
- % x para un entero en hexadecimal – imprime en hexadecimal.

Se puede formatear la exhibición, o sea, especificar “la longitud de campo deseada” para cada valor a exhibir (un valor entero luego del %). Esta longitud puede precederse con un signo -, lo cual significa alinear por izquierda, con + o sin poner nada alinea por derecha. Si tengo números reales, especifico el total de posiciones, un punto y la cantidad de cifras decimales.

Ejemplos

%4d: muestra un número entero en cuatro lugares, si el número tiene menos de cuatro deja espacios en blanco a la izquierda, si tiene más lo muestra igual completo.

% 8.3 f : muestra un número real que en total ocupa 8 espacios con 3 decimales

%0f (sin decimales): muestra un número real sin decimales.

Los datos tipo char – como enteros

Una variable definida “tipo char” también puede ser impresa con un formato entero. Dependiendo del formato que se utilice para imprimirla, ya sea %c ó %d, se obtienen distintos resultados.

La siguiente línea de código muestra la constante de la letra a con su valor en decimal (código ASCII) y en formato caracter

```
printf ("El caracter %c tiene el valor %d ", 'a' , 'a');
```

Muestra en pantalla: El carácter a tiene el valor 97

2) Secuencias de escape.

Luego de mostrarse un valor, el cursor queda después del último carácter exhibido, en la misma línea. Si quiero que vuelva al principio de la próxima línea debo colocar antes de la comilla de cierre el "carácter de escape" \n. Puede colocarse en cualquier lugar de la cadena de control. Existen otras secuencias de escape, están formadas por una \ y un carácter que puede ser una letra o una combinación de dígitos. Se usan para tabular, cambio de línea y representar caracteres no imprimibles.

SECUENCIAS	ACCION
\n	nueva línea
\t	tabulación horizontal
\"	imprime comillas
\'	imprime un apóstrofe
\b	retrocede un espacio el cursor
\r	retorno de carro

3) Leyendas, títulos ó mensajes aclaratorios.

Todo texto que se ponga entre las comillas de la cadena de control que no sea un formato o una secuencia de escape se verá tal cual en pantalla, por lo que pueden escribirse mensaje para el usuario. Hay que aclarar que los caracteres especiales del español como la letra ñe y acentos no se visualizan correctamente.

6.1.1 Lista de argumentos:

Es una lista de expresiones, constantes y variables, cuyos valores se van a imprimir, separadas por comas y en concordancia con los formatos especificados en la cadena de control.

Ejemplo 1 :

```
int codi=7 ; float temp= -31.42;
printf ("\n El codigo es %d y la temperatura es = %7.2f \n", codi, temp);
```

Comienza cambiando el cursor de línea y luego imprime:

El codigo es 7 y la temperatura es = -31.42

Luego el cursor baja una línea.

Ejemplo 2

```
printf ("31+21 es igual a: %d cms", 21+31);
```

Imprime: 31+21 es igual a: 52 cms

Ejemplo 3:

```
printf("chau %c seis = %d", '@', 6 );
```

Imprime: chau @ seis = 6

6.2 ENTRADA - función scanf()

Se utiliza para el ingreso de información a la computadora, a través, de las variables indicadas. Es la encargada de leer los datos que se ingresan desde el teclado. Al encontrar esta función, la computadora queda a la espera del ingreso de uno ó más valores desde el teclado, luego de ingresar el último dato, debo pulsar "enter". Si hay más de un dato a digitar, se pueden separar con espacio, una tabulación o con enter, pero siempre al final se debe presionar enter.

Forma general:

```
scanf( "formatos", lista de argumentos);
```

Donde el primer parámetro se compone de una lista de especificaciones de formatos entre comillas. Se debe indicar una especificación de formato para cada variable que se quiera leer. Son los mismos que se utilizan para el printf(...).

Es importante no dejar espacios ni poner otros caracteres entre los formatos para las variables para evitar problemas al leerlos.

La lista de argumentos está formada por la dirección de memoria de las variables que se desean leer. Para obtener la dirección de memoria de una variable se utiliza el operador &. Este operador indica el lugar de memoria donde será enviado el valor ingresado por teclado y registrado por la función, mediante el nombre del identificador. Se debe respetar el orden de los formatos según las direcciones de memoria que se especifiquen en el argumento, el primer formato corresponderá con la primer dirección de memoria, el segundo con la segunda y así sucesivamente.

Ejemplo1

Leer un entero:

```
int a;  
  
scanf("%d", &a);
```

Ejemplo 2

Leer dos enteros:

```
int n1, n2;  
  
scanf("%d%d", &n1, &n2);
```

Ejemplo 3

Leer un real y un entero:

```
int num;  
  
float f;  
  
scanf("%f%d", &f, &num);
```

Todos los datos al ser ingresados por teclado se almacenan temporalmente en un buffer de teclado, que es una posición de memoria intermedia que guarda todas las teclas presionadas. Luego, la función scanf toma los datos de ese buffer y los pasa a la dirección de memoria indicada según el formato establecido. Este procedimiento funciona perfectamente para variables numéricas pero el scanf tiene un problema al leer caracteres. El formato %c lee un solo carácter, por lo tanto, lo va a buscar al buffer de teclado, pero el buffer muchas veces no está vacío ya que si antes se leyó otro valor el código de la tecla enter que se presiona para confirmar los datos queda en el buffer ya que no es parte del dato ingresado. Este código de la tecla enter es tomado automáticamente por el scanf, y por lo tanto, no permite ingresar el dato. Entonces para poder leer un carácter de teclado si antes se leyó otro dato hay que limpiar el buffer de teclado. Para ello hay dos formas:

- Usando la función fflush(stdin): esta función limpia el buffer de la entrada estándar, es decir, del teclado. El problema es que esta no es una función estándar por lo que no funciona en todos los compiladores.
- Quitar el carácter que sobra del buffer utilizando getchar(): getchar es una función que recupera un carácter del buffer, por lo tanto, se puede poner luego de leer un dato para "limpiar" ese carácter del enter que quedó en el buffer.

Ejemplo de un programa completo que lee primero un dato entero y luego una letra, limpiando el buffer en el medio:

```
#include <stdio.h>
int main()
{
    int num;
    char letra;
    printf ("Ingrese un numero: ");
    scanf("%d", &num);
    getchar();
    printf("Ingrese una letra: ");
    scanf("%c",&letra);
    printf("Se ha ingresado el numero: %d y la letra: %c", num, letra);
    return 0;
}
```

7 Sentencias de asignación y aritméticas

Las sentencias secuenciales se escriben directamente tal como se explicó en la unidad 2, tanto para operaciones matemáticas como para asignaciones. La única diferencia con el diagrama es que en el código C cada instrucción al final se termina con el punto y coma.

7.1 Combinación de operadores / operadores especiales

El C lenguaje admite la simplificación de una variable en las sentencias aritméticas cuando se combinan los operadores aritméticos y el igual.

Lo común es:

variable= variable operador expresión;

Se puede reemplazar por:

variable operador= expresión;

Algunas combinaciones posibles son:

* =	mult. + asignación
/ =	división + asignación
% =	resto + asignación
+ =	suma + asignación
- =	resta + asignación

Ejemplos:

x = x + 3;	se puede escribir:	x += 3 (suma y asignación)
x = x * 3;	se puede escribir :	x *= 3 (multiplica y asigna)

El lenguaje C dispone además de dos operadores especiales el ++ y el --.

El operador ++ permite incrementar en 1 el valor de una variable, y el operador —decrementar en 1 el valor de una variable.

Por ejemplo, `a = a + 1`; puede escribirse como `a++`; pero también, puede escribirse como `++a`; o incluso usando la combinación de operadores como `a += 1`;

El operador ++ o -- puede usarse tanto antes como después de la variable. Si se la aplica solo a la variable tiene el mismo efecto pero en una asignación el efecto es diferente. Por ejemplo el siguiente programa:

```
#include <stdio.h>
int main()
{
    int n1=5,n2;
    n2=n1++;
    printf("n1:%d  n2:%d", n1,n2);
}
```

Mostrará: n1: 6 n2: 5

Ya que el operador ++ se toma como un post-incremento y se aplica luego de asignar el valor de `n1` a `n2`. En cambio, si esa línea se reemplaza por `n2=++n1`;

Mostrará: n1: 6 n2: 6

Ya que el operador ++ se toma como un pre-incremento y se aplica antes de asignar el valor a `n2`.

Esto mismo aplica para el operador --

7.2 Evaluación de las sentencias de asignación

Se realiza en tres pasos:

- Se evalúa la expresión, o sea, se la resuelve obteniendo un resultado.
- Si es posible, adecua el tipo de dato del resultado de la expresión al tipo de dato de la variable de la izquierda. Si no es posible puede o no indicar error y asignar cualquier valor (entero=real : asigna parte entera).
- Asigna el resultado de la expresión a la variable de la izquierda.

PRECAUCIONES:

No colocar dos operadores consecutivos, salvo los permitidos, usar paréntesis si es necesario o existen dudas sobre la precedencia de los operadores.

La división por cero es una operación prohibida, cancela el programa.

7.3 Pausa para mostrar los resultados

Los programas que se realizarán, son programas de consola. Si se trabaja con el sistema operativo Windows al ejecutar desde la interfaz gráfica un programa de consola (haciendo doble click al ejecutable) el mismo se ejecuta, pero al finalizar automáticamente se cierra y no permite ver los resultados. Por lo tanto, es necesario antes de finalizar el programa poner una pausa para permitir al usuario visualizar los resultados. Para ello podemos utilizar:

- La función getch() disponible en la biblioteca conio.h, esta función es similar al getchar pero no muestra el carácter presionado en pantalla. El problema es que getch() no es una función estándar, por lo tanto, no funciona con todos los compiladores.
- Hacer una llamada al sistema operativo e invocar la pausa. En Windows la instrucción sería: system("pause"); y para poder usar la función system debe incluirse la biblioteca stdlib.h.
- Poner un mensaje propio personalizado y esperar a que se presione un tecla cualquiera (usando getchar o scanf) o un tecla particular validando el ingreso de datos utilizando sentencias repetitivas que se verán en unidades siguientes.

8 Programa de ejemplo

Siempre existen diversas formas para realizar un programa. Puede cambiar desde el nombre que se le den a las variables hasta las instrucciones que se utilicen, por lo tanto, es posible que nuestros programas nunca queden exactamente igual al de un compañero o al del profesor pero aún así siguen estando correctos. Vea un sencillo ejemplo y distintas formas de resolverlo:

Programa a realizar: Ingresar dos números enteros, calcular y mostrar la suma:

Primera forma: con dos scanf y una variable para el resultado

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n1,n2, suma;
    printf("Ingrese el primer numero a sumar:");
    scanf("%d", &n1);
    printf("Ingrese el segundo numero a sumar:");
    scanf("%d", &n2);
    suma = n1+n2;
    printf ("El resultado de la suma es:  %d\n", suma);
    system("pause");
    return 0;
}
```

Segunda forma: con un solo scanf y una variable para el resultado

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n1,n2, suma;
    printf("Ingrese dos numeros a sumar:");
    scanf("%d%d", &n1, &n2);
    suma = n1+n2;
    printf ("El resultado de la suma es:  %d\n", suma);
    system("pause");
    return 0;
}
```

Tercera forma: con un solo scanf y sin variable de resultado:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
```

```
int n1,n2;
printf("Ingrese dos numeros a sumar:");
scanf("%d%d", &n1, &n2);
printf ("El resultado de la suma es:  %d\n", n1+n2);
system("pause");
return 0;
}
```



Universidad Nacional de la Matanza

Programación Inicial

**UNIDAD 4. ESTRUCTURA DE SELECCIÓN SIMPLE /
MÚLTIPLE**

INDICE

1	ESTRUCTURA DE SELECCIÓN SIMPLE (SENTENCIA IF)	2
2	CODIFICACIÓN EN C	8
3	OPERADOR CONDICIONAL (? :) [IF REDUCIDO]UAI	9
4	CASO PARTICULAR DE LA SENTENCIA IF	10
5	OPERADORES LÓGICOS.....	12
6	ESTRUCTURA DE SELECCIÓN MÚLTIPLE (SWITCH)	16

UNIDAD 4

Estructura selección simple / Múltiple

OBJETIVOS: Conocer en detalle y confeccionar programas utilizando las estructuras de selección simple y múltiple. Elaborar juegos de prueba que abarquen las distintas variantes de los programas. Conocer y utilizar los operadores lógicos y de comparación.

1 Estructura de Selección Simple (Sentencia if)

Esta estructura permite realizar la selección entre dos posibles cursos de acción, en base a la verdad o falsedad de una expresión que se escribe en forma de "expresión lógica".

Para escribir una expresión lógica es necesario utilizar "operadores de comparación". Estos operadores permiten evaluar una condición entre dos partes, ya sean dos variables, o una variable y una constante. Los operadores de comparación son los que se muestran en la tabla 1.

Tabla 1: Operadores de comparación

Operador	Símbolo
Mayor	>
Menor	<
Mayor o igual	>=
Menor o igual	<=
Igual	==
Distinto	!=

La representación gráfica de la estructura de selección simple puede verse en la Figura 1.

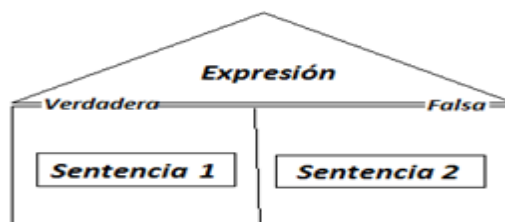


Figura 1: Representación gráfica de la estructura de selección simple

Por ejemplo, dada una variable A numérica, se puede escribir como expresión $A > 6$ en cuyo caso será verdadera ó falsa según el resultado de la evaluación del contenido ingresado en A. Si es verdadera, se ejecuta la "Sentencia 1", y si es falsa se ejecuta la "Sentencia 2". Dentro del verdadero y/o falso puede haber una o más sentencias.

El lenguaje C admite también que se utilice una expresión aritmética, tomando el resultado de la evaluación como "falso" si es igual a cero y verdadero por distinto de cero.

Ejemplo: si se evalúa $5-6+3$ esta expresión devuelve un valor distinto de cero, por lo cual, continua la secuencia por verdadero. En cambio, al evaluar $4+3-7$ esta expresión devuelve como resultado un valor igual a cero, por lo que la secuencia continua por el lado falso.

Se puede utilizar, si es que lo desea, solo la parte verdadera, o sea que las acciones por 'falso' son opcionales. Las expresiones que se utilizarán al principio serán simples, es decir, una relación entre dos valores y luego se ampliarán utilizando los operadores lógicos.

Inicial

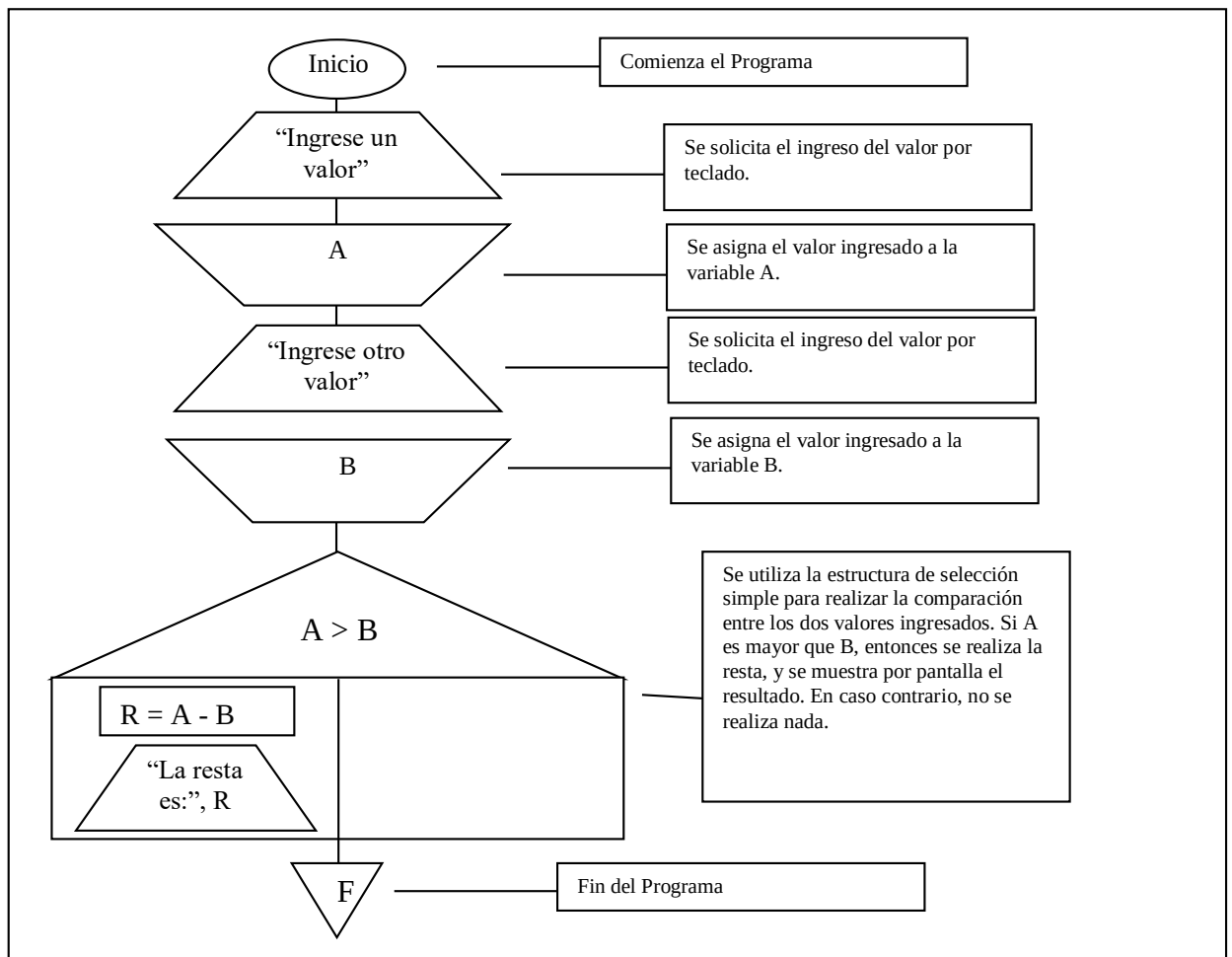
Estas estructuras se pueden enlazar en forma 'descendente', es decir, que dentro del verdadero y/o falso puede haber otras estructuras condicionales. Esto se verá en otros ejercicios más adelante. Esta estructura tiene un solo punto de entrada y uno solo de salida cualquiera fuese la alternativa ejecutada.

Ejemplo 1: Ingresar dos valores reales en dos variables, que se llamen A y B. Si A es mayor a B calcular e informar la diferencia entre A menos B.

ESTRATEGIA

1. Ingresar los dos valores en A y B
2. Si A es mayor que B calcular la diferencia
3. Informar la diferencia calculada si se cumplió la condición del punto 2.

DIAGRAMA:



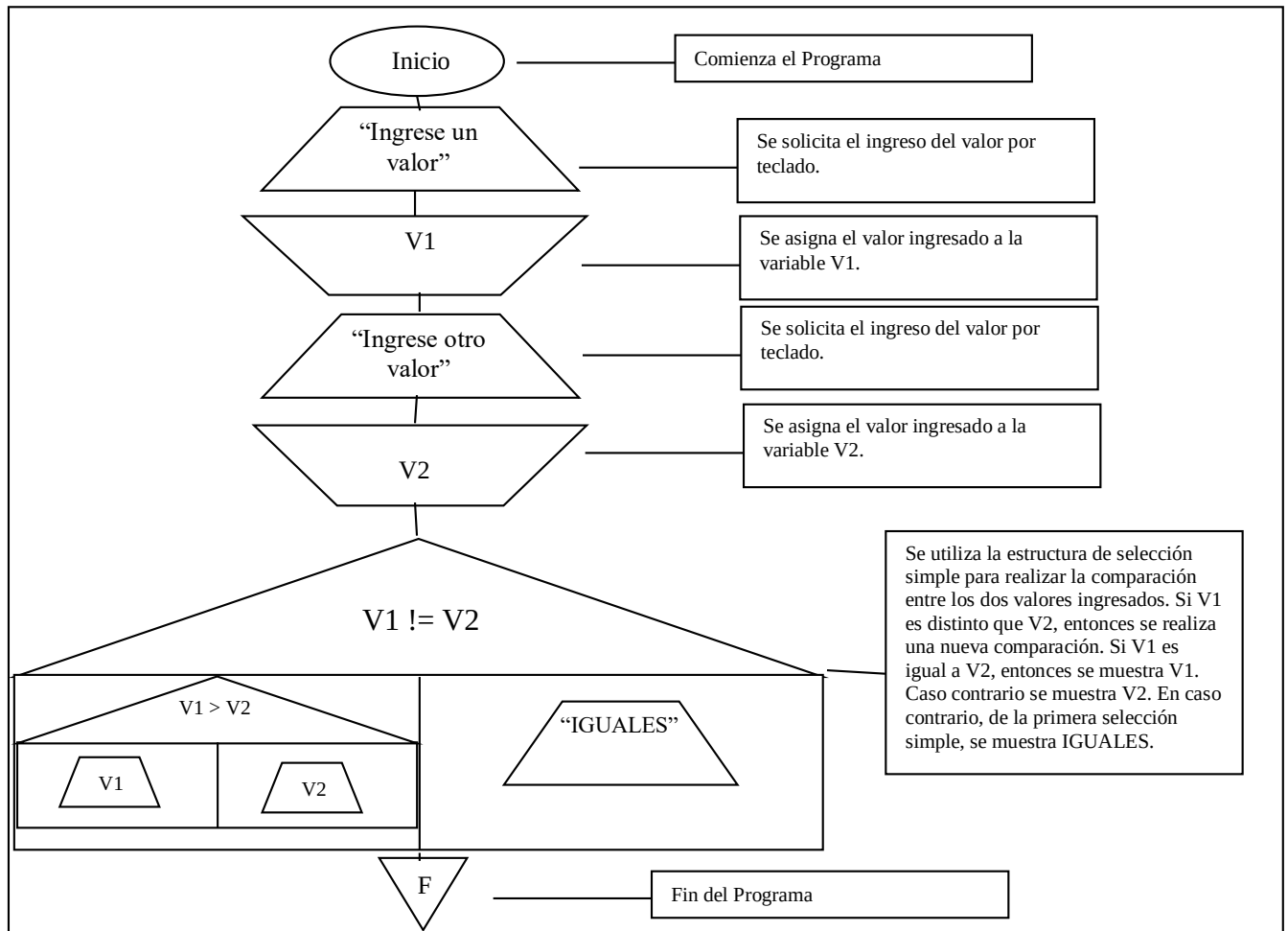
Ejemplo 2: Confeccionar un programa que ingrese dos valores numéricos y determine e informe al mayor de ellos si son distintos, o un mensaje que diga IGUALES en caso de serlo.

ESTRATEGIA

- 1] Ingresar los dos valores
- 2] Si no son iguales, determino el mayor
- 3] Informo el mayor ó el mensaje IGUALES

Inicial

DIAGRAMA:



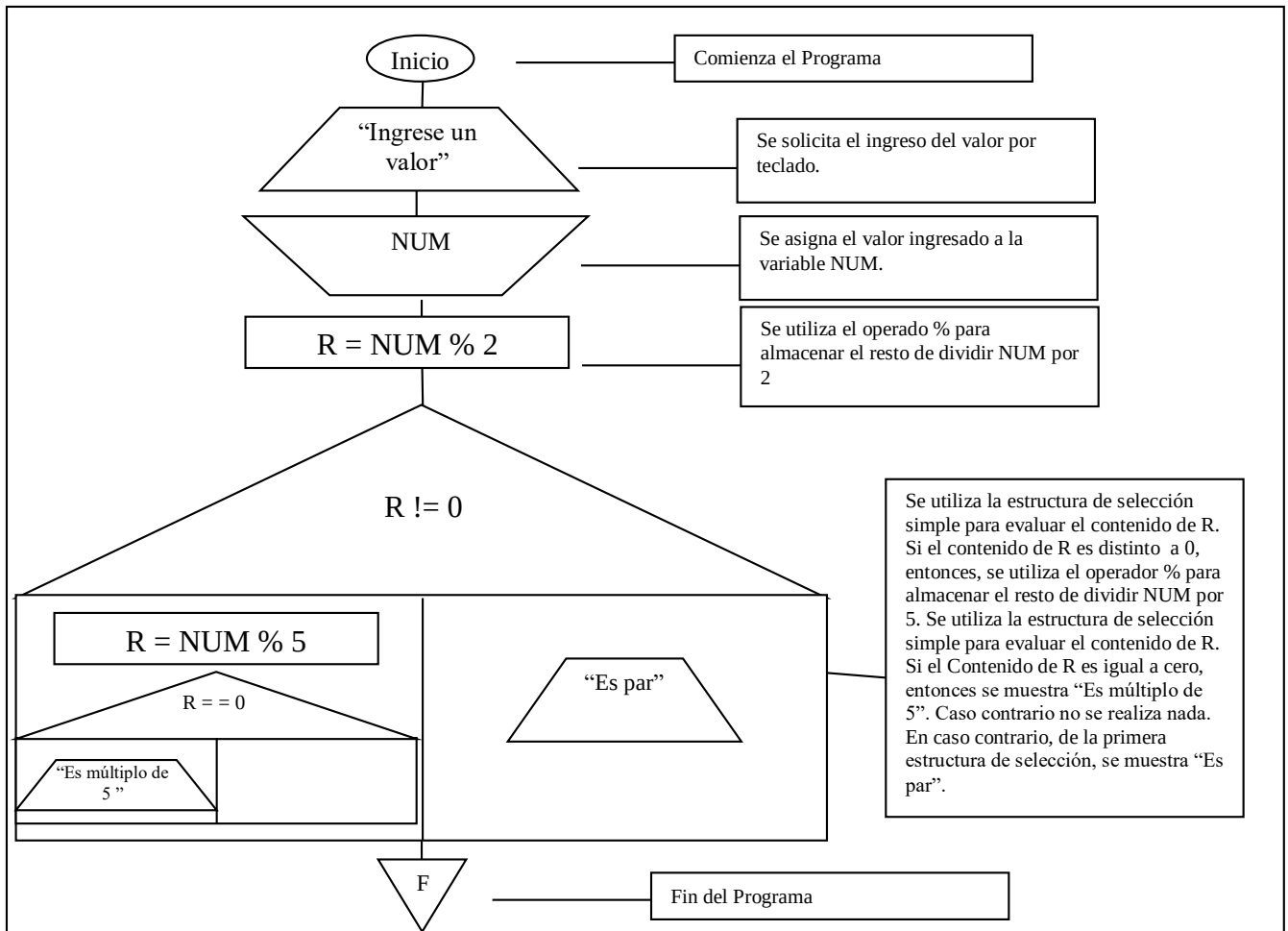
Ejemplo 3: Confeccionar un programa que pueda recibir un valor entero y nos informe si es un valor PAR, en caso de no serlo determinar e informar si dicho valor es múltiplo de 5.

ESTRATEGIA

1. Ingresar valor
2. Realizar operación para saber si es par, usando el operador % que almacena el resto.
3. Si R es igual a 0 informa "PAR"
4. De lo contrario, realiza operación para saber si es múltiplo de 5. Utilizando %
5. Si R es igual a 0 informa "MULT 5"

Inicial

DIAGRAMA:



VARIANTE: ¿Cómo se debería modificar el diagrama si se quiere detectar los pares que a su vez son múltiplos de 5?

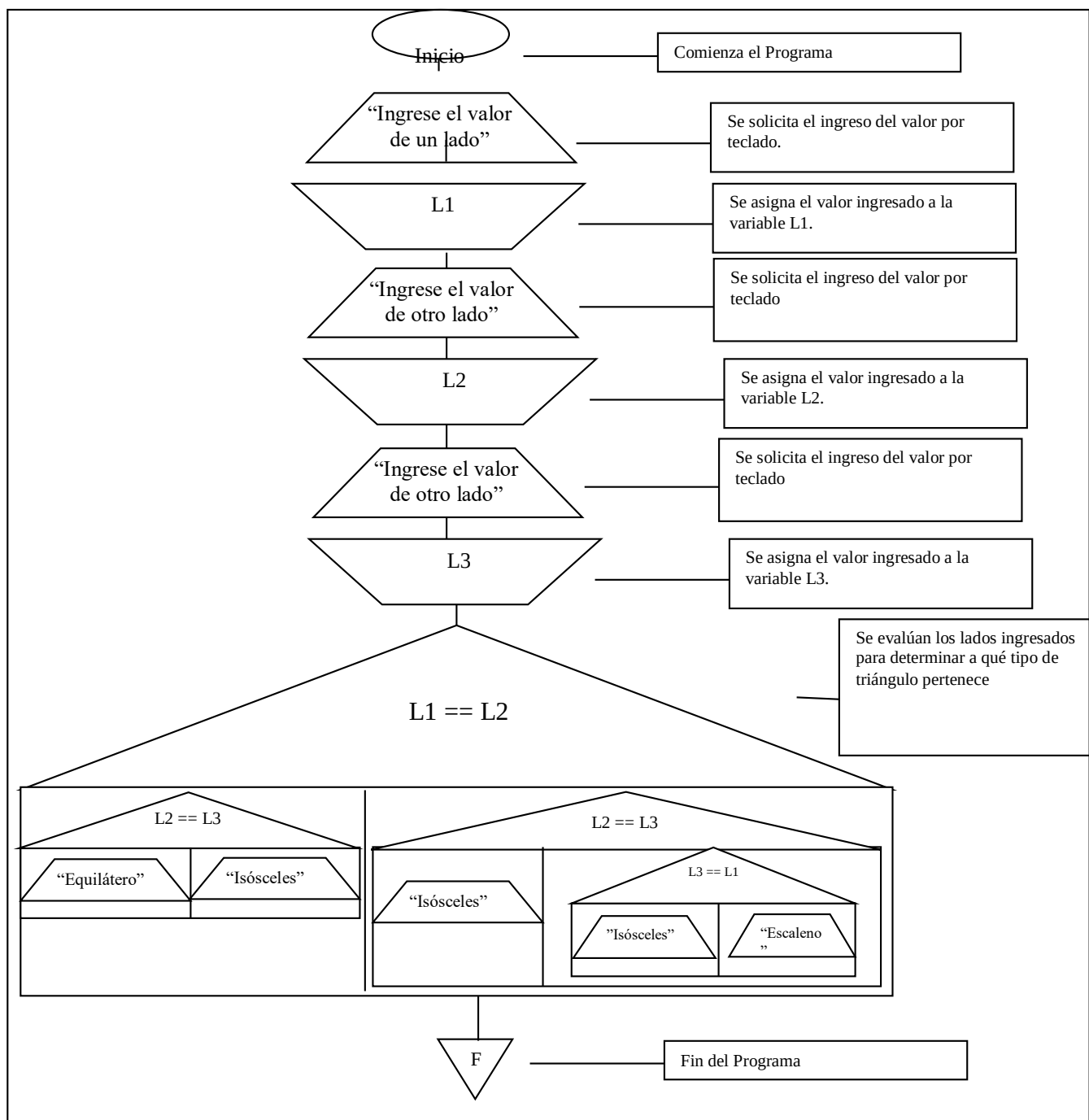
Ejemplo 4: Confeccionar un programa que ingresando los valores de los tres lados de un triángulo determine e informe si el triángulo es EQUILATERO, ISOSCELES ó ESCALENO.

ESTRATEGIA

1. Ingresar los valores de los 3 lados (L1, L2 y L3)
2. Determino si es equilátero (tres lados iguales), si no lo es, determino si es escaleno (tres lados distintos) y si o lo es, será isósceles (dos lados iguales y uno no).

DIAGRAMA

Inicial



La única forma que hay de verificar si un programa que se realiza funciona adecuadamente para las diversas alternativas que se pueden presentar, es confeccionando un JUEGO o LOTE de PRUEBA. Este consiste en una serie de datos con los cuales se ejecuta el programa efectuando un seguimiento de las sentencias tal cual lo efectuaría la computadora y se analizan los resultados obtenidos. Es importante que al diseñar un lote de prueba se tenga en cuenta todos los casos posibles y asegurarse que se ejecuten todos los caminos posibles del programa. La tabla 2 muestra un posible lote de prueba para el ejemplo anterior.

Tabla 2: Posible lote de prueba para determinar el tipo de triángulo

L1	L2	L3	Resultado esperado
5	5	5	Equilátero
5	5	7	Isósceles

Inicial

5	7	7	Isósceles
5	7	5	Isósceles
5	6	7	Escaleno

2 Codificación en C

La estructura de selección simple se codifica de la siguiente forma:

```

if ( Expresión Lógica )
{
    //instrucciones que se ejecutan si se cumple la condición es
    VERDADERA
}
else
{
    //instrucciones que se ejecutan si la condición es FALSA
}

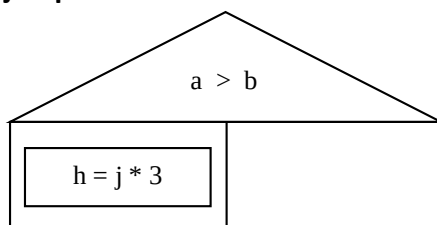
```

Puede verse que tanto para la parte verdadera como para la falsa habrá un bloque que contiene las instrucciones entre llaves. La utilización del bloque entre llaves solo es necesario si dentro del verdadero o del falso hay más de una instrucción. Si solo hay una instrucción las llaves son opcionales.

Si desea realizar una acción "solo" si se cumple la condición, se utiliza solo la parte verdadera. El uso del else es opcional, por lo tanto si no hay instrucciones por la parte falsa directamente no se escribe el else.

Es importante ser ordenados al escribir el código fuente para poder interpretarlo rápidamente al leerlo, para ello se utilizará lo que se conoce como indentación. La indentación es escribir el texto dejando espacio delante, se recomienda dejar ese espacio presionando la tecla tab (tabulación). En estas estructuras servirá para indicar cuáles son las instrucciones que están en la parte verdadera y cuales en la parte falsa, rápidamente en un golpe de vista. En los siguientes ejemplos se mostrará el código correctamente indentado.

Ejemplo 1:

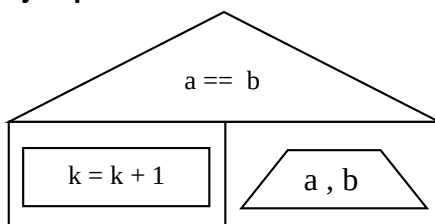


```

if (a > b)
    h = j * 3;

```

Ejemplo 2:

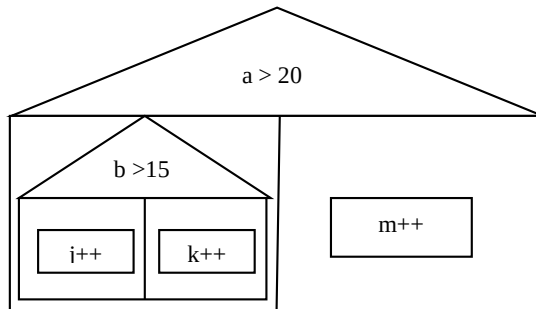


```

if ( a == b )
    k = k + 1;
else
    printf(" %f %f ", a, b);

```

Ejemplo 3:

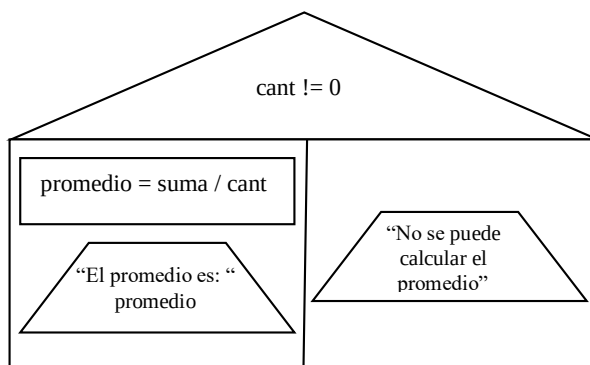


```

if (a>20)
    if (b>15)
        j++;
    else
        k++;
else
    m++;

```

Ejemplo 4:



```

if (cant!=0)
{
    promedio = suma/cant;
    printf ("el promedio es: %f", promedio);
}
else
    printf ("no se puede calcular el promedio");

```

3 Operador Condicional (? :) [if Reducido]

Forma general:

```

expresion? Instruccion1: instruccion2;

```

Si la expresión es verdadera se ejecuta la instrucción1, caso contrario se ejecuta la instruccion2.

Por ejemplo, el siguiente programa muestra si un número ingresado es par o impar utilizando el operador condicional.

```

#include <stdio.h>
int main()
{
    int n;
    printf("Ingrese un numero:");
    scanf("%d", &n);
    (n%2==0)?printf("par"):printf("impar");
    return 0;
}

```

Inicial

El mismo programa utilizando la sentencia if quedaría:

```
#include <stdio.h>
int main()
{
    int n;
    printf("Ingrese un numero:");
    scanf("%d", &n);
    if (n%2==0)
        printf("par");
    else
        printf("impar");
    return 0;
}
```

Solo sirve para el caso de que se tenga una única instrucción en el verdadero y una única instrucción en el falso. Se utiliza generalmente para asignar uno de dos valores a una variable. Por ejemplo, realizar un programa que ingrese dos números y calcula la resta si el primero es mayor o igual al segundo y sino calcule la suma.

```
#include <stdio.h>
int main()
{
    int n1,n2, resultado;
    printf("Ingrese dos numeros:");
    scanf("%d%d",&n1,&n2);
    resultado = (n1>=n2)?n1-n2:n1+n2;
    printf ("El resultado es: %d", resultado);
    return 0;
}
```

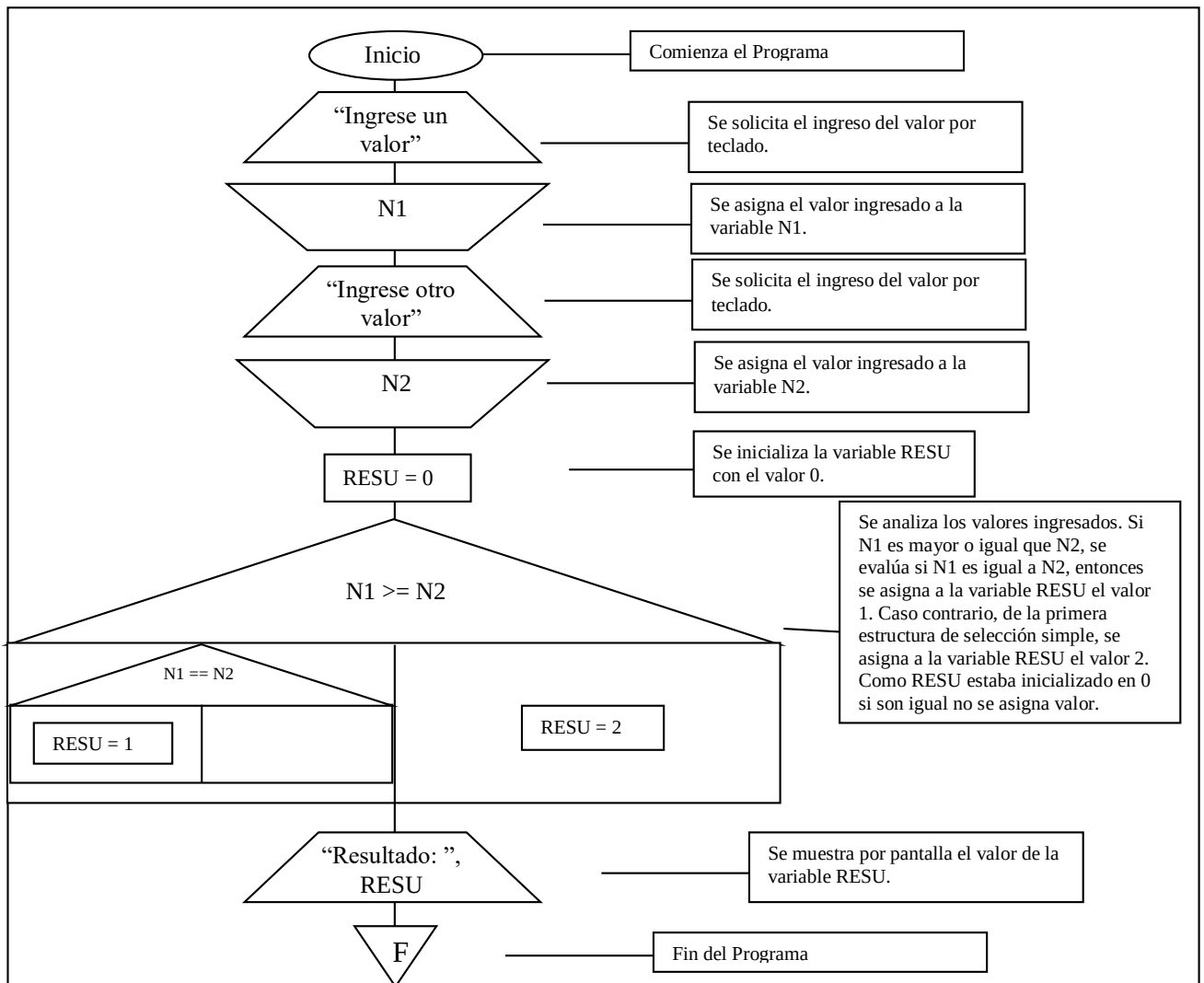
4 Caso Particular de la sentencia if

Este caso se presenta cuando una estructura "completa" tiene en su rama "verdadera" a una estructura de selección "incompleta". La solución para este caso llega con un par de llaves que encierren a la estructura de selección incompleta, transformándola en una estructura completa.

Recuerde también que cada estructura se caracteriza por tener un "único" punto de entrada y un "único" punto de salida, lo que equivale a hablar de una "sola" sentencia. La causa es debido a que por regla general "cada else siempre es apareado con el inmediato if anterior no cerrado"

Inicial

Ejemplo: realizar un programa que permita ingresar dos números y de cómo resultado 0 si el primero es mayor que el segundo, 1 si son iguales, 2 si el segundo es mayor que el primero.



Recuerde que siempre para resolver un ejercicio hay muchas soluciones posibles, seguramente esta no es la mejor forma de plantearlo pero sirve para ejemplificar el caso particular que se quiere mostrar ya que dentro del verdadero del primer if hay un segundo if incompleto (sin else), pero el segundo if si tiene su parte falsa, por lo tanto, si se codifica de la siguiente forma:

```

#include <stdio.h>
int main()
{
    int n1,n2, resu=0;
    printf("Ingrese dos numeros:");
    scanf("%d%d",&n1,&n2);
    if (n1>=n2)
        if (n2==n1)
            resu=1;
    else
        resu=2;
    printf ("Resultado: %d",resu);
    return 0;
}

```

Inicial

Al probarlo si se ingresa, por ejemplo, el valor 10 para el primero número y 2 para el segundo, se debería obtener como resultado un 0 ya que el primer número es mayor al segundo, sin embargo, muestra un 2 ya que el código está mal hecho.

Es incorrecto ya que el else será tomado como parte del if inmediatamente anterior. Para solucionarlo se debe armar un bloque para completar la secuencia if incompleta de la siguiente forma:

```
if (n1>=n2)
{
    if (n1==n2)
        resu =1;
}
else
    resu =2;
```

Otra forma de resolverlo sería incluyendo un else vacío de la siguiente forma:

```
if (n1>=n2)
    if (n1==n2)
        resu =1;
    else;
else
    resu =2;
```

5 Operadores Lógicos

Los operadores lógicos permiten asociar varias expresiones lógicas ó aritméticas formando solo una, lo cual simplificará la cantidad estructuras de selección que se deben utilizar.

Los operadores lógicos son 3:

- AND (y)
- OR (ó)
- NOT (negación)

En el lenguaje C, se utilizan los símbolos de la tabla 3 para expresar los operadores:

Tabla 3: Operadores lógicos en el lenguaje C

Operador	Símbolo
AND	& &
OR	
NOT	!

Estos operadores trabajan sobre valores lógicos, por lo tanto, pueden aplicarse sobre condiciones o sobre valores numéricos de una variable recordando que el 0 es falso y cualquier valor distinto de cero es verdadero.

El operador AND (&&) evalúa la dos condiciones y retorna verdadero solo en el caso de que ambas condiciones sean verdaderas, es decir, se deben cumplir ambas al mismo tiempo.

El operador OR (||) evalúa las dos condiciones y retorna verdadero cuando al menos una de las dos es verdadera, es decir, se debe cumplir al menos una de las condiciones para que sea verdadera la expresión.

Inicial

El operador NOT (!) trabaja sobre una sola condición negándola. Es decir, que si es verdadera la hace falsa y si es falsa la hace verdadera.

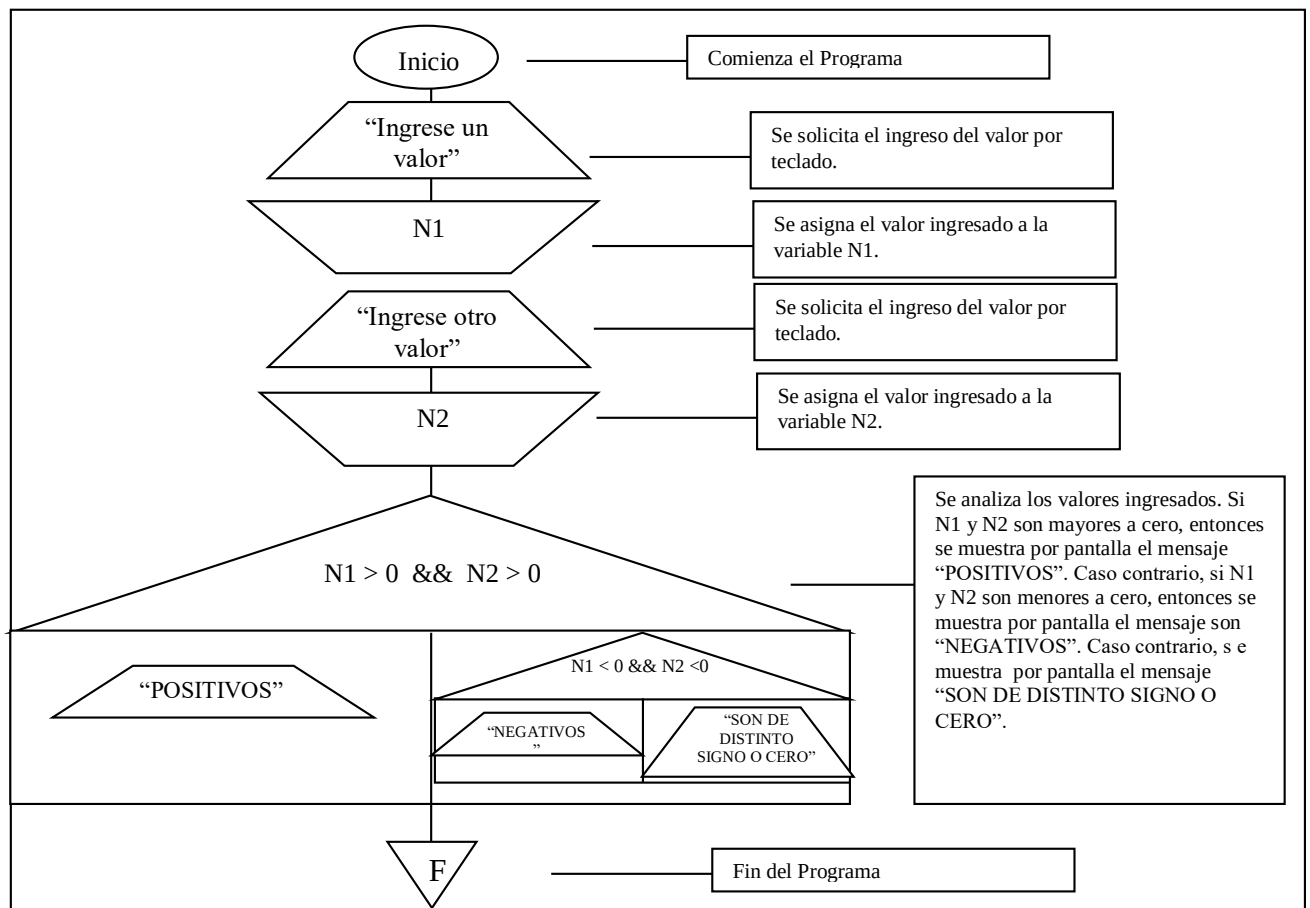
La tabla 4 muestra la tabla de verdad de los operadores lógicos, siendo A y B condiciones lógicas.

Tabla 4: Tabla de verdad de los operadores lógicos

A	B	A && B	A B	!A
F	F	F	F	V
F	V	F	V	V
V	F	F	V	F
V	V	V	V	F

Las precedencias de los operadores son: NOT (!) - AND (&&) - OR (||)

Ejemplo 1: Ingresar dos números. Indicar si ambos son positivos, ambos son negativos, o son distinto signo o cero.

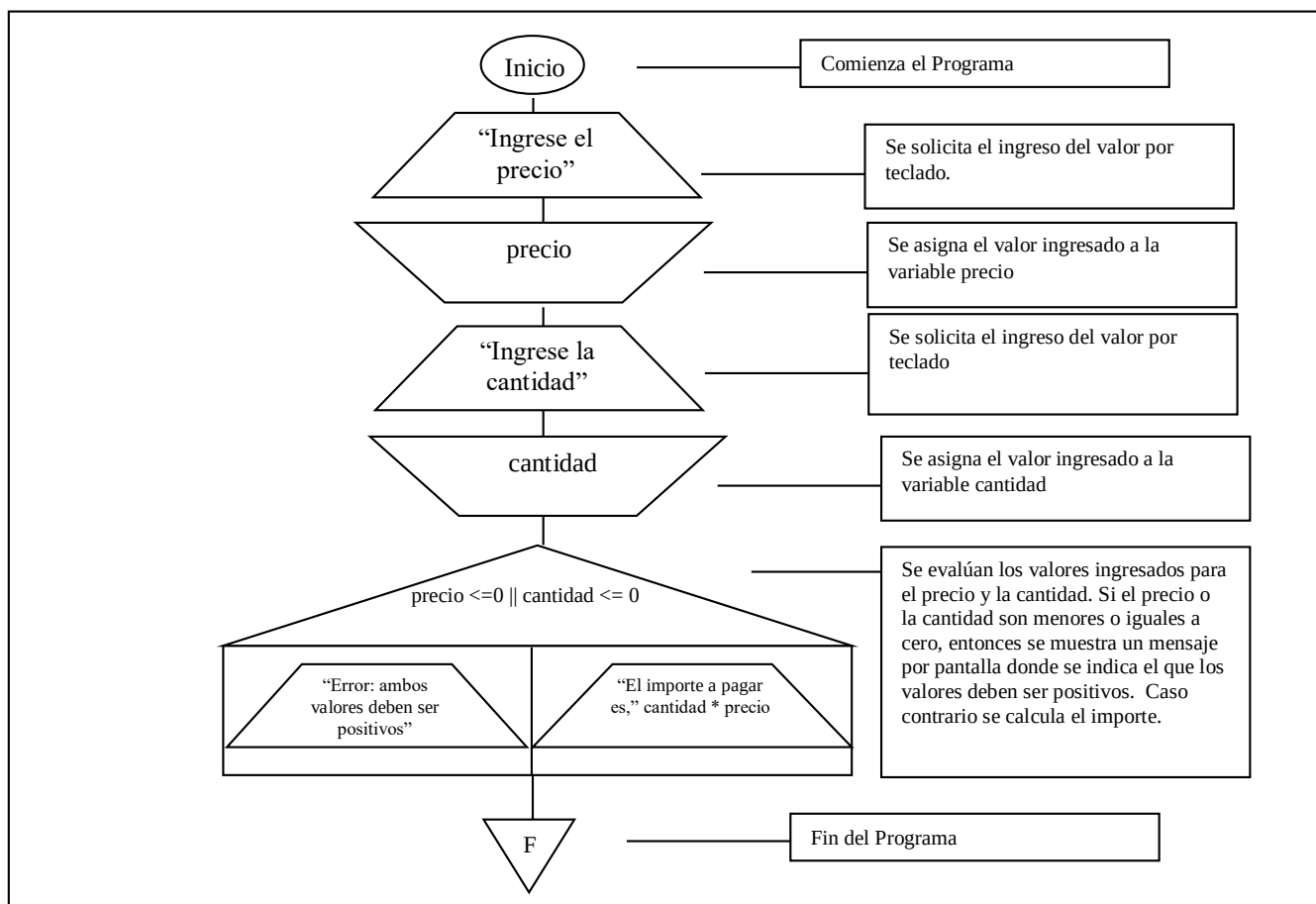


Esta es solo una de las formas de resolver este ejercicio ya que otra persona podría comenzar preguntando por ejemplo si ambos son negativos, o si son cero o de distinto signo. Recordemos que siempre hay varias formas para resolver un mismo ejercicio.

ACLARACIÓN IMPORTANTE: El lenguaje C evalúa las condiciones en orden, si detecta que ya se cumple o no, no sigue evaluando las otras condiciones. Otros lenguajes evalúan todas las condiciones por más que ya se sepa el resultado.

Por lo tanto, en este ejemplo si en N1 se ingresa un número negativo o cero y al ser la primera condición un AND, la misma ya es falsa y no necesita evaluar el valor de N2.

Ejemplo 2: Ingresar precio y cantidad de productos a vender. Mostrar el importe a abonar. Si alguno de los dos es menor o igual a cero informar un error.

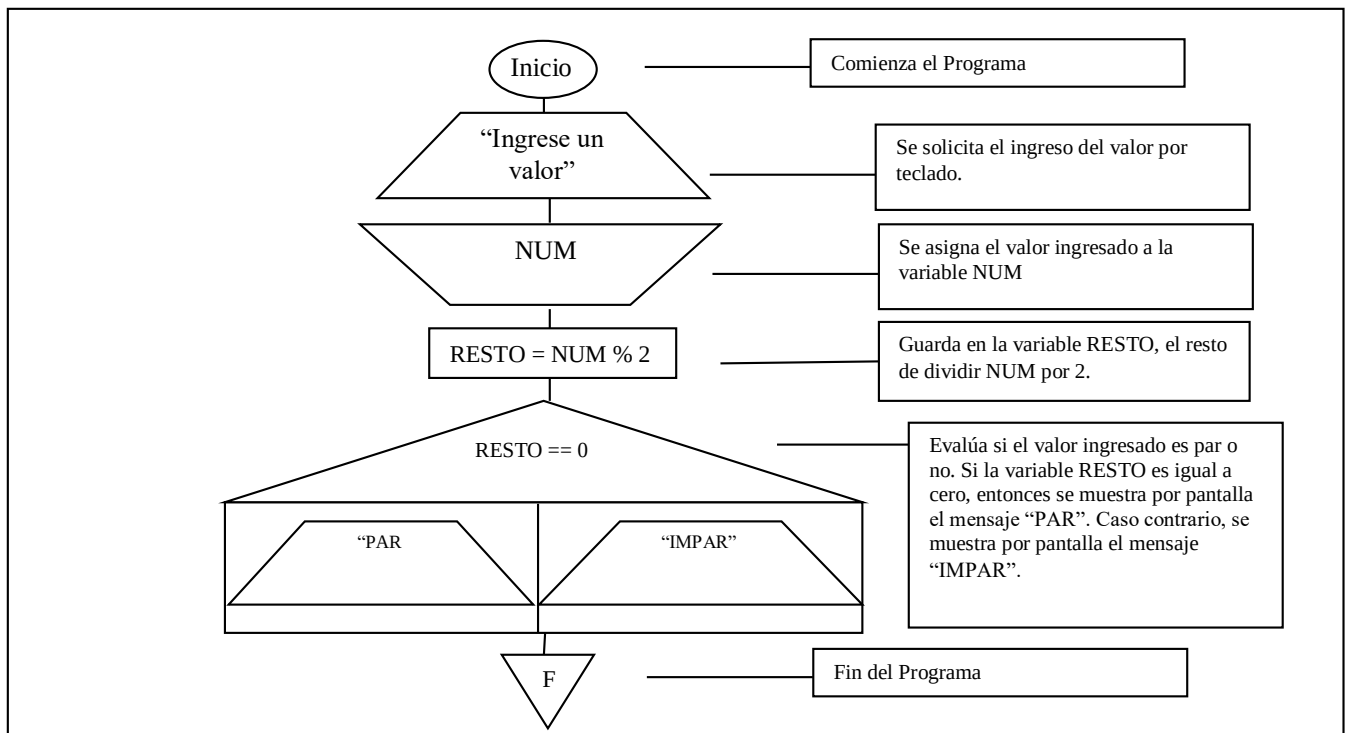


Nuevamente hay que tener en cuenta cómo evalúa las condiciones el lenguaje C. En la primera condición si precio es menor o igual a cero al ser un OR ya la condición es verdadera, y por lo tanto, no evalúa el valor de cantidad.

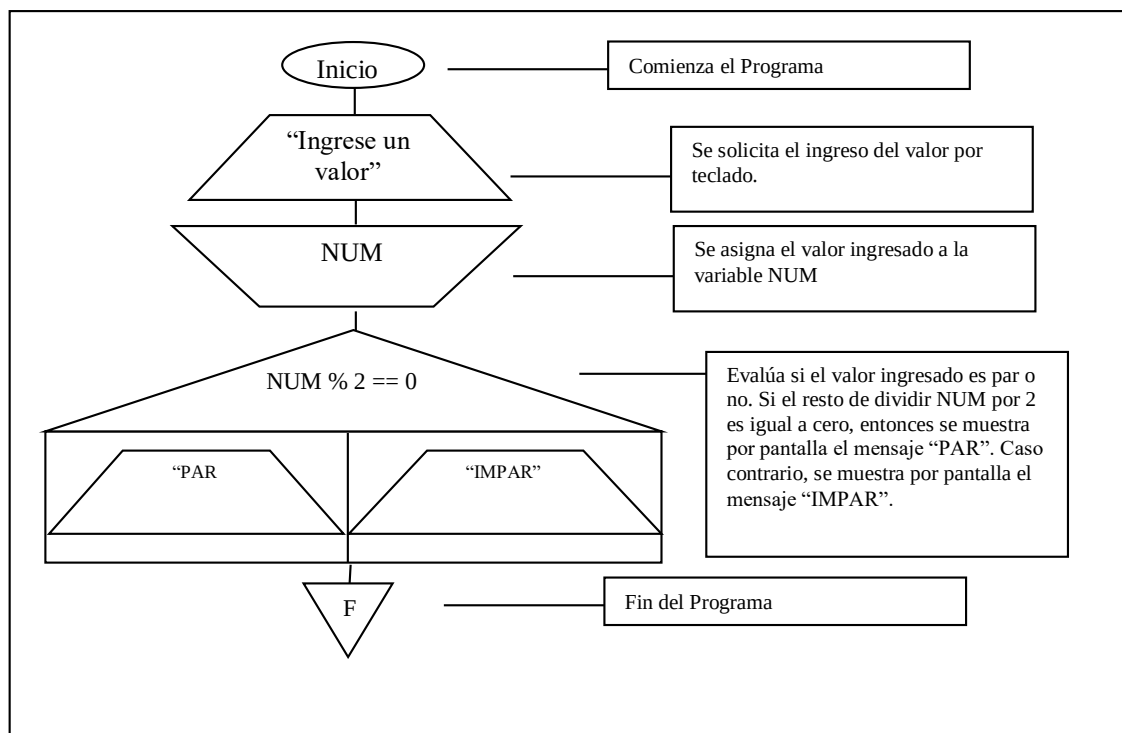
Ejemplo 3: Ingresar un número e indicar si es par o impar.

Una forma de resolver este ejercicio es la siguiente:

Inicial



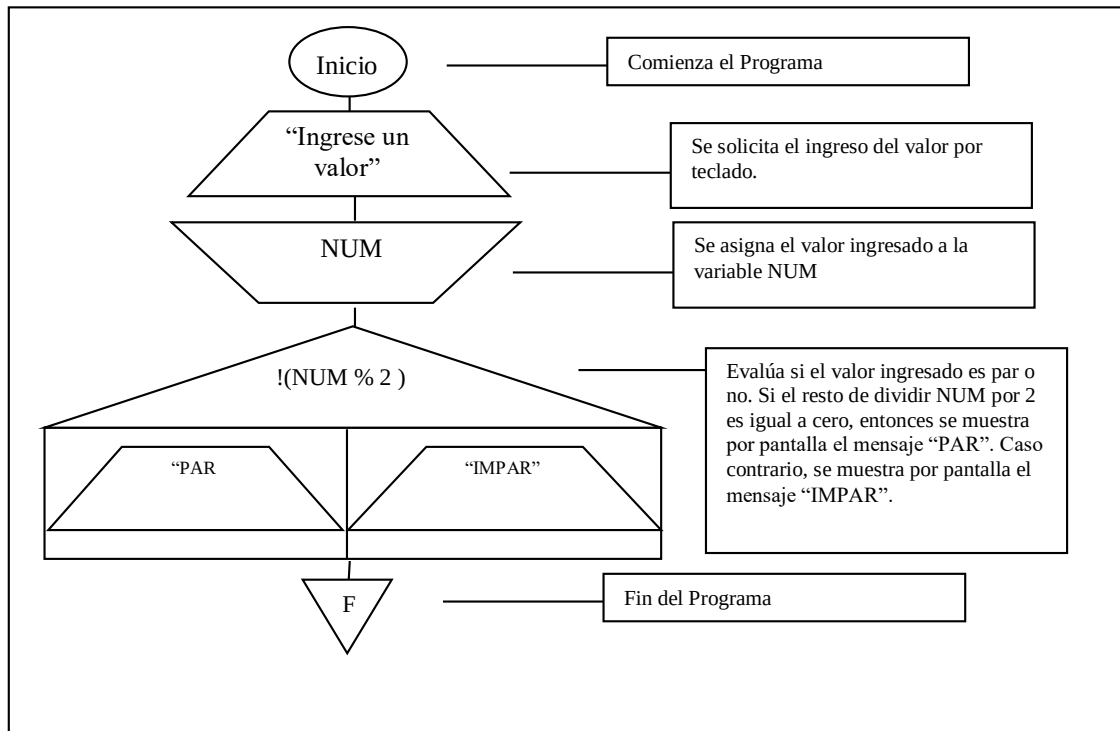
Tambi n se puede quitar la variable resto y hacer la pregunta directamente en la condici n de la siguiente manera:



Si se toma en cuenta que en el lenguaje C el 0 es falso y cualquier valor distinto de cero es verdadero se puede directamente evaluar el resultado de la operaci n sin utilizar un comparador. Si la

Inicial

operación $\text{NUM} \% 2$ da como resultado 0 será entonces falsa, si se quiere hacer verdadera cuando sea cero se debe negar quedando de la siguiente forma:

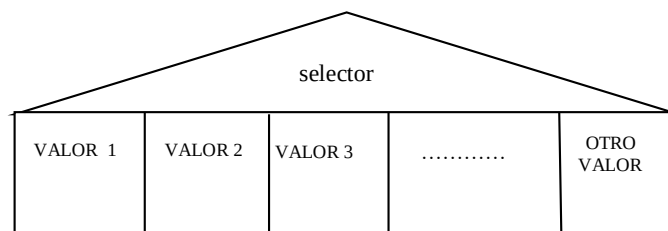


6 Estructura de Selección Múltiple (switch)

Piense como sería un programa que ingresando el número de un mes tiene que informar el nombre. Utilizando la sentencia `if` se tienen que anidar las condiciones preguntado si el mes es 1 mostrar enero, sino preguntar si el mes es dos mostrar febrero, sino preguntar si el mes es 3, y así sucesivamente, hasta abarcar todas las condiciones.

Para casos de este tipo existe una "estructura condicional múltiple", que generaliza a la sentencia `<if-else>`. Esta estructura es útil, por ejemplo, cuando hay un problema en el cual se deben realizar acciones diferentes en base al valor de un código que puede adoptar diversos valores, lo cual obliga a utilizar diversas sentencias `if` encadenadas.

Su diagrama y codificación son los siguientes:



```

switch (selector)
{
    case valor1:
        //instrucciones a ejecutar si selector toma el valor1
        break;

```

Inicial

```
    case valor2:
        //instrucciones a ejecutar si selector toma el valor1
        break;
    case valor3:
        //instrucciones a ejecutar si selector toma el valor1
        break;
    ...
    default: //instrucciones a ejecutar si selector no toma ninguno de los valores
anteriores
}
```

Consideraciones:

- Se evalúa el selector y si su valor coincide con alguno de los casos establecidos se pasa directamente a ejecutar el código de dicho caso
- El default es el valor por defecto, es opcional y se ejecuta si selector no toma ninguna de los valores anteriores.
- Selector puede ser una variable o una expresión pero debe ser del tipo **entera** o **carácter**.
- Los valores deben ser constantes (numéricas o caracteres). **NO pueden ser rangos**. Si se necesita evaluar rangos se debe utilizar la sentencia if.
- Cada caso termina con la instrucción break que hace que la ejecución se corte y salga de la estructura del switch. Si no se coloca el break en un caso seguirá ejecutando el código de los casos que están debajo hasta encontrar un break o llegar a la llave de fin. Es por esto que el default, o si el default no está, el último caso no necesita de la instrucción break. Esta particularidad permite que distintos casos compartan el mismo código.

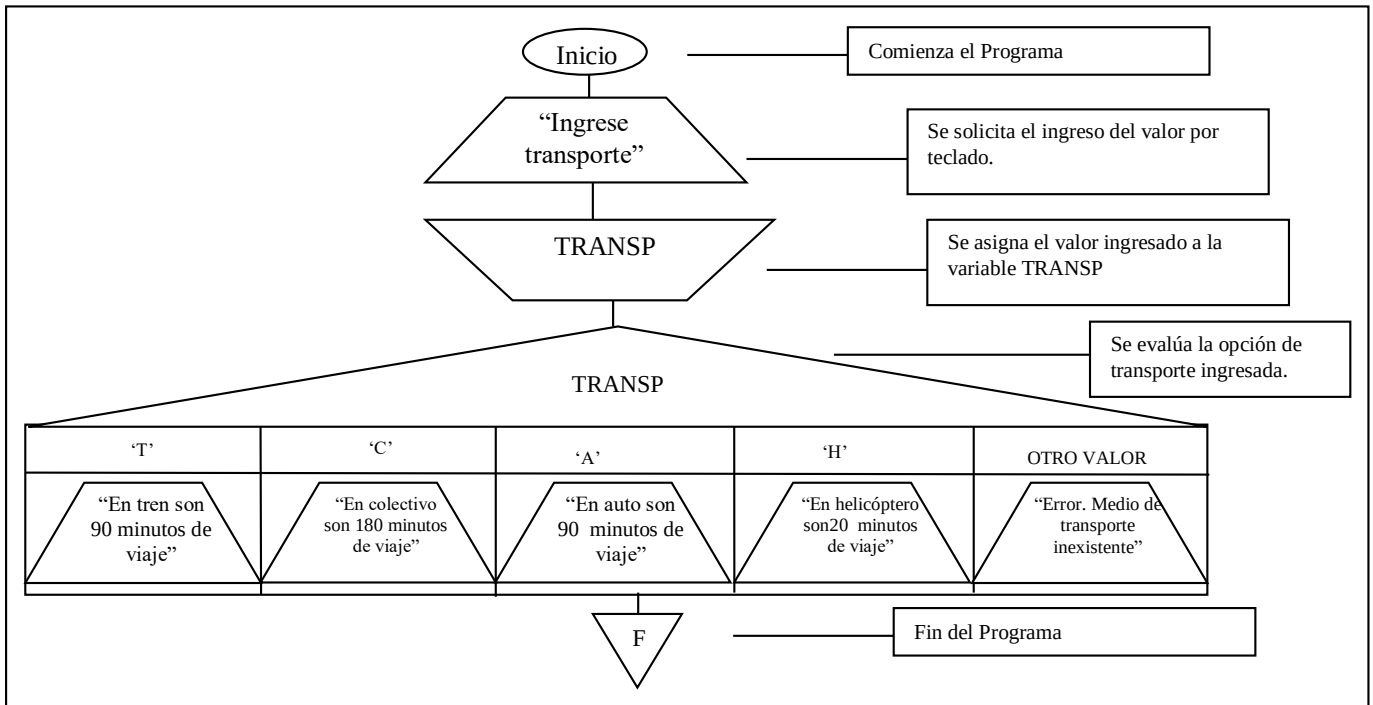
Los casos no tiene necesidad de estar ordenados, pero cada valor solo puede aparecer una vez.

Ejemplo 1: Confeccionar un programa que solicite el medio de transporte para llegar desde La Plata hasta la UNLaM. Mostrando en pantalla el tiempo estimado de viaje para cada medio de transporte. El medio de transporte se ingresa con una letra siendo:

- T: tren, tiempo estimado de viaje 90 min.
- C: colectivo, tiempo estimado de viaje 180min

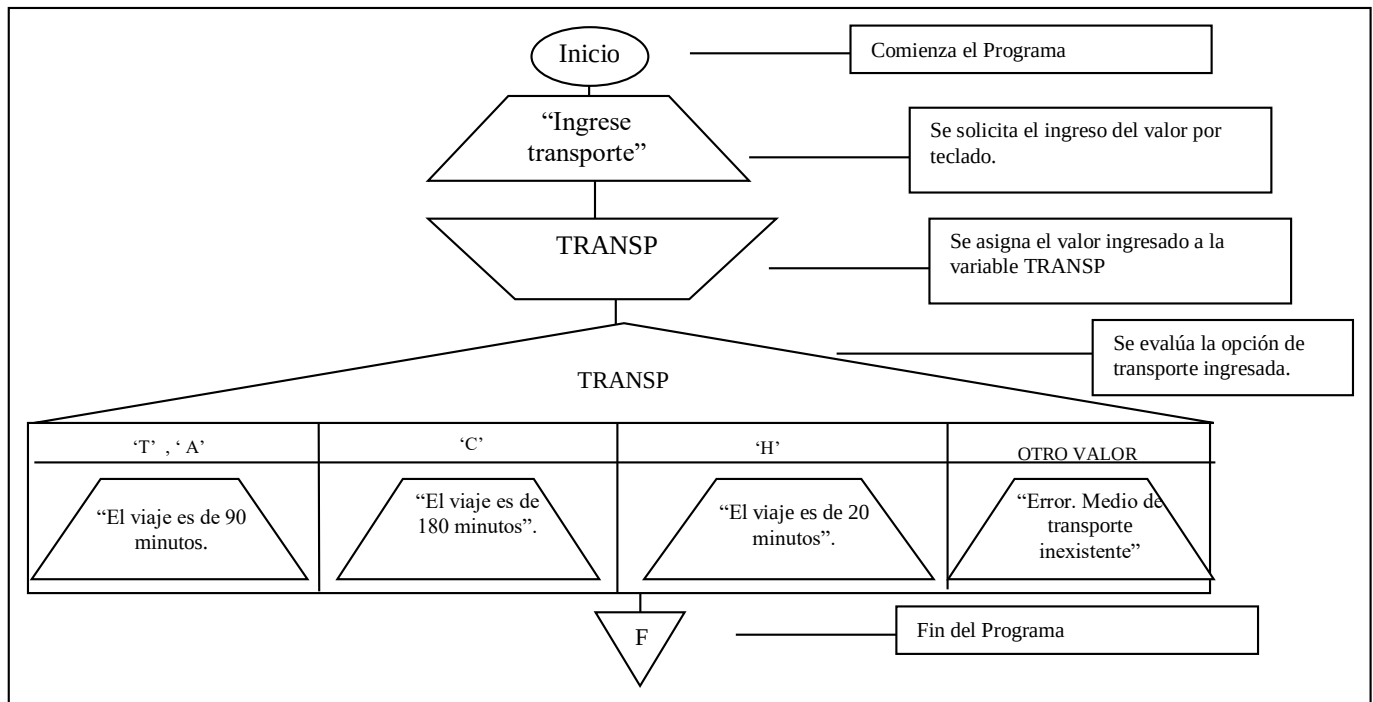
Inicial

- A: auto, tiempo estimado de viaje, 90 min.
- H: helicóptero, tiempo estimado de viaje, 20 min.



Nótese que los tiempos en tren y auto son los mismos.

Entonces el algoritmo anterior se puede mejorar de la siguiente manera.



El diagrama quedará de la siguiente manera en código:

```
#include <stdio.h>
int main()
{
    char tansp;
    printf("Ingrese transporte:");
    scanf("%c",&transp);

    switch(transp)
    {
        case 'T':
        case 'A': printf("El viaje es de 90 minutos.");
                break;
        case 'C': printf("El viaje es de 180 minutos.");
                break;
        case 'H': printf("El viaje es de 20 minutos.");
                break;
        default: printf("Error. Medio de transporte
inexistente.");
    }

    return 0;
}
```

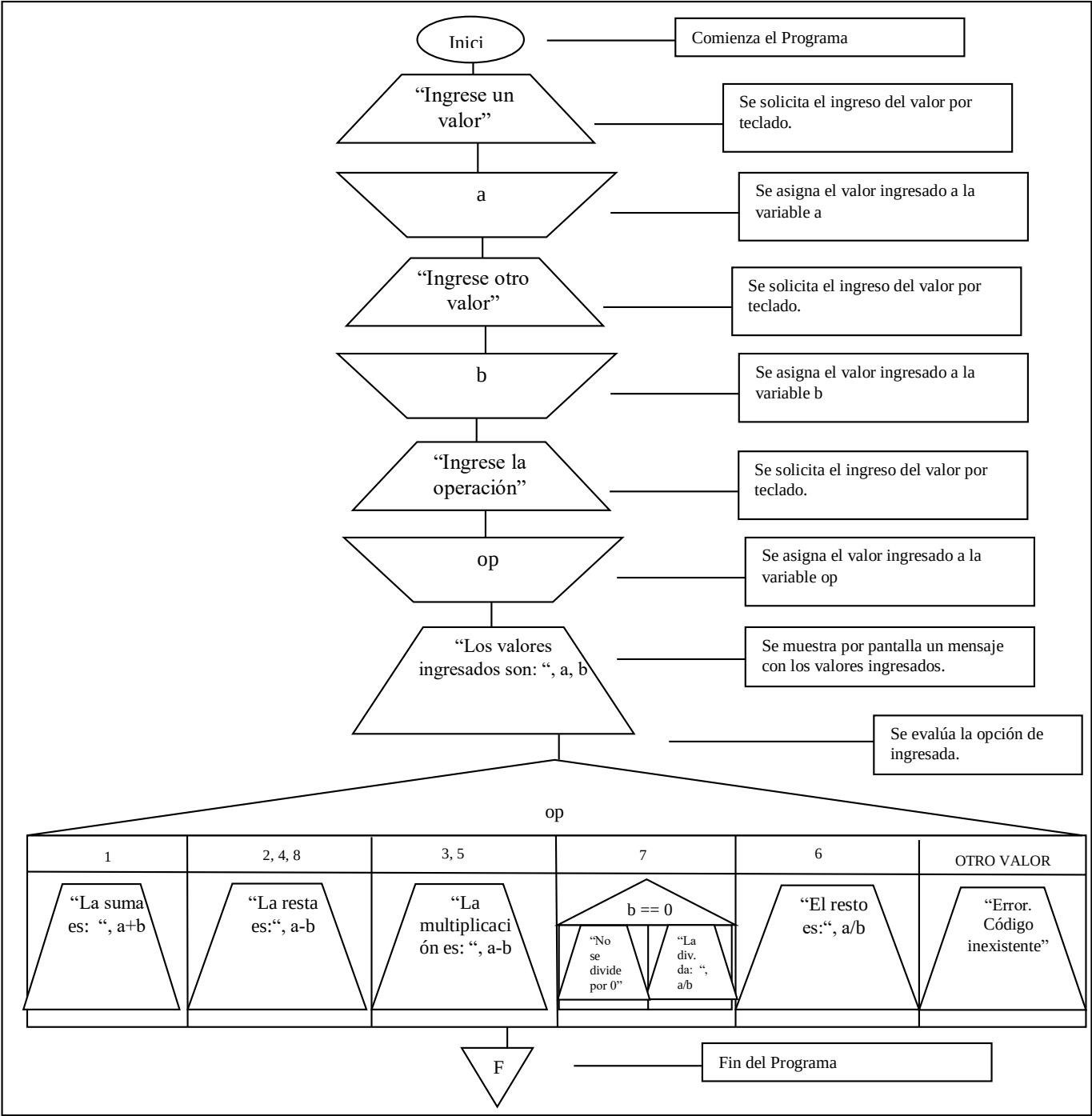
Ejemplo 2: Confeccionar un programa que permita ingresar un par de valores enteros a y b, que corresponden a las temperaturas registradas en un día y un código que indica la operación que se debe realizar con dichos valores.

El código de operación puede variar entre 1 y 8. Según el valor del código se pide calcular e informar:

Valor código	Calcular
1	$a + b$
2, 4 <u>ú</u> 8	$a - b$
3 ó 5	$a * b$
7	a / b
6	$a \% b$
< 1 <u>ó</u> > 8	rechazar

Por cada código ingresado imprimir los valores leídos y el resultado obtenido, con leyendas.

Inicial



El diagrama quedará de la siguiente manera en código:

```
#include <stdio.h>
int main()
{
    int a,b,op;

    printf("Ingrese un valor:");
    scanf("%d",&a);

    printf("Ingrese otro valor:");
    scanf("%d",&b);

    printf("Ingrese la opción:");
    scanf("%d",&op);

    printf("Los valores ingresados son:%d %d",a,b);

    switch(op)
    {
        case 1: printf("La suma es: %d",a+b);
                break;
        case 2:
        case 4:
        case 8: printf("La resta es: %d",a-b);
                break;
        case 3:
        case 5: printf("La multiplicación es: %d",a*b);
                break;
        case 7: if(b==0)
                    printf("No se puede dividir por cero");
                else
                    printf("La división es: %d",a/b);
                break;
        case 6: printf("El resto de la división es: %d",a%b);
                break;
        default: printf("Error. Código inexistente.");
    }

    return 0;
}
```



Universidad Nacional de la Matanza

Programación Inicial

UNIDAD 5. ITERACION

INDICE

1. ITERACIÓN O REPETICIÓN	2
2. ESTRUCTURAS DE ITERACIÓN DEFINIDA.....	2
3. CONTADORES Y ACUMULADORES.....	5
4. LENGUAJE C: ESTRUCTURAS DE ITERACIÓN DEFINIDA	6
5. MÁXIMOS Y MÍNIMOS	8
6. ESTRUCTURAS DE ITERACIÓN DEFINIDA ANIDADAS	10
7. ESTRUCTURAS DE ITERACIÓN CONDICIONADA	11

UNIDAD 5

Iteración

Objetivos: Aplicar en la solución de los problemas las estructuras iterativas, en sus variantes. Realizar programas con algoritmos de búsqueda de máximos y mínimos. Comprender los conceptos de variables acumuladoras y contadoras. Utilizar las estructuras de iteración para validar los datos de entrada.

1. Iteración o repetición

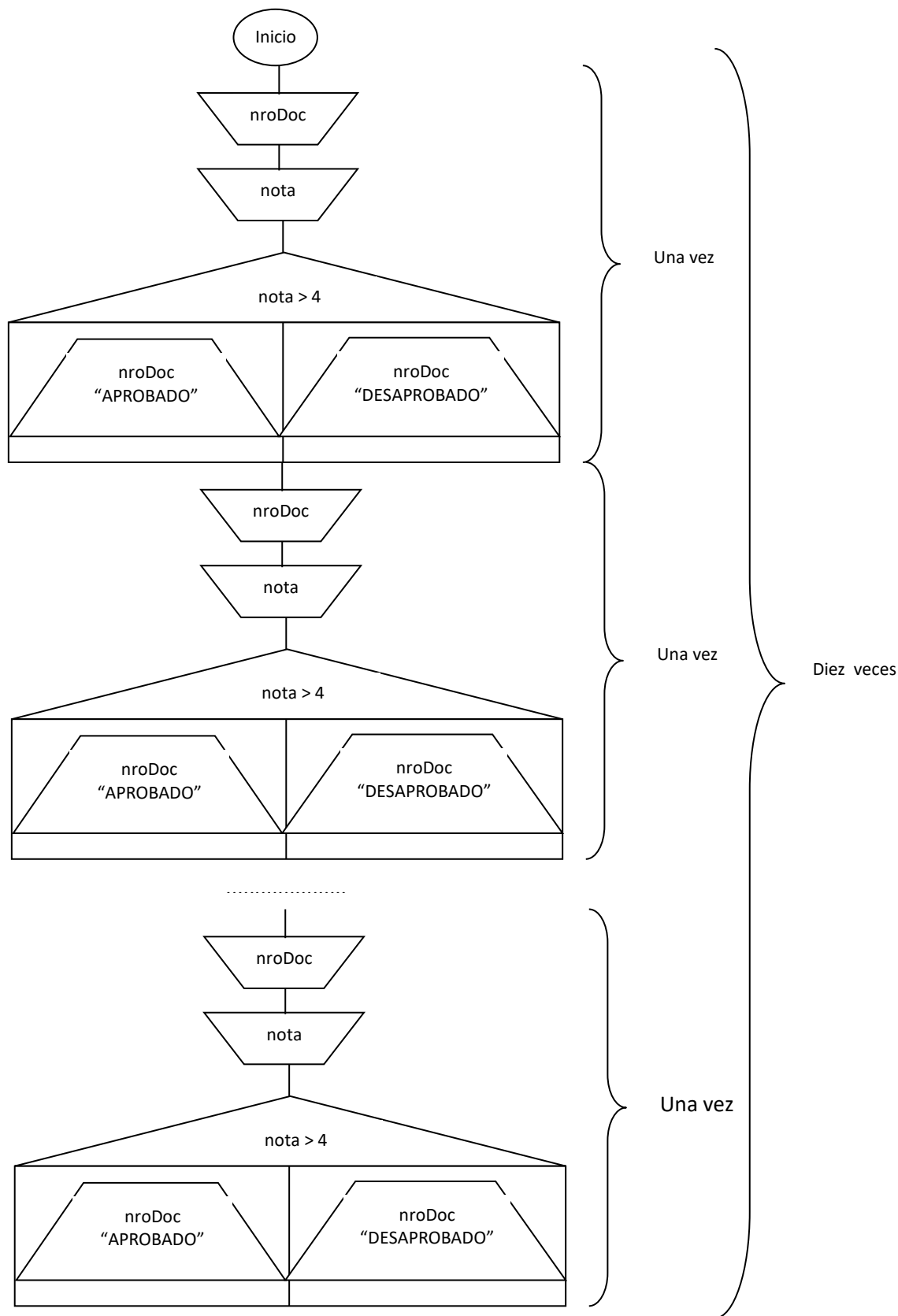
En los programas vistos hasta ahora cada instrucción se ejecutaba una sola vez, en el orden que aparece en el programa. Pero a veces es necesario repetir la ejecución de un grupo de instrucciones, para lo cual se van a utilizar las estructuras de repetición o iteración.

Existen dos tipos de ciclos de repetición:

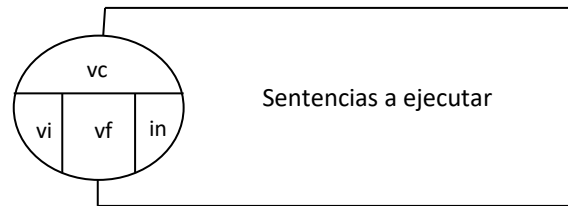
- a) ITERACIÓN DEFINIDA: Cuando se conoce de antemano la cantidad “exacta” de veces que se debe repetir ese proceso o grupo de sentencias.
- b) ITERACIÓN CONDICIONADA: Cuando NO se conoce la cantidad de iteraciones a efectuar, es decir, que la repetición depende del cumplimiento de cierta condición.

2. Estructuras de iteración definida

Se desarrolla el siguiente ejemplo: Ingresar número de documento y nota de los 10 alumnos de un curso, informar por cada uno el número de documento y la leyenda “APROBADO” o “DESAPROBADO” si la nota es mayor a 4 o no.



Analizando el diagrama se observa que un grupo de instrucciones se repiten 10 veces. Para eliminar las repeticiones se debe escribir las instrucciones repetidas una sola vez y efectuar un

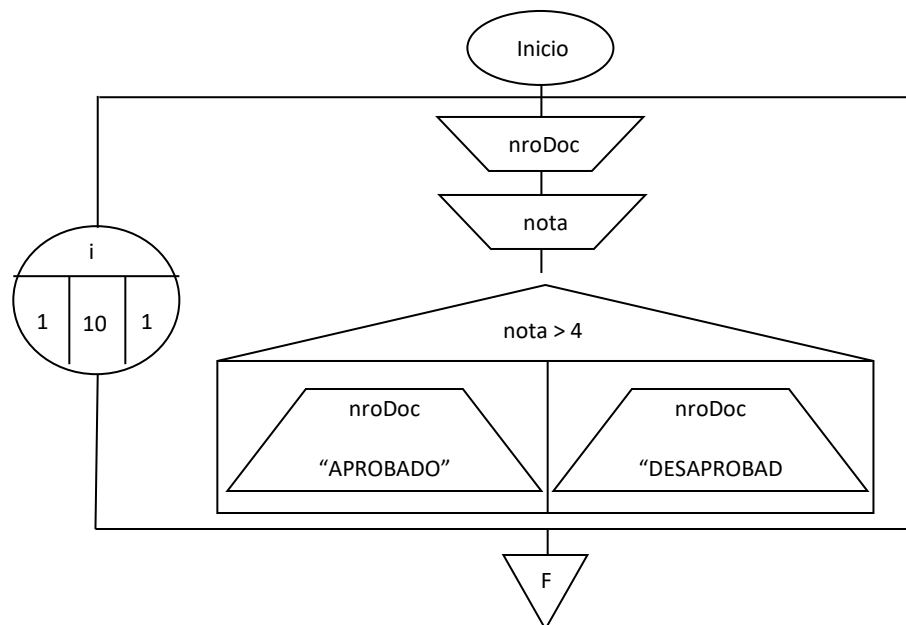


mecanismo para realizar la repetición 10 veces. Para ello, se debe agregar en el diagrama el ciclo de repetición definida:

Donde:

- **vc**: es la variable de control, ya que con ella se controla el ciclo.
- **vi**: es el valor inicial que toma la variable de control.
- **vf**: es el valor final que debe tomar la variable.
- **in**: incremento que se le aplica a la variable cada vez que se ejecuta el ciclo.

Funcionamiento: Se asigna a la variable de control el valor inicial, se compara con el valor final, si es menor o igual que éste, se ejecutan las sentencias dentro del ciclo, luego incrementa a la variable con el valor del incremento, si sigue siendo menor o igual al valor final realiza otra ejecución y así hasta que llegue a superar el valor final.



Se aplica esta estructura al problema anterior:

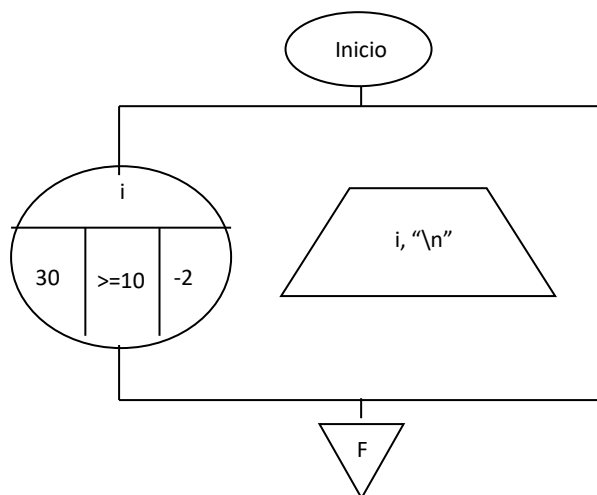
El ciclo de iteración definida no siempre necesariamente debe iniciar en 1 ni ser creciente, por ejemplo, para repetir 10 veces un proceso también se puede:

- Poner como valor inicial el 10
- Poner como valor final la condición > 0
- Poner -1 como el valor de incremento, generando de esta forma un decremento

Observe entonces que en realidad el valor final no es un número sino una condición, que se denomina condición de permanencia dentro del ciclo. Si esa condición se cumple entonces el ciclo se repite. Si no se escribe el operador de comparación se toma por defecto el $\leq$ (menor o igual) en cualquier otro caso se debe escribir el operador de comparación.

Tampoco el incremento o decremento debe ser siempre 1 o -1 sino que puede ser cualquier otro valor que modifique la variable de control.

Ejemplo: Realizar un programa para mostrar los números pares entre 30 y 10 en forma descendente.



3. Contadores y acumuladores

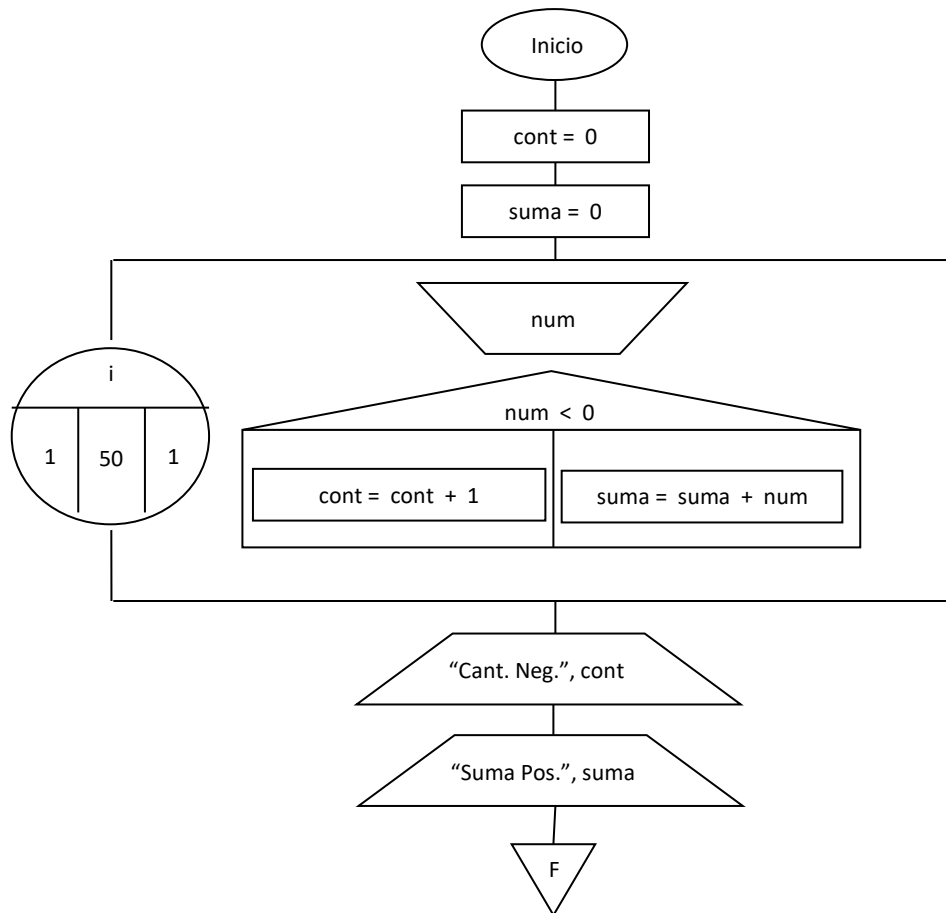
Un contador es una variable en la memoria que se incrementará en una unidad cada vez que se ejecute el proceso. El contador se utiliza para llevar la cuenta de determinadas acciones que se pueden solicitar durante la resolución de un problema. Hay que realizar la inicialización del contador o contadores. La inicialización consiste en poner el valor inicial de la variable que representa al contador (generalmente con el valor 0).

Un acumulador es una variable que suma sobre sí misma un conjunto de valores, para de esta manera tener la suma de todos ellos en una sola variable. De igual forma, se pueden efectuar decrementos en un acumulador.

La diferencia entre un contador y un acumulador es que mientras el primero va aumentando de uno en uno, el acumulador va aumentando o decrementando en una cantidad variable.

Ejemplo: Ingresar 50 números, informar la cantidad de números negativos y la sumatoria de los positivos.

Se deben ingresar los números en un ciclo de repetición definido. Por cada número se tiene que consultar si es menor que cero o no. Si es menor que cero se cuenta utilizando un contador (incrementando el contador en uno), si no se suma en un acumulador (sumarle al acumulador ese número).



4. Lenguaje c: estructuras de iteración definida

El ciclo de repetición definida se codifica en lenguaje C con el siguiente formato:

```

for (expresión de inicialización; expresión de verificación; expresión de
incremento)
{
    sentencial1;
    sentencial2;
    .....
}
  
```

Las llaves son opcionales de haber una sola sentencia.

Ejemplo: Se escribe en lenguaje C el Ejemplo anterior que ingresa 50 números, cuenta y acumula:

```

#include <stdio.h>
int main( )
{
    int cont, suma, i, num;
    cont = 0;
    suma = 0;
    for ( i = 1; i <= 50; i++)
    {
        printf("\n Ingrese un numero: ");
        scanf("%d", &num);
    }
}
  
```

```
        if (num < 0)
            cont = cont + 1;
        else
            suma = suma + num;
    }
    printf("La cantidad de numeros negativos es %d \n", cont);
    printf("La sumatoria de los numeros positivos es %d \n",
suma);
    return 0;
}
```

Para incrementar el contador, en lugar de `cont = cont + 1` se podría usar el operador de autoincremento `cont++`. En el uso del acumulador, se podría usar el operador de acumulación `suma += num`.

OBSERVACIONES IMPORTANTES:

- La prueba de la condición de final se efectúa siempre al inicio del bucle.
- El valor de la variable que actúa como índice, en el lenguaje C, puede ser modificado dentro del ciclo, pero se recomienda NO hacerlo ya que se modifica el comportamiento de la estructura de repetición definida.
- Finalizado el for, el valor del índice queda con el valor de la última ejecución realizada más el valor del incremento, o sea el valor con el cual no pudo ejecutar otra pasada.
- La expresión del incremento puede ser del tipo `in = in + x`, puede contener variables que no figuran como parámetros del for.
- Tener la precaución de "no" colocar un ; (punto y coma) luego de los paréntesis del for, lo cual hace ignorar el bucle, generando un ciclo vacío.
- Pueden omitirse una o todas las expresiones del for, pero NO los ";;"
 - a) Si se omite la primera expresión, NO SE ASIGNA VALOR INICIAL dentro del ciclo, pero puede asignarse antes:

Ejemplo:

```
j = 1;
suma = 0;
for ( ; j <=10 ; j++ )
    suma += j; // Suma los números del 1 al 10.
```

- b) Si se omite la segunda, se genera un ciclo INDEFINIDO, porque al no estar la expresión 2, se establece que la prueba siempre será verdadera y por lo tanto nunca saldrá del ciclo.

Ejemplo: `for (j=1; ; j++)`

- c) Si se omite la tercera, se puede realizar el incremento dentro del ciclo for. NO SE RECOMIENDA realizar esta operación ya que es poco clara. Siempre el mejor utilizar todas las expresiones del for.

Ejemplo

```
j = 1;
suma = 0;
for ( ; j <=10 ; )
{
    suma += j; // Suma los números del 1 al 10.
    j++;
}
```

5. Máximos y mínimos

Es un **problema** común querer encontrar el mayor o el menor valor de una determinada serie de valores. Si se quiere encontrar el mayor valor, se debe pensar igual a un trabajo manual con una pila de hojas donde cada una tiene un valor numérico. Tomo el primero y lo retengo en la mano izq. (variable **mayor**), tomo al segundo con la mano derecha (variable **dato**) y lo comparo con el de la izquierda, si es más grande descarto el de la mano izquierda y paso el de la derecha a la izquierda; se procede igual con los restantes valores, quedando finalmente el mayor en la mano izquierda o sea, en la variable mayor.

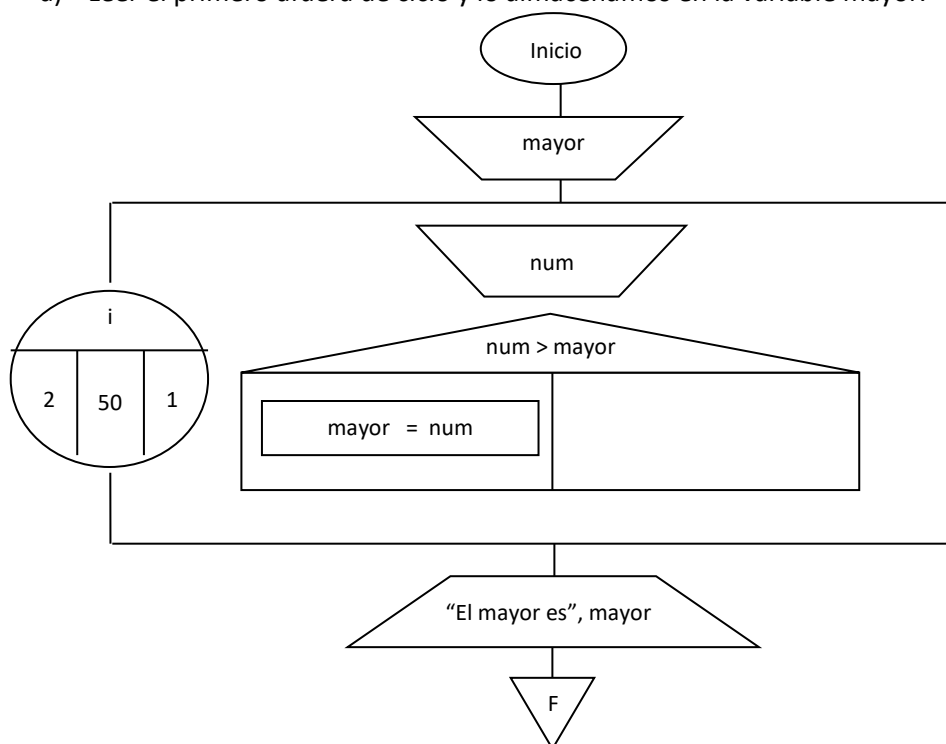
Es decir, que para calcular un máximo de un conjunto de valores se toma el primer valor como referencia, siendo este momentáneamente el más grande. Luego al ingresar otro número se compara con el que se tiene de referencia. Si el nuevo número es mayor al de referencia entonces se reemplaza por el recién ingresado, de esta forma siempre quedará como valor de referencia el más grande y se comparará con cada uno de los número ingresados. Si se desea buscar el valor mínimo el procedimiento es exactamente igual pero se compara si el número recién ingresado es menor al de referencia en cuyo caso se lo reemplaza.

Ejemplo: Ingresar 50 valores numéricos y determinar e informar el mayor.

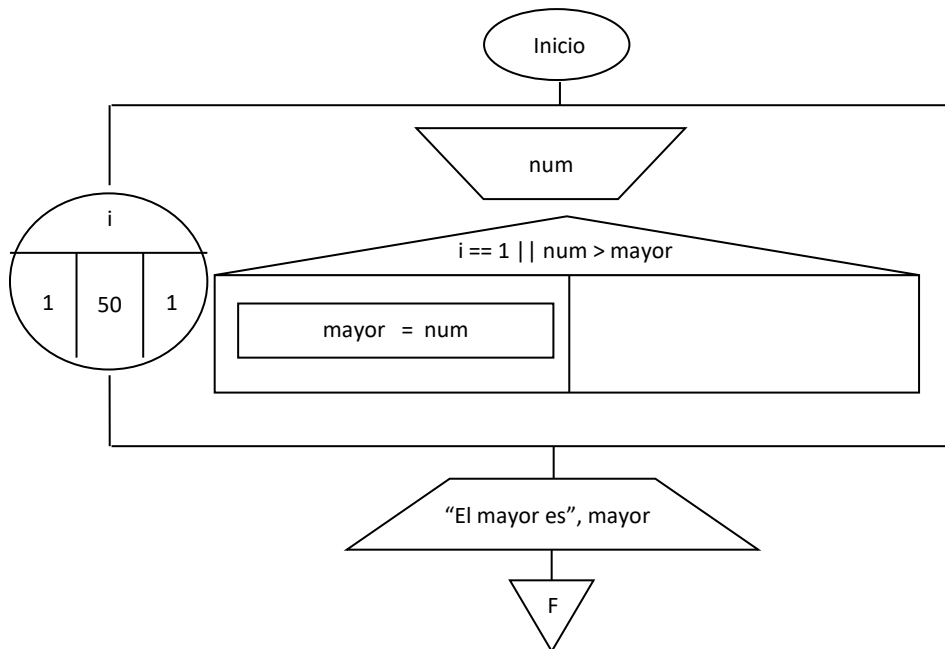
Se puede realizar de 3 formas:

- Leer el primero afuera del ciclo
- Una pregunta para separar el primero del resto.
- Usar una señal para separar el primero del resto.

a) Leer el primero afuera de ciclo y lo almacenamos en la variable mayor.

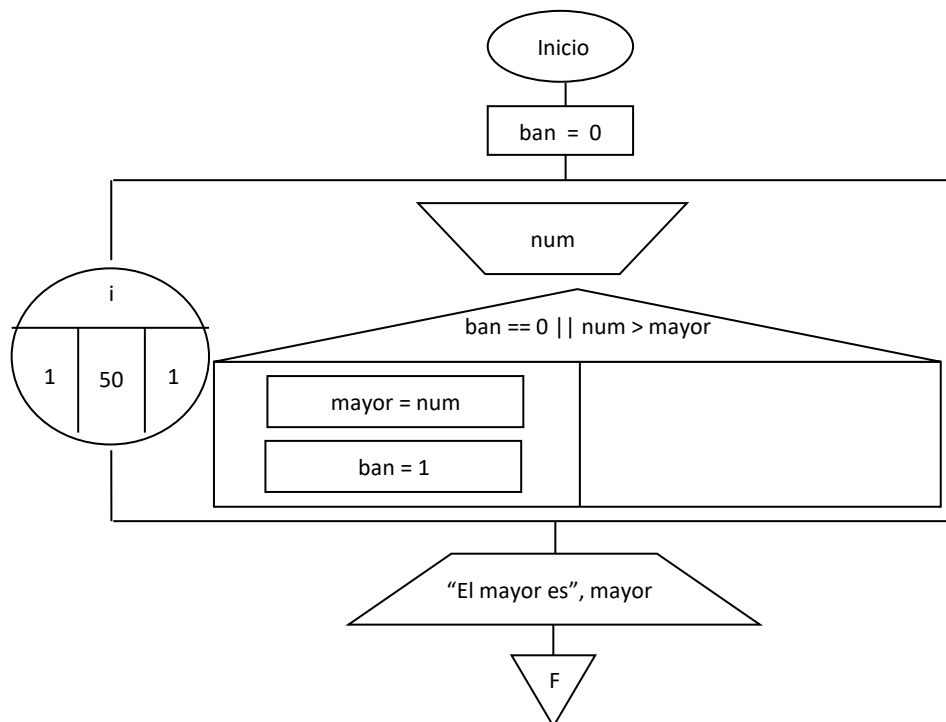


b) Una pregunta para separar el primero del resto, para darle valor inicial a la variable mayor.



En este ejemplo se realiza un solo if con dos condiciones, se pregunta si es la primera vez que se ingresa al ciclo, condición que se da cuando *i* vale 1. Luego esa condición será siempre falsa y por lo tanto se evalúa si el número recién ingresado es más grande que el guardado en la variable *mayor*. Sin embargo la primera vez que se ingresa la variable *mayor* aún no tiene valor asignado pero como el lenguaje C no necesita evaluar todas las condiciones, la primera vez que se ingresa al ciclo la primer condición es verdadera y al ser un or con una condición que sea verdadera es suficiente y por lo tanto no se evalúa la segunda condición.

- c) Una pregunta para separar el primero del resto, para darle valor inicial a la variable *mayor*, utilizando una señal.



Una bandera o señal cumple con la función de tener el conocimiento de que un evento haya sucedido o no, es una variable a la cual se le asigna solo dos valores, uno se le asigna al inicio del proceso y el otro, al suceder el evento buscado.

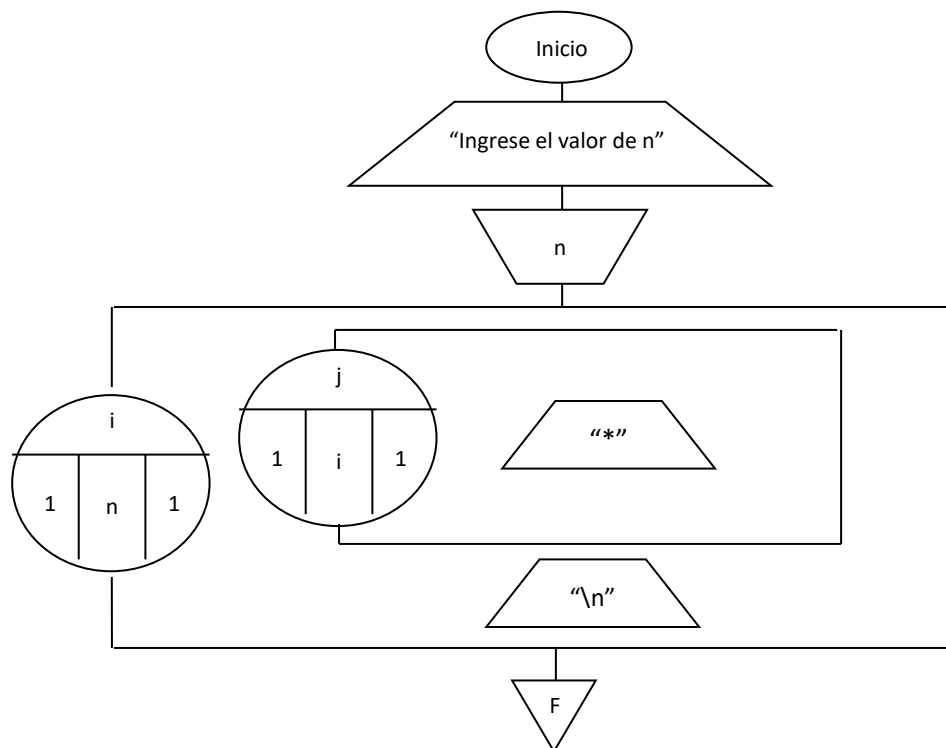
6. Estructuras de iteración definida anidadas

Las estructuras **de** control iterativas pueden anidarse, es decir, incluir a un ciclo completo dentro de otro. Las estructuras interior y exterior no necesitan ser del mismo tipo, pero es "esencial" que la estructura interior esté "totalmente" incluida en la exterior, no pueden "solaparse". Estamos analizando "for anidados", destacando que pueden ser 2 o más y que las variables utilizadas como control en los ciclos deben ser distintas.

Ejemplo 5: Hacer un algoritmo que imprima un triángulo rectángulo de * con base y altura de n cantidad de *. El valor de n debe ser ingresado por teclado.

Por ejemplo si n es 4, debe imprimir:

```
*
**
***
****
```



En código C:

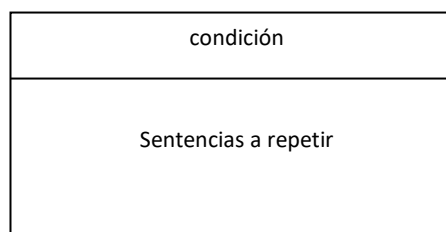
```
#include <stdio.h>

int main()
{
    int n, i, j;
    printf("Ingrese el valor de n \n");
```

```
scanf("%d",&n);
for(i=1; i<=n; i++)
{
    for(j=1; j<=i; j++)
    {
        printf("*");
    }
    printf("\n");
}
return 0;
}
```

7. Estructuras de iteración condicionada

El ciclo definido no da **respuesta** a los casos en que desconocemos la cantidad de datos a procesar. Existen otras estructuras iterativas que permiten una mayor flexibilidad en la resolución de los problemas, sin tener que conocer previamente la cantidad de datos. Esta estructura requiere que pueda establecerse previamente una condición para poder finalizar, que, en general, es sencilla de definir.



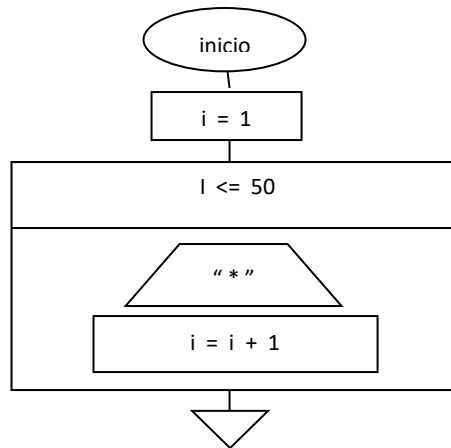
En esta estructura, primero se evalúa la condición, si es verdadera se ejecutan las sentencias que están dentro del ciclo, si es falso se pasa a la sentencia que continua, o sea se abandona el ciclo. Se debe tener precaución en no quedar atrapado dentro del ciclo, para lo cual dentro de las sentencias a repetir se debe incluir alguna que altere la condición de iteración.

Su codificación en lenguaje C es:

```
while ( condición )
{
    Sentencia 1;
    Sentencia 2;
}
```

Las llaves son opcionales de haber una sola sentencia.

Ejemplo: Confeccionar un programa para imprimir una línea con 50 asteriscos.

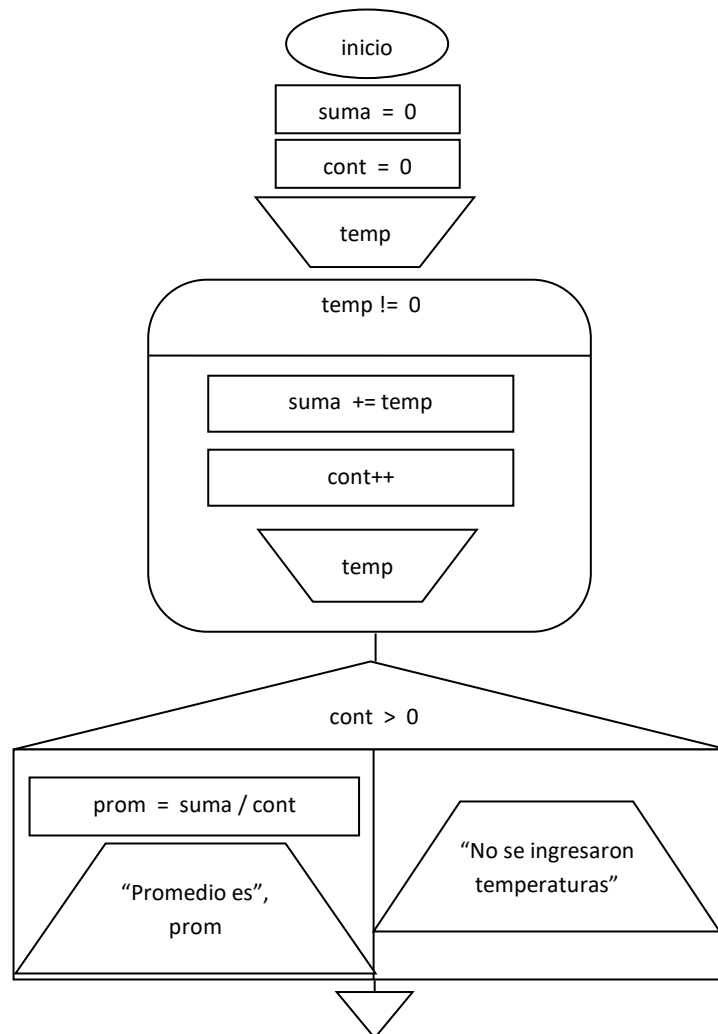


```

#include <stdio.h>
int main()
{
    int i = 1;
    while(i <= 50)
    {
        printf("*");
        i++;
    }
    return 0;
}
  
```

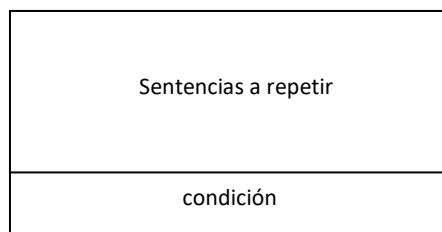
Podríamos resolverlo con una estructura de iteración definida, pero también podemos utilizar una estructura de iteración condicionada como se muestra en el ejemplo.

Ejemplo: Confeccionar un programa para ingresar diversos valores de temperatura hasta que aparezca uno igual a cero. Calcular e informar el promedio de los valores ingresados, sin considerar el cero.



```
#include <stdio.h>
int main()
{
    int cont;
    float temp, suma, prom;
    suma = 0;
    cont = 0;
    printf("Ingrese una temperatura (0 fin): ");
    scanf("%f",&temp);
    while(temp != 0)
    {
        suma += temp;
        cont ++;
        printf("Ingrese una temperatura (0 fin):");
        scanf("%f",&temp);
    }
    if(cont > 0)
    {
        prom = suma/cont;
        printf("El promedio de las temperaturas es: %.2f",prom);
    }
    else
        printf("No se ingresaron temperaturas (la primera fue cero)");
    return 0;
}
```

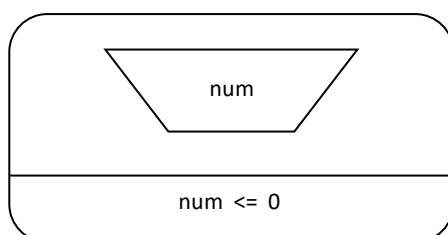
Existe otra estructura de iteración condicionada, similar en gran medida al “while” denominada do while.



Se van a ejecutar las sentencias SIEMPRE UNA VEZ y todas las otras veces que sean necesarias mientras la condición sea “verdadera”.

Es útil cuando es necesario ejecutar una serie de sentencias por lo menos una vez. Es un ciclo (1 – n), a diferencia con el ciclo while que es (0 - n). Una aplicación puede ser para controlar el ingreso de los datos.

Ejemplo: Solicitar el ingreso de un valor que debe ser positivo, reiterar la solicitud mientras no se ingrese un valor positivo.



```
do
{
    printf("Ingrese un entero positivo para
    continuar: ");
    scanf("%d",&num);
} while(num <= 0);
```

Comparación entre while y do/while

- a) En ambas estructuras la repetición se produce cuando la condición es verdadera.
- b) En el while la condición se evalúa “antes” de ejecutar el bucle. Ciclo 0 –N por lo tanto las variables de la condición deben tener valor asignado antes del ciclo.
- c) En el do/while la condición se evalúa “después” de ejecutar una vez el cuerpo del bucle. Ciclo 1 – N. Por lo tanto las variables de la condición pueden asignarse y modificarse directamente dentro del ciclo.
- d) Las variables que figuren en la condición deben modificar su valor dentro del ciclo sino una vez dentro nunca se saldrá del mismo.



Universidad Nacional de la Matanza

Programación Inicial

UNIDAD 6. FUNCIONES

INDICE

1.	PROGRAMACIÓN MODULAR	2
2.	FUNCIONES	2
3.	PARÁMETROS Y VALOR DE RETORNO	3
4.	DECLARACION DE VARIABLES - GLOBALES Y LOCALES	6
5.	CODIFICACIÓN DE LAS FUNCIONES	6
5.1	DECLARACIÓN DE UNA FUNCIÓN – PROTOTIPO	7
5.2	LLAMADA A UNA FUNCIÓN.	8
5.3	DESARROLLO DE UNA FUNCIÓN	8
6.	METODOLOGÍAS PARA LA PROGRAMACIÓN MODULAR	9

UNIDAD 6.

Funciones

OBJETIVOS: Realizar programas partiendo de pequeñas partes conocidas, denominadas "subprogramas", y que en C se conocen como "Funciones". Crear nuevas funciones. Comprender y aplicar los mecanismos para el intercambio de información entre las funciones.

1. Programación modular

Para simplificar la resolución de los problemas, basado en la programación estructurada, es posible la división del problema PRINCIPAL en pequeños problemas o sub-problemas de fácil solución y mantenimiento. Cada sub-problema se transformará en una FUNCION.

La solución de los problemas se efectúan desarrollando un Algoritmo, el cual estará compuesto por un tronco o "Algoritmo principal", acompañado de una serie de sub-algoritmos, que unidos adecuadamente resuelven el problema.

El algoritmo principal genera el programa principal, que en C es una función llamada "main". Los sub-algoritmos serán funciones con nombres propios, definidos por el programador.

En general un programa, va a constar de un "tronco" ó programa principal y una serie de subprogramas vinculados con éste. Cuando se ejecuta el programa que contiene funciones, se transfiere en cada llamada, el control al subprograma, se ejecutan sus instrucciones y retorna el control al programa llamador, es como si estuviese intercalado dentro del programa principal.

Las funciones constituyen una herramienta esencial en la programación modular. Por un lado, permiten desarrollar sólo una vez procedimientos (rutinas o subrutinas) que son de utilización frecuente o que se utilizan en diferentes lugares de un mismo programa; por otro lado, permiten utilizar mecanismos de abstracción que facilitan la construcción de un programa concentrándose en los aspectos más relevantes y despreocupándose de detalles que pueden ser encarados en otra etapa del diseño.

Una función es, básicamente, un subprograma; por lo tanto, tiene una estructura similar a la de cualquier programa:

Entradas → Proceso → Salida

Sólo que interactúa (se comunica) con otra función o directamente con el programa principal; dicha comunicación se establece a través de argumentos (parámetros) que conforman los datos (entradas) y resultados (salidas). Como cualquier programa, utilizan variables de trabajo denominadas variables locales proveyendo mecanismos de protección que hacen a éstas inaccesibles desde cualquier otro ámbito fuera de la misma función.

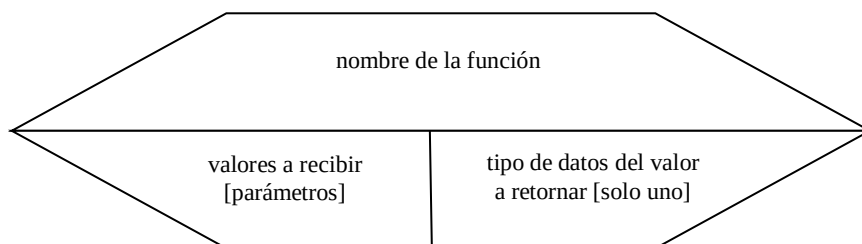
2. Funciones

Una función es una colección de declaraciones y sentencias que realizan una tarea específica, tiene cuatro componentes.

1. El nombre
2. La información que recibe o toma a través de los parámetros para realizar la tarea. (opcional)
3. Las sentencias que realizan la tarea
4. El valor que devuelve cuando termina su tarea (opcional).

Cada función se diagrama por separado, como si fuese otro programa. En la codificación serán incluidas a continuación de la función principal llamada main().

Para la primera línea de la “definición de la función” se utiliza un gráfico especial, que es el siguiente



Para las llamadas a las funciones se utiliza el gráfico que corresponde a una instrucción donde se invocará a la función por su nombre y se le enviarán los datos para utilizarla si es que los requiere.

Tanto los parámetros como el valor de retorno son opcionales, por lo tanto, podemos tener las siguientes combinaciones:

1. Funciones que NO reciben parámetros y NO retornan ningún valor: por ejemplo, una función que muestra un mensaje fijo, como un encabezado o título del programa.
2. Funciones que reciben parámetros y NO retornan ningún valor: por ejemplo, alguna función que realice algún cálculo y muestre el resultado dentro de la función. También se utiliza este tipo de funciones para los siguientes temas como arrays y archivos.
3. Funciones que NO reciben parámetros y retornan un valor: por ejemplo, para solicitar el ingreso de un dato, validarlo y retornar el número ingresado al programa principal.
4. Funciones que reciben parámetros y retornan un valor: por ejemplo, para funciones que realicen cálculos pero no lo muestren sino que retorne el resultado para ser utilizado en el programa principal.

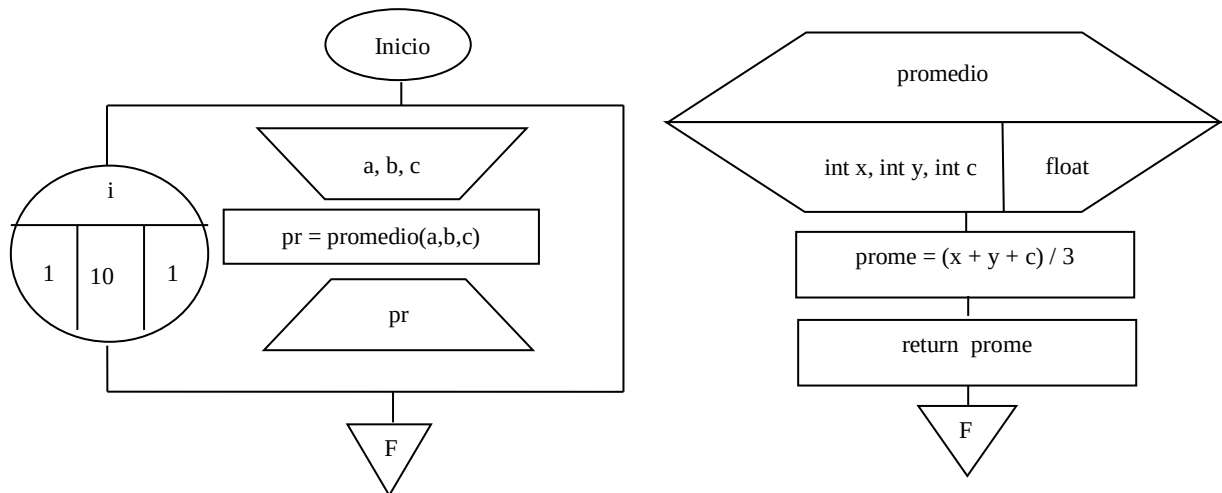
VENTAJAS DEL USO DE FUNCIONES

- Facilita la programación porque divide el problema en subproblemas.
- Simplifica la puesta a punto, ya que puedo corregir o cambiar una función sin tocar el resto del programa. Idem los futuros cambios.
- Permite trabajar simultáneamente a varios programadores.
- Permite la reutilización de estas funciones sin volver a programarlas, solo se utilizan.
- Ahorra memoria, reduce el tamaño ocupado por el programa cuando uso la misma función varias veces.

3. Parámetros y valor de retorno

A una función opcionalmente se le pueden enviar datos, esos datos son recibidos en la función para ser utilizados dentro de ella. Al especificar la función se detallan esos datos que recibe con su tipo de dato y un identificador que será el nombre de la variable LOCAL a la función donde se guardará el dato recibido. Estos datos que la función recibe se denominan parámetros formales. Veamos el siguiente ejemplo:

Ejemplo: Confeccionar un programa que solicite el ingreso de 10 ternas de valores reales y para cada una de las ternas calcule e informe el promedio de sus valores. Para el cálculo del promedio confeccionar y utilizar una función.

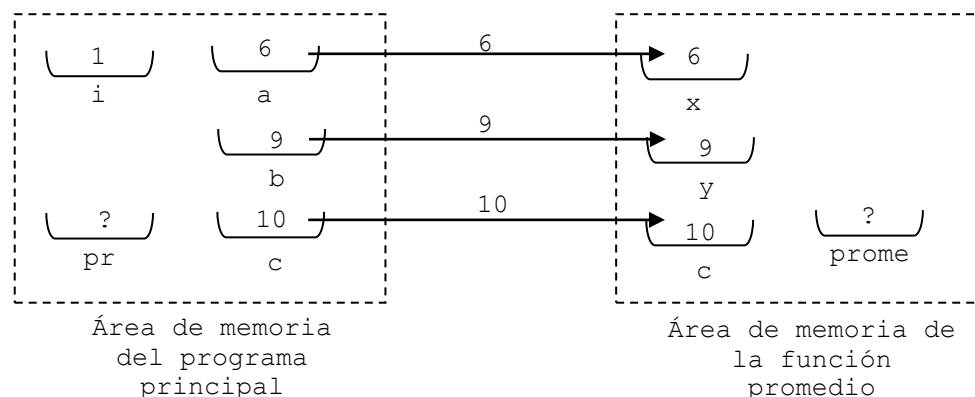


Como se mencionó anteriormente se realizan dos diagramas separados, un diagrama para el programa principal y otro diagrama para el desarrollo de la función. En este ejemplo se ingresan 3 números reales en el programa principal que se almacenan en las variables *a*, *b* y *c*. Luego, se invoca la función *promedio* enviándole como parámetro los tres valores recién leídos. Los parámetros trabajan por posición, es decir, que el primer valor que se indique en la llamada a la función va a ser guardado en una variable llamada *x* dentro de la memoria de la función *promedio*, el segundo la variable *y*, y el tercero en la variable *c*.

Las funciones trabajan con direcciones de memoria totalmente separadas a las del programa principal, al invocar a una función lo que se envía es un valor, es decir, el contenido de una variable y no la variable. Este valor es almacenado en un área distinta de memoria propia de la función. Cada vez que se invoca a una función se reserva un área de memoria separada distinta a la que se utilizó en otras llamadas. Además, al ser un área distinta de memoria es posible utilizar el mismo identificador que en el programa principal pero teniendo en cuenta que aún así es una variable diferente.

Suponga que el usuario ingresa los valores 6, 9 y 10 en el programa principal en las variables *a*, *b* y *c* respectivamente. Al invocar la función lo que se envían son esos números que se COPIAN en los parámetros de la función *promedio*, por cada parámetro se crea automáticamente una variable local a la función por lo que no deben volver a definirse. En la figura 1 puede ver un esquema de las áreas de memoria del programa principal y de la función *promedio*.

Figura 1: Esquema de memoria y envío de parámetros del programa principal a una función



Cada casillero representa un espacio de memoria con un identificador que es el nombre de la variable definida. El programa principal tiene 5 variables: k, utilizada como índice del ciclo for, pr para guardar el resultado del promedio y los tres valores ingresados por el usuario a, b y c. La primera vez que se ejecute el ciclo for k toma el valor 1 y a, b y c tomarán los valores ingresados por el usuario mediante el teclado. La variable pr, no tiene valor asignado aún por lo que no se sabe qué valor tiene, tiene un valor aleatorio que estaba ya en la memoria, pero no se sabe cuál es. Como no se inicializa la variable se dice que trae “basura” de memoria porque es un valor anterior que no tiene ningún sentido para el programa.

La función promedio tiene 4 variables ya que se crea automáticamente una variable para almacenar cada uno de los parámetros formales y además se declara dentro de la función una variable local llamada prome. Puede notar que la función promedio tiene una variable c al igual que el programa principal, pero son variables distintas porque están en áreas de memoria separadas. Dentro de la función solo se puede acceder a sus variables locales, por lo tanto, no puede acceder a las variables definidas en el main o en otras funciones. Cada función trabaja con sus propios datos lo que facilita que pueda reutilizarse.

Al invocar a la función desde el programa principal los valores de las variables a, b y c viajan hacia la función, se copian. Esos valores se denominan argumentos de la función y son recibidos y almacenados en las variables definidas automáticamente por los parámetros formales. Desde el programa principal como argumentos de una función se pueden enviar:

- Variables (se enviará el contenido de la misma)
- Constantes
- Operaciones (se resuelve la operación y se envía el resultado)
- Funciones que retornen un valor (al invocar a una función esta si retorna un valor, puede ser utilizado como parámetro de otra función y lo que se envía es el valor retornado por dicha función)

Dentro de la función se calcula el promedio y el resultado se guarda en la variable local de la función prome. Luego se utiliza un comando que permite retornar un valor al programa principal, el comando return. Este comando retorna un valor cuyo tipo de dato debe coincidir con el especificado como retorno en la función. En este caso la función debe retornar un valor float, por lo tanto, la variable local prome debe ser también float para poder ser retornada. El esquema de la figura 2 muestra el estado de la memoria al retornar el valor.

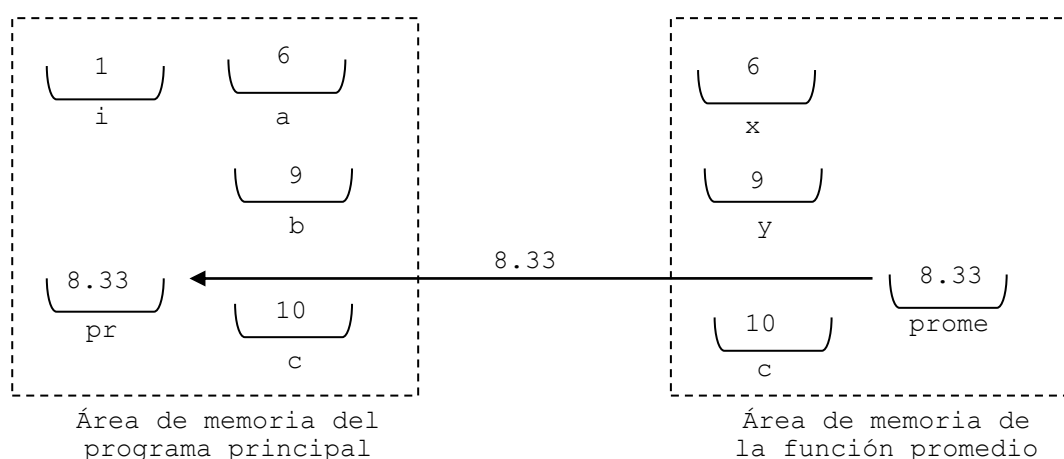


Figura 2: Esquema de memoria y envío del valor de retorno desde la función al programa principal

El valor de la variable prome retorna al programa principal como resultado de la función. Dicho resultado es almacenado en la variable pr. Si no se le asigna el resultado de la función a la variable con la instrucción `pr = promedio(a,b,c)` el valor de retorno se pierde, por lo tanto si se desea guardar el resultado de una función dicho resultado debe asignarse a una variable local desde donde se la invoca. Luego de retornar el valor la función finaliza, es decir, que siempre al llegar a

una instrucción `return` se sale de la función por lo que es recomendable usar solo una instrucción de este tipo al final de la misma para evitar confusiones. Al salir toda la memoria asignada a la función se pierde, ya no es accesible, por lo tanto, todo lo declarado y calculado dentro de la función se pierde.

4. Declaración de variables - globales y locales

Existen 2 lugares en el programa donde se pueden definir variables:

- a) Antes del bloque principal, éstas serán variables GLOBALES. Su ámbito de incumbencia es todo el programa, incluso todas las funciones que éste contenga.

Esta definición debe evitarse debido a que puede complicar el programa ya que si se altera por error una variable global dentro de una función, dicho cambio se reflejará en todo el programa. Además, complica el uso de funciones realizadas por otras personas, ya que debería analizar los nombres y uso de todas las variables.

- b) Dentro del `main()` (es una función más) ó dentro de cualquier función, estas variables se llaman LOCALES, porque su ámbito de incumbencia queda reducido solo a la función donde se la define.

Las variables LOCALES ocupan memoria solo cuando se utiliza la función, por lo cual no es posible conservar datos en una variable local, entre 2 ejecuciones sucesivas de la misma función. El uso de las variables LOCALES, aparte de ahorrar memoria permite escribir FUNCIONES AUTOSUFICIENTES, que pueden usarse en otros programas sin más que copiarlas, evitando errores en el uso de las variables. Como ejemplo puede ver el caso de las Funciones de Biblioteca.

Algunas consideraciones:

- Pueden utilizarse variables con igual nombre en distintas funciones: esto puede realizarse, ya que la función busca primero a la variable a usar dentro de las variables definidas como locales, si no la encuentra pasa a buscarla dentro de las variables globales definidas en el programa llamador. Esta es la causa por la cual puedo usar el mismo nombre para una variable local de una función que para una global, o una definida en el `main()`.
- En general, si se usan los mismos nombres se debe tener cuidado en no confundirse.
- La llamada a una función se puede realizar, en general, desde cualquier sentencia.
- La utilización más eficiente de las funciones es haciendo uso de los PARAMETROS, que son realmente quienes establecen la comunicación entre el programa principal (`main`) y las funciones, usando variables locales.

5. Codificación de las funciones

La estructura de una función es similar a la de la función principal `main()`, salvo que comienza su cabecera con "su nombre" en lugar de la palabra "`main`".

En "tres" lugares del programa se debe mencionar a cada función que se utiliza:

- a) Declaración del prototipo (solo la primer línea, con los parámetros y sus tipos de datos, ó solo los tipos de datos), al comienzo del programa, luego de las directivas al preprocesador y antes de la línea `main()`.
- b) Uso de la función (llamada a la misma) dentro del programa, en el lugar adecuado. Puede usarse en el programa principal o dentro de otra función.
- c) Desarrollo de la función completa, con su cabecera y sentencias que la componen. Este desarrollo se realiza debajo de la función `main()`.

```

#directivas al preprocesador (include y define)
declaración de variables "globales";
(a) prototipo → {tipo/void} nombre-función ([parámetros(solo el tipo) ] ) ;
void main()
{
    - declaración de variables locales del main;
    - sentencias;
(b) uso → [var =] nombre-función ([argumentos]);
}
(c) desarrollo → {tipo /void} nombre-función ([parámetros (tipo y nombre)])
{
    - declaración variables locales de la función;
    - sentencias;
    [return expresión;]
}

```

A continuación, se muestra la codificación del ejemplo presentado anteriormente para calcular el promedio de 10 ternas de valores ingresadas por teclado mediante una función.

```

#include <stdio.h>
float promedio (float,float ,float );//prototipo de la función
int main()
{
    float a, b, c, pr;
    int k;
    for (k=1;k<=10;k++)
    {
        printf("\nIngrese una terna de valores : ");
        scanf("%f%f%f",&a,&b,&c);
        pr = promedio (a,b,c); //llamada a la función
        printf ("\n Promedio = %6.2f", pr);
    }
    return 0;
}

float promedio(float x, float y, float c) //desarrollo de la función
{
    float prome;
    prome = (x+y+c)/3;
    return (prome);
}

```

5.1 Declaración de una función – Prototipo

Esta declaración, que comprende solo la cabecera de la función y que se hace previa al comienzo del main(), le permite al compilador conocer el nombre de la misma, conocer los parámetros y sus tipos de datos y luego poder chequear a éstos con los de la llamada y también el tipo del resultado. No se define el cuerpo de la función.

Una función no puede ser llamada si no se escribió su prototipo, salvo que la función sea declarada completa antes del main(), en dicho caso no es necesario el prototipo.

Si la función no recibe parámetros se pueden dejar los paréntesis vacíos o escribir la palabra void, que es el tipo de dato vacío, es decir, indica que no recibe nada. En el valor de retorno siempre hay que especificar un tipo de dato, si la función no retorna nada se DEBE especificar la palabra void. Si se olvida escribir un tipo de dato por defecto el compilador asume que la función retornará un int (entero). Es recomendable escribir siempre el tipo de dato de retorno para mayor claridad. Si se asigna un identificador a los parámetros en el prototipo los mismos serán ignorados.

```

[void ó tipo] nombre ([parámetros formales con tipos ó solo el tipo de
datos] );

```

Ejemplos :

```
int suma (int, int, int);  
void CalcularYMostrarPromedio(float, int);  
char IngresarVocal();  
int IngresaNumeroPar(void);  
void MostraTitulo();
```

Orden de los prototipos: Si hay varias funciones y una función llama a otra debe ponerse primero el prototipo de la función más interna. Es decir, que: si la función "a" llama a la "b", debe definirse previamente la "b". Pero si a su vez la función "b" llama a una función "c" entonces el primer prototipo que se debe escribir es el de la función "c", luego el de la "b", y por último, el de la "a". El desarrollo de las funciones puede hacerse en cualquier orden si los prototipos fueron escritos en la parte superior correctamente.

5.2 Llamada a una función.

Es similar al llamado a una función de biblioteca. La llamada significa la ejecución de la función.

Tiene la forma general:

```
[variable = ] nombre ( [ argumentos ] ) ;
```

Por ejemplo, en esta llamada a la función potencia:

```
raiz = potencia (3.5 , a);
```

La variable raiz representa el lugar donde se guardará el resultado traído de la función; potencia es el nombre asignado a la función y los valores 3.5 y a, son los argumentos y son los valores que se envían a la función para ser asignados a sus parámetros formales. Ambas listas de parámetros deben coincidir en cantidad y tipo.

Ejemplos:

```
sum = suma (n1, n2, n3);  
sum = suma (n1, 10, n3);  
CalcularYMostrarPromedio(suma, 5);  
CalcularYMostrarPromedio(suma, cant);  
CalcularYMostrarPromedio(100, 5);  
vocal = IngresarVocal();  
num= IngresaNumeroPar();  
MostraTitulo();
```

5.3 Desarrollo de una función

```
[void ó tipo] nombre ([parámetros formales con tipos de datos])  
{  
    [ declaración de variables locales ]  
    sentencias;  
    [ return expresión ; ]  
}
```

Tipo: si no se indica la palabra void significa que la función retorna UN VALOR, que puede ser de cualquier tipo, menos array (esto se verá en unidades posteriores). Por omisión es int

Nombre: es un identificador del nombre de la función

Parámetros formales: Los parámetros formales son las variables que reciben los valores pasados en la llamada a la función. Si no se reciben valores se deja vacía o se coloca la palabra void. Por cada parámetro formal se debe especificar el tipo de dato y el nombre.

Declaraciones: si se declaran variables, éstas serán LOCALES de la función.

Sentencias: Estas son las instrucciones que forman el cuerpo de la función

Valor retornado por una función: Cada función puede retornar un solo valor cuyo tipo se indica en la cabecera de la función. Este valor a retornar se debe indicar en la sentencia return puede ser una constante una variable o una expresión.

Ejemplos:

```
return 1;      //retorna siempre el valor 1
return a+b;    //retorna la suma del contenido de las variables a y b
return suma;   //retorna el contenido de la variable suma
```

6. Metodologías para la programación modular

La programación modular permite separar el problema en sub-problemas menores. Es decir, que parte de la lógica se puede delegar para que sea resuelta por una función lo que permite encarar el diseño de un algoritmo de dos formas distintas:

Botton-Up (ascendente)

Esta metodología se basa en comenzar a diseñar el algoritmo desarrollando primero las funciones o viendo que funciones se tienen para luego con esas funciones armar el diseño del programa principal. Es decir, que parte de lo particular para luego generar una solución al problema.

Top-Down (descendente)

Se realiza un diseño general del sistema centrándose en la lógica del programa principal, delegando algunas tareas en funciones que serán desarrolladas luego. Es decir, que primero se hace un diseño general y luego se va a lo particular especificando las partes de la solución que falten.



Universidad Nacional de la Matanza

Programación Inicial

UNIDAD 7. INTRODUCCION A VECTORES

INDICE

1. INTRODUCCIÓN	2
2. VECTORES.....	2
3. INICIALIZACIÓN DE VECTORES	6
4. ARREGLOS COMO PARÁMETROS DE FUNCIONES	7
5. VECTORES PARALELOS.....	11

UNIDAD 7

Introducción a Vectores

OBJETIVOS: Tener un primer acercamiento a la utilización de vectores para almacenar y procesar datos.

1. Introducción

Un array es un conjunto finito de elementos del mismo tipo de dato, almacenados en posiciones consecutivas de memoria. Es una forma de declarar muchas variables del mismo tipo que pueden ser referenciadas por un nombre común y subíndices haciendo que el acceso a cada una de las variables pueda generalizarse mediante estructuras repetitivas.

El tipo de dato de los elementos se denomina tipo base del arreglo. Los arrays en el lenguaje C pueden ser de una o más dimensiones. Dentro de esta materia veremos los arrays de una dimensión llamados vectores.

El tamaño de los arrays debe ser definido al escribir el programa, es decir, que se debe conocer o estimar la cantidad máxima de elementos que va a necesitar almacenar para así dar un tamaño adecuado a los arrays.

En el lenguaje C los arrays se definen de forma similar a una variable poniendo el tipo y el identificador, pero se agrega luego del mismo, entre corchetes, la cantidad de elementos que contendrá el array en cada una de sus dimensiones. Por ejemplo:

```
int a[10]; //define un array de una dimensión (vector) que puede
almacenar 10 variables enteras
```

Las variables del tipo array en el lenguaje C en realidad guardan una dirección de memoria. Esa dirección de memoria corresponde a la dirección del primer elemento del conjunto de datos definido. Luego, mediante los subíndices se calcula la dirección del elemento puntual al que se quiere acceder. En el lenguaje C, las variables que guardan direcciones de memorias se denominan punteros, porque al contener una dirección de memoria pueden “apuntar” a otra variable, es decir guardar la dirección, la referencia de donde se encuentra otra variable. El tema de punteros está fuera del alcance de esta materia. Solo debe saber entonces que el identificador del vector guarda la dirección de memoria de inicio del mismo y que es un puntero denominado puntero estático, debido a que esa dirección no se puede cambiar una vez definida. Esto dará un comportamiento particular cuando se envíe un array como parámetro de función como se verá más adelante.

2. Vectores

Los vectores son arrays de una dimensión en el cual cada elemento se identifica con el nombre del conjunto y un subíndice que determina su ubicación relativa en el mismo.

Para definir un vector se utiliza la siguiente notación:

```
tipo_de_dato_base nombre_del_vector [cantidad de elementos];
```

Donde **cantidad de elementos** es una constante que define la capacidad del vector, es decir, la cantidad máxima de elementos que puede almacenar.

En la declaración:

```
int ve [100]
```

`ve` es el nombre de un vector de 100 elementos cuyo tipo base es `int` (o sea, un vector de 100 enteros).

El acceso a cada elemento del vector se formaliza utilizando el nombre genérico (nombre_del_vector) seguido del subíndice encerrado entre corchetes. Por ejemplo, `ve[i]` hace referencia al elemento `i` del vector. Luego, la información almacenada puede recuperarse tanto en forma secuencial (asignando valores al subíndice desde 0 hasta la capacidad -1) como en forma directa con un valor arbitrario del subíndice que esté dentro de dicho rango.

El subíndice define el desplazamiento del elemento con respecto al inicio del vector; puede ser una constante, una variable, una expresión o, inclusive, una función en la medida que resulte ser un entero comprendido entre 0 y la capacidad del vector menos 1, dado que representa el desplazamiento del elemento con respecto al inicio del vector.

Ej.: `VECTOR[5]`, `VECTOR[M]`, `VECTOR[M - K + 1]`, siempre que 5, M y `M - K + 1`, respectivamente, sean un entero entre 0 (cero) y cantidad de elementos - 1.

Podemos representar gráficamente un vector `int v[12]` de la siguiente manera:

v	23	12	5	-57	0	45	1	-96	32	7	10	30
---	----	----	---	-----	---	----	---	-----	----	---	----	----

`v[0] : 23; v[4] : 0; v[7] : -96; El último elemento es v[11]: 30`

Habitualmente, no se conoce exactamente cuántos elementos han de procesarse en un determinado programa y, además, en distintas ejecuciones de dicho programa, pueden procesarse diferentes volúmenes. Por ejemplo, si se define un vector para almacenar datos sobre los alumnos de un curso es evidente que distintos cursos tienen distinta cantidad de alumnos; en esos casos, se define un vector con capacidad suficiente para almacenar la información del curso más numeroso. Este dato suele no ser preciso por lo que generalmente se sobredimensiona el array (conviene no exagerar en el sobredimensionamiento pues esto significa mantener memoria ociosa). Así, si en una universidad se manejan cursos de 50 ó 60 alumnos, podría definirse el array de 100 elementos. Aún así, es necesario controlar, cuando se agregan elementos, que no se supere esa capacidad: Si quisiéramos agregar el elemento 101, éste ocupará memoria que no está reservada para el conjunto con lo cual pueden destruirse otras variables del programa.

El lenguaje C no chequea los límites de los arreglos. Por lo tanto, es responsabilidad del programador cuidar que el programa se mantenga siempre dentro de los límites definidos.

Debe mantenerse siempre una variable donde guardar la cantidad de elementos que realmente tiene almacenado el vector.

`int v[10]; int cant_elem;`

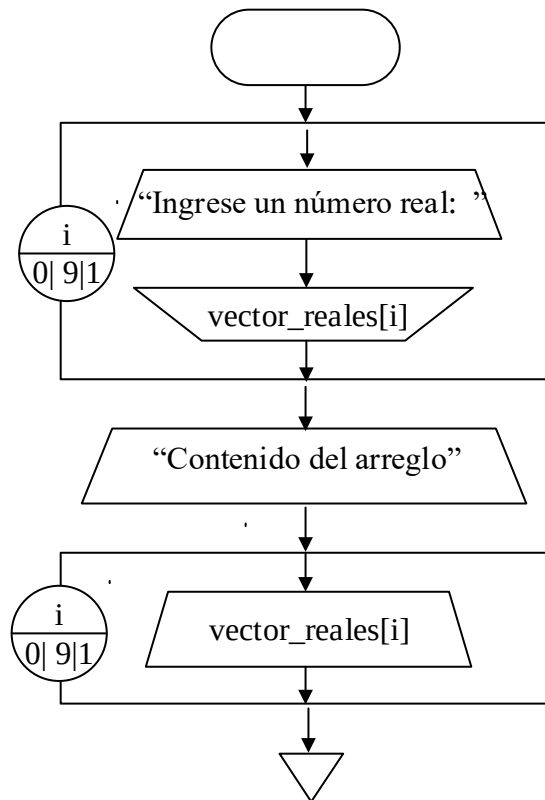
v:	23	1	0	38	15	227	4			
	0	1	2	3	4	5	6	7	8	9

`cant_elem`

Note que en las posiciones 7, 8 y 9, si bien en el gráfico no se especificó contenido alguno por ser éste irrelevante, el mismo existe: siempre hay algo en cualquier dirección de memoria y, obviamente, por ser un vector de enteros, lo que hay se interpretará como un entero. Esto significa que si se recorre el arreglo desde 0 hasta `cant_elem - 1`, siempre se encontraran valores enteros

aunque no se hayan colocado en el transcurso del programa, esos datos no los tenemos que tener en cuenta. Decimos de forma genérica que en esas posiciones hay “basura” ya que son datos no controlados.

Ejemplo 1: Generar un vector de 10 números reales leyendo dichos valores del teclado y luego mostrarlo:



Codificación en C

```

#include <stdio.h>
int main()
{
    float vector_reales[10];
    int i;
    // Lee los 10 números reales y los ubica secuencialmente en el arreglo
    for(i=0; i<=9; i++)
    {
        printf("\nIngrese un numero real: ");
        scanf("%f", &vector_reales[i]);
    }
    // Muestra el contenido del arreglo
    printf("\n Contenido del arreglo\n");
    for(i=0; i<=9; i++)
    {
        printf("%6.2f\t", vector_reales[i]);
    }
}

```

La salida de este programa es:

```

Ingrese un numero real: 23
Ingrese un numero real: 31
Ingrese un numero real: 3.25
Ingrese un numero real: .98

```

```

Ingrese un numero real: 5.5
Ingrese un numero real: 678
Ingrese un numero real: 500.60
Ingrese un numero real: 12
Ingrese un numero real: 0
Ingrese un numero real: 1

```

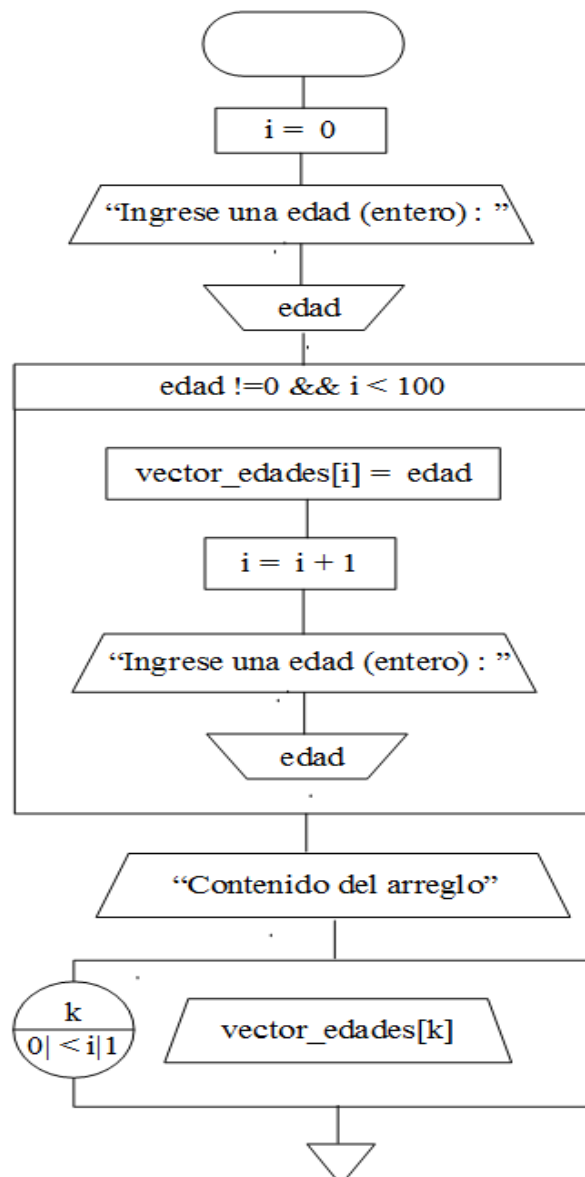
```

Contenido del arreglo
23.0 31.00 3.25 0.98 5.50 678.00 500.60 12.00 0.00 1.00

```

Ejemplo 2: Generar un vector con edades de los alumnos de un curso que se ingresan por teclado. Se estima que los cursos no tienen más de 100 alumnos. La carga finaliza con una edad igual a 0.

En este caso, no se conoce cuántos datos se van a leer; se asume que, como máximo, hay 100. Se define entonces un vector de 100 enteros pero debe verificarse en la carga de la información que no se supere dicho valor.



Nota importante: si bien puede ingresarse un elemento directamente en el arreglo, en este caso no debe procederse así pues, si se ingresaran más de 100 elementos, el elemento 101 se colocaría en la posición 100 del vector, que es memoria no reservada para el mismo. Luego, se ingresa el

dato en una variable auxiliar y, una vez confirmada la disponibilidad de espacio, se lo pone en el arreglo.

Codificación en C

```
#include <stdio.h>

int main()
{
    int vector_edades[100];
    int edad, i, k;
    i=0;
    printf("\nIngrese una edad (entero): ");
    scanf("%d", &edad);
    while (edad != 0 && i < 100)
    {
        vector_edades[i]=edad;
        i++;
        printf("\nIngrese una edad (entero): ");
        scanf("%d", &edad);
    };

    /* queda el arreglo armado con, a lo sumo, 100 edades.
       La cantidad real quedo en i */

    printf("\n Contenido del arreglo\n");
    for(k=0; k<i; k++)
    {
        printf("%5d", vector_edades[k]);
    }
    return 0;
}
```

3. Inicialización de Vectores

Al momento de declarar una variable del tipo vector es posible asignarle valores, para ello se pueden detallar cada uno de sus elementos. Por ejemplo, la instrucción:

```
int vec[5] = {2,52,100,58,18};
```

Define un vector de 5 posiciones donde en cada lugar ya tiene un número asignado, quedando el vector de la siguiente manera

Vec

2	52	100	58	18
---	----	-----	----	----

El mismo resultado se puede obtener con la siguiente instrucción:

```
int vec[] = {2,52,100,58,18};
```

Dejar vacío el tamaño del vector al declarar memoria SOLO es válido si se detallan todos sus elementos ya que de forma automática le asignará el tamaño según la cantidad de elementos especificados en este caso será un vector de 5 posiciones.

Un caso muy habitual es utilizar cada posición de un vector como un contador o un acumulador, en dicho caso y para no poner muchos ceros como datos iniciales se puede escribir:

```
int vCont[10] = {0};
```

Esta instrucción crea un vector de 10 posiciones y pone todas las posiciones del mismo en 0. De esta forma rápidamente se puede poner en 0 un vector de cualquier tamaño.

Pero supongamos ahora que necesitamos que cada posición del vector se va a utilizar para calcular una productoria (es decir el contenido se lo multiplica por otro valor y se lo vuelve a guardar). En dicho caso será necesario que todas las posiciones del vector comiencen en 1 y no en 0 ya que si lo inicializamos en 0 al multiplicarlo por si mismo siempre daría 0. Si escribimos esta instrucción:

```
int vProd[10] = {1};
```

Obtendríamos el siguiente resultado:

1	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

Es decir que pone el 1 en la primera posición, pero el resto como no está especificadas la completa con 0. Entonces para inicializar un vector de 10 posiciones todo con 1 tendríamos que escribir:

```
int vProd[10] = {1,1,1,1,1,1,1,1,1,1};
```

Esto puede traer confusiones y más cuando son vectores aún más grandes. Lo recomendable en estos casos es recorrer el vector posición a posición y asignar el número 1 en cada una de ellas. El siguiente programa declara un vector de 10 elementos y lo inicializa todo con el valor 1.

```
int main()
{
    int vProd[10],i;
    for (i=0;i<10;i++)
        v[i] = 1;
}
```

Es importante recordar que la inicialización directa del vector (usando las llaves) SOLO puede utilizarse al definir la variable, por lo tanto si en el medio del programa un vector tiene que ser puesto por ejemplo nuevamente en 0, se debe utilizar sí o sí el método de recorrerlo y asignarle a cada posición el dato deseado.

Si se define un vector de mayor cantidad de los datos que se inicializan, el resto de los datos se completan con 0. Este ejemplo:

```
int vec[10] = {1,2,1,8};
```

genera el siguiente vector:

1	2	1	8	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

4. Arreglos como parámetros de funciones

Como se mencionó anteriormente al definir una variable del tipo array (ya sea un vector o una matriz), esa variable guarda la dirección de memoria de inicio del conjunto de datos. Entonces cuando se envíe un array como parámetro de una función, lo que se envía y se copia en una variable local a la función es esa misma dirección, es decir, que dentro de la función se sigue referenciando a la misma dirección de memoria que está reservada para el conjunto de datos en el bloque desde el cual se invoca a la función. Esta particularidad hace que los cambios que se hagan sobre el array dentro de la función se van a ver reflejados desde donde se llamó a la función ya que trabaja directamente sobre la memoria del mismo.

Para definir un vector como parámetro formal de una función se indica el tipo base, el nombre y luego un par de corchetes que es lo que identifica al parámetro como un arreglo. Ejemplos:

```
int ve[], char vc[], float vf[]
```

No es necesario especificar el tamaño del vector ya que solo se copia la dirección de memoria de inicio y NO los datos del vector

Por ejemplo si en el programa principal se define un vector de 5 elementos y se desea invocar una función para que solicite los datos por teclado y los guarde en el vector, a la función se puede enviar el vector y la cantidad de datos a completar. Una posible cabecera para dicha función sería: void CargarVector (int v[], int N). Donde V es la dirección de inicio del vector donde se van a guardar los datos y N es la cantidad de elementos a solicitar. Nótese que la función no retorna ningún valor (void) ya que los datos cargados en el vector dentro de la función se verán reflejados en el programa principal porque se trabaja directamente sobre la memoria de este. La Figura 1 muestra el esquema de memoria durante el pasaje de parámetros (las direcciones de memoria son ficticias ya que su largo depende de la arquitectura del procesador de la computadora donde se ejecute el programa. El dato 5 no sale de una variable de la memoria principal ya que se envía como una constante en la llamada a la función. Además, en la función se declara una variable local i que servirá de subíndice para recorrer el vector.

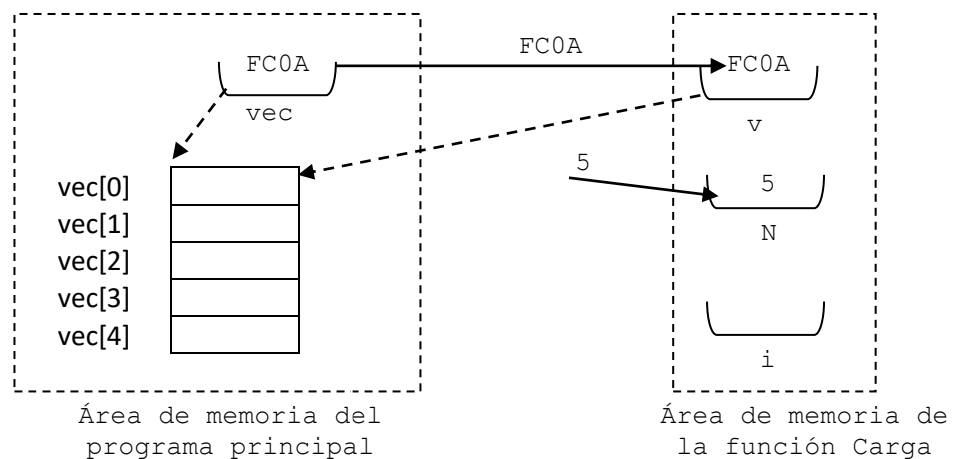
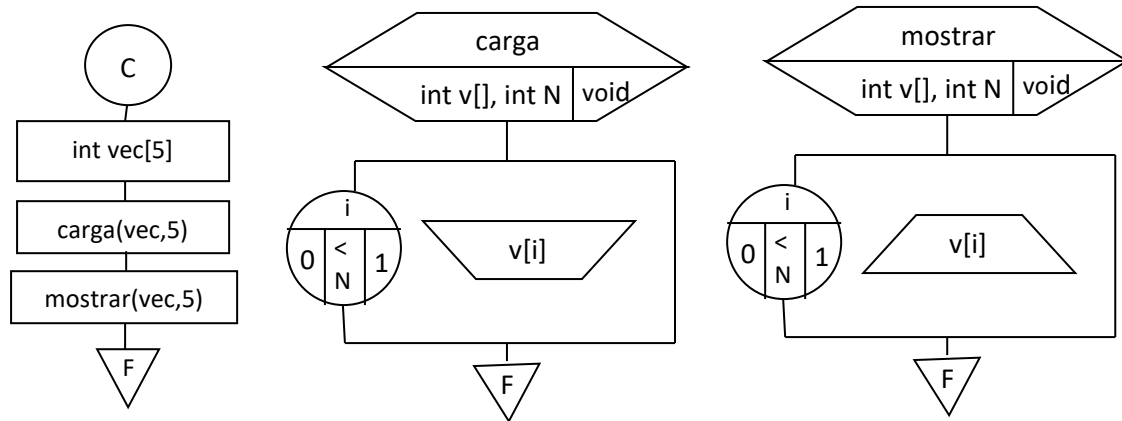


Figura 1: esquema de memoria al enviar un vector como parámetro a una función

Al enviar solo la dirección de memoria del vector, la misma función carga puede reutilizarse para cargar vectores de distinto tamaño siempre que sean del mismo tipo de dato. Esta función servirá igualmente para cargar un vector de 5, uno de 10 o uno de 1000 elementos por ejemplo, simplemente se debe declarar una variable con la capacidad suficiente en el programa principal y enviar en N la cantidad de elementos a cargar.

A continuación, se muestra un programa que, utilizando dos funciones, carga un vector de 5 elementos y los muestra por pantalla (si bien en el diagrama no es necesario declarar las variables a utilizar, es recomendable hacerlo en el caso de los arreglos para conocer la cantidad de datos disponibles).



Codificación en C:

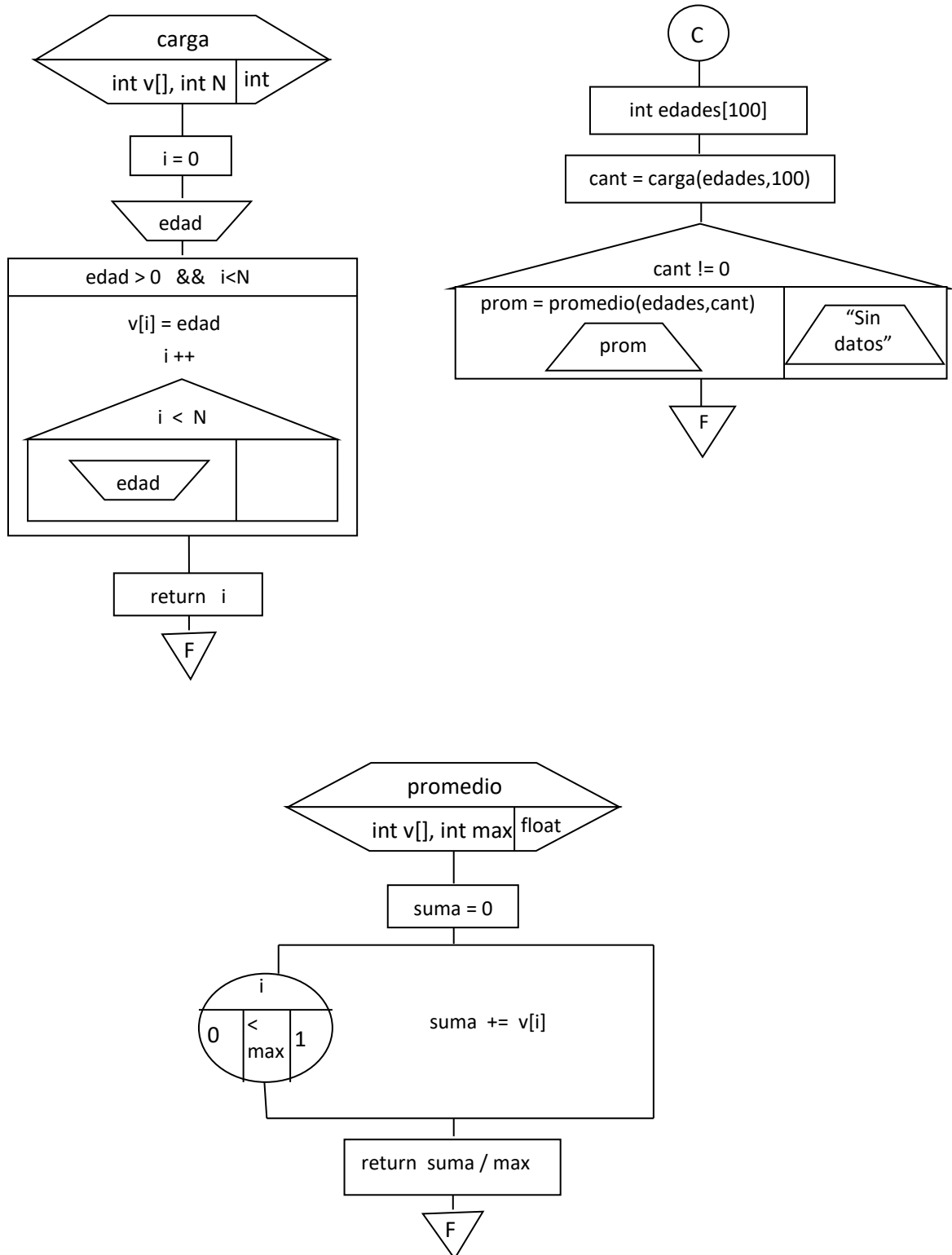
```
#include <stdio.h>
void carga(int[], int);
void mostrar(int[], int);
int main ()
{
    int vec[5];
    carga(vec,5);
    mostrar(vec,5);
    return 0;
}

void carga(int v[], int N)
{
    int i;
    for (i=0;i<N;i++)
    {
        printf("Ingrese un numero: ");
        scanf("%d",&v[i]);
    }
}

void mostrar(int v[], int N)
{
    int i;
    for (i=0;i<N;i++)
        printf("%d\n",v[i]);
}
```

Muchas veces no se sabe exactamente la cantidad de datos a ingresar por lo que se dimensiona el vector con un tamaño suficiente y se debe realizar una función de cargar que complete los datos sin pasarse del tamaño del vector, pero esta función debe además retornar la cantidad de elementos realmente ingresados.

A modo de ejemplo realizaremos el siguiente programa: ingresar las edades de los alumnos de un curso. Calcular luego el promedio de edades. No se sabe la cantidad exacta de alumnos, pero sí que no son más de 100. El ingreso de datos finaliza con una edad menor o igual a 0.



Como puede verse la función de carga retorna la cantidad de datos ingresada y luego este dato se pasa a la función que calcula el promedio para que no tome posiciones del vector que no tengan datos válidos. El promedio solo debe calcularse si se ingresaron datos, caso contrario se produciría una división por cero, generando un error de ejecución en el programa.

Dentro del ciclo repetitivo de la función de carga se agrega una condición para no solicitar una edad sino queda espacio en el vector para guardarla.

Código C:

```
#include <stdio.h>
int carga(int[], int);
float promedio (int[], int);
int main()
{
    int edades[100], cant;
    float prom;
    cant=carga(edades,100);
    if (cant!=0)
    {
        prom = promedio(edades, cant);
        printf("El promedio de edades del curso es %.2f", prom);
    }
    else
        printf("Sin Datos");
    return 0;
}

int carga(int ve[], int N)
{
    int i=0,edad;
    printf ("Ingrese la edad del alumno:");
    scanf("%d",&edad);
    while (edad>0 && i<N)
    {
        ve[i]=edad;
        i++;
        if (i<N) //evita ingresar un dato cuando no hay espacio para almacenarlo
        {
            printf ("Ingrese la edad del alumno:");
            scanf("%d",&edad);
        }
    }
    return i;
}

float promedio (int v[], int max)
{
    int suma=0,i;
    for (i=0;i<max;i++)
        suma+=v[i];
    return (float)suma/max;
}
```

5. Vectores Paralelos

Muchas veces para resolver un problema es necesario guardar más de un dato de una misma entidad y por lo tanto con un solo vector no será suficiente. Por ejemplo si se desea ingresar la lista de precios de un negocio donde por cada producto se tenga el código y su precio, será necesario definir dos vectores uno que guarde el código del producto y un segundo vector que guarde los precios. Estos vectores estarán relacionados, ya que en la posición 0 del vector de códigos se guardará el código del producto y en la posición 0 del vector de precios se guardará el precio de ese mismo producto. Es decir que los vectores están relacionados guardando en los elementos con igual subíndice información de una misma entidad. A estos vectores se los denomina vectores paralelos o apareados.

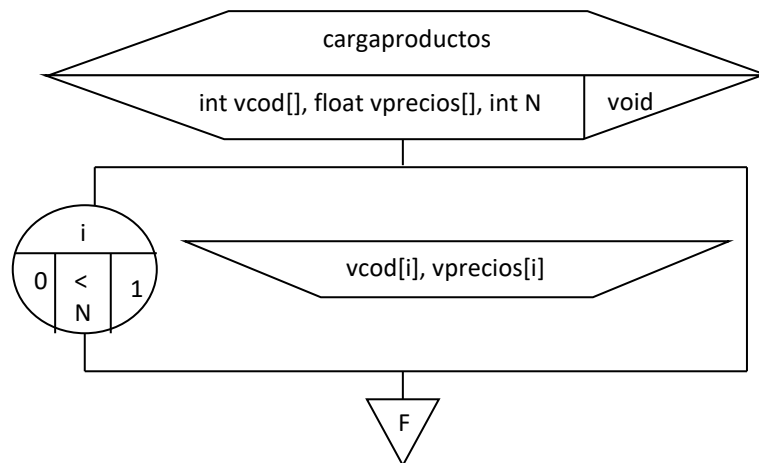
La Figura 2 muestra un ejemplo donde en vectores paralelos se cargó la información de los productos.

int vcod[4]	float vprecios[4]
1500	10.55
5878	17.35
2565	5.40
1566	20.30

Figura 2: vectores en paralelo con código y precio de productos

El producto con código 1500 tiene un precio de 10.55 mientras que el producto con código 5878 tiene un precio de 17.35 y así sucesivamente siempre la información está relacionada en ambos vectores.

Al momento de realizar una función de carga para este tipo de problema se debe solicitar la información completa de la entidad y guardarla en los vectores, es decir que se ingresa primero el código y precio del primer producto luego código y precio del segundo y NO primero todos los códigos y luego todos los precios ya que de esa forma sería confuso para el usuario. A continuación, se muestra una función de carga de la información de los productos donde se solicita el código y el precio de cada uno.



```

void cargaProductos(int vcod[], float vprecios[], int N)
{
    int i;
    for (i=0;i<N;i++)
    {
        printf("Ingrese el código de producto: ");
        scanf("%d",&vcod[i]);
        printf("Ingrese el precio del producto %d: ", vcod[i]);
        scanf("%f",&vprecios[i]);
    }
}
  
```